

2025 - 2026

GRADUATION PROJECT

NATIONAL ENGINEERING DEGREE

SPECIALTY : DATA SCIENCE

**Design of a Multi-Agent AI Pipeline
for Evidence Collection from
Illegal Streaming Sites**

By: Ahmed Arfaoui

Academic supervisor: Naadine Maazoun

Corporate Internship Supervisor: Wael Hkiri

**SOFT
STARS.**

Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Ahmed Arfaoui

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Wael Hkiri

Cachet & Signature / Stamp & Signature

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Naadine Maazoun

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and
scanned.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover
page.

Abstract

This report presents Open Web Catcher (OWC), an evidence-collection platform designed during a PFE internship at Soft Stars in a beIN SPORTS rights-protection context. The project addresses a practical problem: illegal streaming evidence often appears only after a browser renders the page, handles popups, traverses iframes, activates players, and observes media requests. The work produced two connected artifacts: an n8n landing-agent workflow used as an operational backup for dynamic landing pages, and the broader OWC platform built with FastAPI, Next.js, PostgreSQL, MCP browser tooling, Puppeteer/Chrome, specialist agents, traces, screenshots, provider enrichment, and telemetry. The evaluation focuses on reviewability rather than automatic enforcement: the system must preserve enough evidence for a human reviewer to understand what happened and what remains uncertain.

Keywords: Open Web Catcher; multi-agent system; browser automation; illegal streaming; evidence collection; stream extraction; channel detection; observability.

Résumé

Ce rapport présente Open Web Catcher (OWC), une plateforme de collecte de preuves conçue pendant un stage PFE chez Soft Stars dans un contexte de protection des droits lié à beIN SPORTS. Le projet répond à un problème concret: les preuves de streaming illégal apparaissent souvent seulement après le rendu navigateur, la gestion des popups, l'inspection des iframes, l'activation du lecteur et l'observation des requêtes média. Le travail a produit deux réalisations complémentaires: un workflow n8n utilisé comme solution de secours pour les pages dynamiques, puis la plateforme OWC construite avec FastAPI, Next.js, PostgreSQL, des outils MCP, Puppeteer/Chrome, des agents spécialisés, des traces, des captures d'écran, l'enrichissement fournisseur et la télémétrie. L'évaluation met l'accent sur la revue humaine et la traçabilité des preuves.

Mots clés: Open Web Catcher; système multi-agent; automatisation navigateur; streaming illégal; collecte de preuves; extraction de flux; détection de canaux; observabilité.

*To the people who carried me through the quiet parts of this
journey*

Dedication

To my parents, for their love, patience, and constant support.
To my family, for giving me strength when the work became heavy.
To my friends, for the encouragement, the late conversations, and
the reminders that I was not doing this alone.

This work is dedicated to you.

Acknowledgements

with gratitude

THIS project was a technical internship, but it was also a demanding personal journey. I am grateful to everyone who helped me keep going when the work became heavy, uncertain, or difficult to balance.

TO MY FAMILY

Thank you for the patience, confidence, and quiet support you gave me throughout this period. Your presence gave me the stability I needed to finish the project and prepare this report with care.

TO MY FRIENDS

Thank you for staying close during the hard moments. The conversations, encouragement, and simple reminders not to give up helped more than you may realize.

TO MY ACADEMIC SUPERVISOR

I am sincerely grateful to Naadine Maazoun for her guidance, feedback, and attention to the academic quality of this work. Her remarks helped me keep the report clear, structured, and focused on the engineering contribution.

TO MY CORPORATE SUPERVISOR AND THE SOFT STARS TEAM

I would also like to thank Wael Hkiri for his technical direction, availability, and trust during the internship. The discussions around operational constraints, browser tooling, deployment choices, and evidence review shaped the project in a very practical way. I also thank the Soft Stars team for welcoming me, sharing their experience, and giving me the opportunity to work on a concrete rights-protection problem.

AHMED ARFAOUI

2025–2026

Contents

Abstract	1
Dedication	2
Acknowledgements	3
List of Acronyms	14
1 Introduction	15
1.1 Host Organization and Internship Context	15
1.1.1 Target Users and Business Context	16
1.2 Existing Solutions and Project Positioning	17
1.3 Problem Statement	18
1.4 Internship Mission and Scope	18
1.4.1 Mission Deliverables	19
1.4.2 Scope Boundaries	19
1.5 Main Contributions	19
1.6 Report Structure	20
2 Methodology, Requirements Analysis, and Design Constraints	21
2.1 DSRM as the Methodological Backbone	21
2.1.1 Why DSRM Fits the Internship	22
2.1.2 DSRM Phases Used in the Report	22
2.1.2.1 Problem identification and motivation	23
2.1.2.2 Objectives of the solution	23
2.1.2.3 Design and development	23
2.1.2.4 Demonstration	23
2.1.2.5 Evaluation	23
2.1.2.6 Communication	24
2.2 DSRM Mapping to the Internship Work	24
2.3 Project Evolution Timeline	24
2.4 Functional Requirements	25
2.5 Design Constraints and Requirement Traceability	25
2.6 Non-Functional Requirements	27

3	Technical Background	29
3.1	Streaming Website Structures	29
3.1.1	The normal page flow	29
3.1.2	Landing pages	30
3.1.3	Hosting pages	31
3.1.4	Embedded pages	31
3.2	DOM Structures and Web Page Representation	32
3.2.1	A simplified DOM tree	32
3.2.2	DOM signals on streaming pages	33
3.3	Streaming Protocols and Embedded Players	34
3.3.1	Common embedding patterns	34
3.3.2	HLS and MPEG-DASH	34
3.3.3	Tokens, cookies, referers, and provider checks	35
3.4	Browser Automation, Scraping, and Runtime Obstacles	35
3.4.1	Static collection versus rendered browser state	36
3.4.1.1	Rendered state as evidence	36
3.4.2	Puppeteer as a browser control layer	36
3.4.3	Ads, cookies, popups, and other annoyances	36
3.4.4	Why screenshots and network logs matter	37
3.5	Agentic Evidence Collection Concepts	37
3.5.1	Agents and ReAct-style loops	37
3.5.2	Prompt engineering, tokens, and token usage	38
3.5.3	Workflow automation tools	39
3.5.4	LangChain, tools, and MCP servers	40
3.6	Chapter Summary	41
4	System Architecture	42
4.1	System Purpose and Architecture Reading Path	42
4.2	Global Runtime Architecture	43
4.3	End-to-End Agent Pipeline	45
4.4	Agent Communication and Orchestrator Handoffs	47
4.5	Agent-by-Agent Design	51
4.5.1	Orchestrator Agent	51
4.5.2	Classification Agent	52
4.5.3	Landing-Page Agent	53
4.5.4	Hosting-Page Agent	54
4.5.5	Embedded-Page Agent	55
4.5.6	Channel Detection Across Agents	56

4.5.7	Provider Analysis and Email Generation	57
4.6	Browser Tooling and Runtime Controls	58
4.6.1	Tool Families	59
4.6.1.1	Inspection and context retrieval	59
4.6.1.2	Navigation and interaction	60
4.6.1.3	Evidence and media extraction	60
4.6.1.4	Memory and site hints	60
4.6.2	Fingerprint Handling and Proxy Rotation	61
4.7	Data, Trace, and Reviewability View	62
4.8	Chapter Summary	69
5	Implementation	70
5.1	Part 1: n8n Workflow Prototype	70
5.1.1	Purpose of the n8n phase	70
5.1.2	n8n prototype-agent screenshots	71
5.1.3	Limitations of the n8n implementation	74
5.2	Part 2: Open Web Catcher Platform	76
5.2.1	FastAPI Backend	76
5.2.2	Next.js Operator Console	77
5.2.3	MCP Browser Containers and Puppeteer Tools	80
5.2.3.1	Tool registry and profile filtering	80
5.2.3.2	Rendered DOM context retrieval	81
5.2.3.3	Puppeteer actions and evidence feedback	82
5.2.4	Agents, Models, Memory, and Prompts	82
5.2.4.1	Design constraints implemented in the agent runtime	83
5.2.4.2	Prompt evolution in the implementation	83
5.2.4.3	Agent design principles	84
5.2.4.4	Model behavior, token usage, and rate limits	84
5.2.4.5	Prompt engineering and stopping behavior	84
5.2.5	PostgreSQL Database and Observability	85
5.3	Part 3: Deployment	86
5.4	Implementation Summary	88
6	Evaluation and Testing	89
6.1	Evaluation Objective and Business Context	89
6.1.1	Business Issue	89
6.1.2	Evaluation Question	90
6.1.3	Reviewable Evidence Bundle	90

6.1.4	Evidence-First Evaluation	90
6.2	Evaluation Protocol and KPI Definitions	91
6.2.1	Batch Test Design	91
6.2.1.1	Input Set and Repeated Runs	91
6.2.1.2	Manual Review After Automated Status	91
6.2.1.3	Website-Level Acceptance Rule	91
6.2.2	Failure Taxonomy	91
6.2.2.1	Why Failure Categories Matter	91
6.2.2.2	External Blockers Are Not Automatically Agent Faults	93
6.2.3	KPI Definitions	93
6.2.3.1	Rates Used in This Chapter	93
6.2.3.2	Current Batch Snapshot	94
6.3	Agent-by-Agent Behavior Evaluation	95
6.3.1	Agent Test Matrix	95
6.3.1.1	Why Agent-Level Evaluation Is Required	95
6.3.2	Classification and Landing Tests	95
6.3.2.1	Classification Tests	95
6.3.2.2	Landing Tests	96
6.3.3	Hosting Tests	96
6.3.3.1	Player Activation	96
6.3.3.2	Server Detection and Switching	96
6.3.3.3	Stream Capture Order	96
6.3.4	Embedded Tests	96
6.3.4.1	Iframe and Player Inspection	96
6.3.4.2	Embedded-Page Inaccessible Cases	97
6.3.5	Provider and Email Tests	97
6.3.5.1	Provider Intelligence Criteria	97
6.3.5.2	Email Readiness Criteria	97
6.3.6	Agent Dashboard Evidence	97
6.4	Run-Based Evidence Case Study	98
6.4.1	Selected Run Summary	98
6.4.2	Trace and Agent Activity	99
6.4.3	Landing Evidence	100
6.4.4	Hosting and Player Evidence	101
6.4.5	Stream, Provider, and Email Evidence	102
6.5	Dashboard Telemetry and Cost Review	103
6.5.1	Overview Metrics	104
6.5.2	Token and Runtime Cost	106

6.5.3	Model and Runtime Selection Rationale	106
6.6	Limitations and Interpretation	107
6.6.1	Operational Limitations	107
6.6.2	Interpretation of the Current Results	107
6.7	Chapter Conclusion	108
7	General Conclusion	109
7.1	Perspectives	110

List of Figures

1.1	Visual context for the host organization and the sports media-rights domain of the internship.	16
1.2	n8n landing-agent backup and OWC reference platform.	17
1.3	Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.	20
2.1	DSRM selection logic for an artifact-centered PFE project.	22
2.2	DSRM backbone used to structure the OWC problem, artifact, demonstration, evaluation, and communication.	22
2.3	Internship timeline from evidence needs analysis to evaluated OWC architecture.	25
2.4	Requirement traceability from the operational problem to the implemented artifact and review outputs.	26
2.5	Main design constraints and the engineering responses selected for OWC. . . .	27
3.1	Common navigation flow from event listing to hosting page and embedded player.	30
3.2	Browser-state view of landing, hosting, and embedded page roles.	30
3.3	Simplified DOM tree showing how a browser represents an HTML document. .	33
3.4	Main paths from a hosting page to media evidence.	34
3.5	Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH. . . .	35
3.6	Observe, act, and verify loop used by browser agents.	38
3.7	Prompt and token-budget composition for an evidence-driven browser agent. . .	39
3.8	General MCP pattern connecting an LLM agent to scoped browser tools. . . .	40
4.1	System component diagram for the OWC runtime.	43
4.2	Frontend console routes and API route groups.	44
4.3	End-to-end workflow from launch to persisted evidence.	45
4.4	End-to-end agent architecture at workflow scale.	46
4.5	Sequence diagram for launching, executing, persisting, and reviewing a workflow run.	46
4.6	Agent interconnection graph showing orchestrator-owned routing and specialist responsibilities.	47
4.7	Prompt compilation and memory inputs for each browser-facing specialist. . . .	49
4.8	Shared activity loop for browser-facing agents.	50
4.9	Orchestrator activity and routing decisions	51
4.10	Orchestrator-agent design card.	52

4.11	Sequence diagram for orchestrator handoff, trace recording, and route updates. .	52
4.12	Classification-agent design card.	53
4.13	Classification-agent sequence: inspect page state and return a routing decision.	53
4.14	Landing-page-agent design card.	54
4.15	Landing-agent sequence: convert listing-page evidence into downstream queues.	54
4.16	Hosting-page-agent design card.	55
4.17	Hosting-agent sequence: activate watch pages and return stream or iframe evidence.	55
4.18	Embedded-page-agent design card.	56
4.19	Embedded-agent sequence: stay in the player context and harvest final media evidence.	56
4.20	Provider-analysis and email-generation design card.	58
4.21	Profile-scoped MCP browser tooling and runtime controls used by OWC agents.	58
4.22	Global browser-tool flow.	59
4.23	Inspection and context-retrieval flow from broad page observation to scoped targets.	59
4.24	Navigation flow split between direct URL navigation and JavaScript-driven page interaction.	60
4.25	Evidence and media-extraction flow from post-action page state to reviewable artifacts.	60
4.26	Memory-tool flow showing hints as guidance that must be validated live.	61
4.27	Class diagram of the runtime result centered on PipelineResult.	64
4.28	Agent JSON contracts and orchestrator handoffs.	67
5.1	Role of the n8n workflow as a feasibility playground for browser-agent evidence collection.	71
5.2	Full n8n prototype workflow.	72
5.3	n8n classification and landing workflow screenshots.	73
5.4	n8n hosting and embedded-player workflow screenshots.	74
5.5	How n8n limitations shaped the Open Web Catcher implementation.	75
5.6	Five implementation areas of the Open Web Catcher platform.	76
5.7	OWC dashboard and live-run console screens.	78
5.8	OWC review screens for run results and provider enrichment, enlarged so tables remain readable in print.	79
5.9	OWC tool telemetry and runtime configuration screens, enlarged for printed review.	80
5.10	Browser tool execution loop through MCP and Puppeteer.	82
5.11	Prompt design contrast between an open-ended browser task and the evidence- driven agent loop used in OWC.	83
5.12	Agent reasoning loop with memory and tool feedback.	84
5.13	Summarized PostgreSQL persistence model derived from the detailed database table schema visualization.	86

5.14	Company deployment flow for the landing-agent backup.	87
6.1	Evaluation protocol used for website-batch regression testing.	91
6.2	Agents dashboard for behavior evaluation.	98
6.3	Selected run summary and status.	99
6.4	Trace rows and agent activity for the selected run.	100
6.5	Landing-page evidence for the selected run.	100
6.6	Hosting/player evidence for the selected run.	101
6.7	Additional hosting evidence for the selected run.	102
6.8	Streams and provider evidence for the selected run.	102
6.9	Provider-intelligence evidence for the selected run.	103
6.10	Overview dashboard metrics for Chapter 6.	104
6.11	Token telemetry dashboard.	106

List of Tables

1.1	Target users and business value of the evidence workflow.	16
2.1	DSRM phase mapping to project activities and outputs.	24
2.2	Functional requirements derived from the evidence-collection workflow.	25
2.3	Non-functional requirements and implementation responses.	28
3.1	Streaming-site page taxonomy and evidence role.	32
3.2	DOM signals used to understand streaming-site pages.	33
3.3	Browser-level obstacles and handling strategies.	37
4.1	Operator-facing responsibilities and the runtime records they create.	43
4.2	Global runtime boundaries used by OWC.	44
4.3	Agent and tool responsibility map.	48
4.4	Channel detection responsibilities across the specialist agents.	57
4.5	Browser runtime controls that support reliable and reviewable evidence collection.	62
4.6	Database table families used to turn an agent run into reviewable records.	63
4.7	Agent JSON contract summary used by the orchestrator.	66
4.8	Runtime data boundaries and persisted review outputs.	68
5.1	n8n prototype agent roles and implementation lessons.	74
5.2	n8n prototype limitations and engineering consequences.	75
5.3	Backend API responsibilities in OWC.	76
5.4	Current Next.js operator-console surfaces and backend dependencies.	77
5.5	Browser automation engine comparison for the MCP tool layer.	81
5.6	Puppeteer MCP tool groups and responsibilities.	81
5.7	Specialist agent objectives and assigned tool surfaces.	82
5.8	Runtime constraints and their implementation responses.	83
5.9	Model-runtime concerns handled by the agent platform.	85
5.10	Prompt engineering rules enforced by the specialist agents.	85
5.11	PostgreSQL observability records and their implementation role.	86
5.12	Landing-agent deployment scope and responsibilities.	88
6.1	Website-level acceptance criteria used during batch evaluation.	92
6.2	Failure taxonomy used to separate productive evidence, external blockers, agent/runtime failures, cancelled runs, and active runs.	92

6.3	KPI definitions used to interpret batch results without hiding partials or external blockers.	93
6.4	Current batch outcome snapshot used for Chapter 6 interpretation.	94
6.5	Agent-specific tests used during manual and batch evaluation.	95
6.6	Selected-run metadata used as the main Chapter 6 evidence case study.	99
6.7	Current evidence, provider, tool, cost, and token telemetry from the overview dashboard/API snapshot.	105
6.8	Model/runtime selection rationale used instead of unsupported model-ranking metrics.	107
6.9	Limitations that remain after evidence-first evaluation.	108

List of Acronyms

Acr.	Meaning and use in the report	Acr.	Meaning and use in the report
AI	Artificial Intelligence; model-assisted browser reasoning.	API	Application Programming Interface; FastAPI and console endpoints.
ASB	Azure Service Bus; deployed n8n landing-agent queue path.	CDN	Content Delivery Network; stream, asset, and challenge hosting layer.
CDP	Chrome DevTools Protocol; browser network/runtime observation.	DASH	Dynamic Adaptive Streaming over HTTP; adaptive streaming protocol.
DB	Database; short label for persisted run and review records.	DMCA	Digital Millennium Copyright Act; takedown-email context.
DNS	Domain Name System; host attribution and provider enrichment.	DOM	Document Object Model; rendered page tree inspected by agents.
DSRM	Design Science Research Methodology; method behind the artifact.	FR	Functional Requirement; requirement identifiers in Chapter 2.
HLS	HTTP Live Streaming; playlist protocol represented by .m3u8.	HTML	Hypertext Markup Language; static source compared with rendered state.
HTTP	Hypertext Transfer Protocol; pages, APIs, manifests, and media.	IP	Internet Protocol; network address used for attribution.
JSON	JavaScript Object Notation; agent outputs, traces, and settings.	KPI	Key Performance Indicator; success, stream yield, cost, and tool rate.
LLM	Large Language Model; reasoning runtime for specialist agents.	M3U8	UTF-8 M3U Playlist; HLS playlist extension used as evidence.
MCP	Model Context Protocol; tool interface for browser actions.	MPD	Media Presentation Description; MPEG-DASH manifest evidence.
MPEG-DASH	MPEG Dynamic Adaptive Streaming over HTTP; DASH standard name.	NFR	Non-Functional Requirement; reliability, cost, security, observability.
OCR	Optical Character Recognition; future screenshot/channel reading.	OWC	Open Web Catcher; reference platform built during the internship.
PFE	Projet de Fin d'Etudes; internship project and report context.	RDAP	Registration Data Access Protocol; ownership metadata source.
RFC	Request for Comments; standards source for HLS.	RPD	Requests Per Day; provider/model rate-limit unit.
RPM	Requests Per Minute; model and API throughput unit.	SQL	Structured Query Language; PostgreSQL query layer.
TPM	Tokens Per Minute; LLM throughput limit.	UI	User Interface; operator console screens and controls.
URL	Uniform Resource Locator; targets, candidates, streams, evidence links.	XHR	XMLHttpRequest; dynamic browser request pattern.

Introduction

This PFE report presents the work carried out at Soft Stars in the context of illegal sports-streaming evidence collection for beIN SPORTS. The project started from a concrete operational need: suspicious pages could be discovered, but the evidence needed for review was often hidden behind browser behavior.

In practice, that evidence could depend on rendered interfaces, popups, iframe players, server buttons, redirects, and short-lived media manifests. The internship focused on a clear goal: design an evidence workflow that observes the page like a browser user, preserves the trace, and returns reviewable evidence instead of only a final URL.

Chapter focus

I use this opening chapter to explain why the internship mattered, who the work served, and how the project was scoped. It distinguishes the existing company workflow, the deployed n8n landing-agent backup, and Open Web Catcher as the reference platform built for the academic study. It also defines the problem, the mission deliverables, the project boundaries, and the report structure used by the following chapters.

1.1 Host Organization and Internship Context

Soft Stars is a software company working in digital media intelligence, content protection, and technical support for rights-enforcement operations. Its work focuses on helping media organizations observe online distribution channels, identify unauthorized use of protected content, and prepare technical evidence that can be reviewed by operational, legal, or business teams.

The internship took place in a sports media-protection context connected to beIN SPORTS. In this domain, the practical problem is not only detecting that a suspicious website exists. A rights-protection workflow also needs enough evidence to answer review questions after the browser session has ended:

- what was visible on the page;
- which channel, event, or match appeared;
- which page led to the player;
- which technical host or provider was involved;
- whether the result is trustworthy enough for human review.

1.1.1 Target Users and Business Context

The project was made for a concrete rights-protection setting. For a broadcaster such as beIN SPORTS, illegal live streams weaken the value of exclusive broadcasting rights, create unauthorized alternatives during high-value events, and make enforcement difficult when the useful evidence disappears after the browser session. The business need is not only to find suspicious domains. It is also to shorten the path from a reported URL to reviewable technical evidence: page screenshots, channel or event context, stream URLs, provider information, and a trace explaining how the result was obtained.

The main target users are the teams who need to transform a suspicious page into a reviewable case. Rights-protection operators need faster triage and fewer manual browser repetitions. Technical analysts need enough runtime detail to understand why the system followed a specific page, server button, iframe, or media request. Legal and compliance reviewers need evidence that can be checked by a human before any external action is taken. Business stakeholders need visibility on whether the monitoring workflow supports the protection of licensed content and operational prioritization.

Target user group	Practical need	Project contribution
Rights-protection operators	Triage suspicious pages quickly and avoid repeating fragile manual browser steps.	Browser-guided evidence collection, screenshots, candidate URLs, and clear run status.
Technical analysts	Understand dynamic pages, popups, iframes, server buttons, redirects, and media requests.	Specialist agents, structured traces, tool-call records, and separated page-role logic.
Legal and compliance reviewers	Review evidence before escalation or takedown preparation.	Human-reviewable screenshots, stream evidence, provider enrichment, and draft email artifacts.
Business and client stakeholders	Protect the value of licensed sports content and prioritize enforcement effort.	A clearer monitoring workflow that connects suspicious pages to auditable evidence and operational reporting.

Table 1.1: Target users and business value of the evidence workflow.



Figure 1.1: Visual context for the host organization and the sports media-rights domain of the internship.

1.2 Existing Solutions and Project Positioning

Before this PFE work, the company already had an operational workflow for monitoring and processing suspicious streaming websites. That workflow remained the primary company process. It was useful for broad discovery, recurring checks, and normal cases where the page structure was stable enough for crawler-style logic.

The limitation appeared on pages where useful evidence was created only at browser runtime:

- JavaScript-rendered match cards or schedules;
- server buttons and popups that had to be handled in order;
- nested iframes and player activation steps;
- short-lived m3u8 or mpd media requests.

In this report, the company workflow means the existing Soft Stars operational monitoring and crawler-based processing path that remains the primary company work.

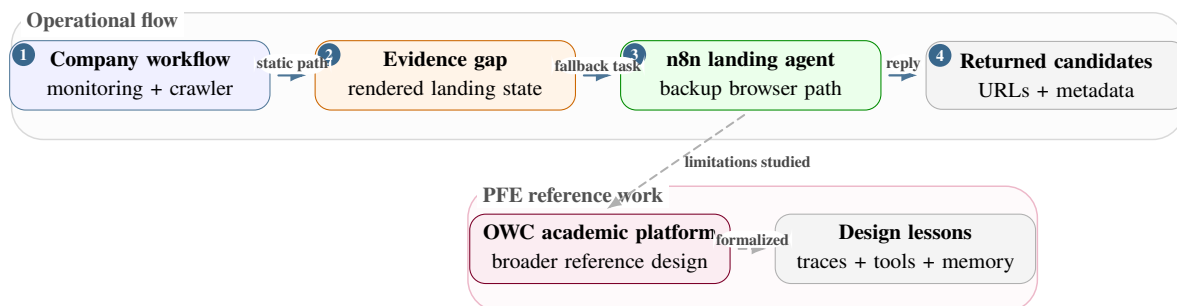


Figure 1.2: n8n landing-agent backup and OWC reference platform.

For that reason, the n8n landing agent was positioned as a backup to the company workflow, not as a replacement for it. When the primary flow could not find enough landing-page evidence, it could send a task with the target URL and execution metadata to the landing agent.

The backup agent then performed a focused browser task:

- open the rendered page;
- inspect visible match, channel, and schedule cues;
- follow candidate links when needed;
- return candidate URLs, metadata, screenshots, rejected links, and reasoning notes for review or downstream processing.

In this report, the n8n landing agent means the focused backup browser workflow used when the company workflow needs rendered landing-page reasoning.

OWC had a different role. It was the academic and engineering reference platform built during the internship to study the n8n limitations and propose a cleaner architecture around:

- specialist agents and browser tools;
- traces, screenshots, and memory;
- model telemetry, persistence, and evaluation.

The deployed company-side decision stayed focused on a landing-agent backup. OWC describes the broader design that could support a more complete future platform. In this report, OWC means the reference platform built for the academic study, not the deployed company workflow itself.

1.3 Problem Statement

Illegal streaming websites are difficult to investigate because they combine unstable content, dynamic rendering, misleading navigation, popup traps, nested frames, and short-lived media URLs. The evidence path often has several roles:

- a landing page lists matches, events, schedules, or channel cards;
- a hosting page exposes play buttons, server controls, overlays, and iframe candidates;
- an embedded page or direct player exposes the final media behavior;
- a provider-enrichment stage connects collected stream evidence to host, network, country, or abuse-contact information.

If those roles are mixed together, the system becomes hard to debug. A failed result could mean several different things:

- the page was classified incorrectly;
- a landing candidate was missed;
- a hosting server was not activated;
- an iframe was not inspected;
- the final media request was not connected to provider evidence.

The project required an architecture that separates page roles, specialist agents, browser tools, screenshots, traces, model usage, provider enrichment, and final review outputs.

1.4 Internship Mission and Scope

My mission during the internship was to study, design, and validate an evidence collection workflow that uses browser-based AI reasoning to support dynamic evidence paths when the existing company workflow needs a backup. The work produced two related artifacts.

The first artifact was an n8n landing-agent workflow. It was used to validate the idea quickly in the company environment and to provide a deployable backup path for landing pages that required rendered-page reasoning. This workflow helped test whether an agent could inspect visible page state, follow candidate links, handle page-specific obstacles, and return useful match or hosting metadata.

The second artifact was Open Web Catcher (OWC), the complete platform designed during the internship as the reference architecture for the report. The internal name was chosen because the project catches evidence from the open web through browser-visible behavior rather than through a static HTML response.

In this report, OWC refers to the whole project:

- FastAPI backend and Next.js operator console;
- PostgreSQL persistence and MCP browser tooling;
- Puppeteer/Chrome runtime and specialist agents;
- model telemetry and evidence review outputs.

1.4.1 Mission Deliverables

The internship deliverables were:

- a validated n8n landing-agent workflow used as a deployable backup to the company workflow for dynamic landing-page targets;
- an OWC architecture separating backend, frontend, database, agents, browser tools, and evidence outputs;
- browser-tool profiles for classification, landing, hosting, and embedded-player investigation;
- support for screenshots, stream candidates, provider enrichment, channel metadata, tool traces, model telemetry, and cost visibility;
- an evaluation and reporting structure based on traceability and reviewability.

1.4.2 Scope Boundaries

The project did not replace the full company enforcement process. It did not automatically send takedown messages, and it did not claim to solve every anti-bot mechanism or every illegal streaming website. The practical scope was to produce an engineering foundation that could investigate dynamic pages, preserve evidence, expose failures clearly, and support future operational integration.

1.5 Main Contributions

The main contributions of the PFE work are:

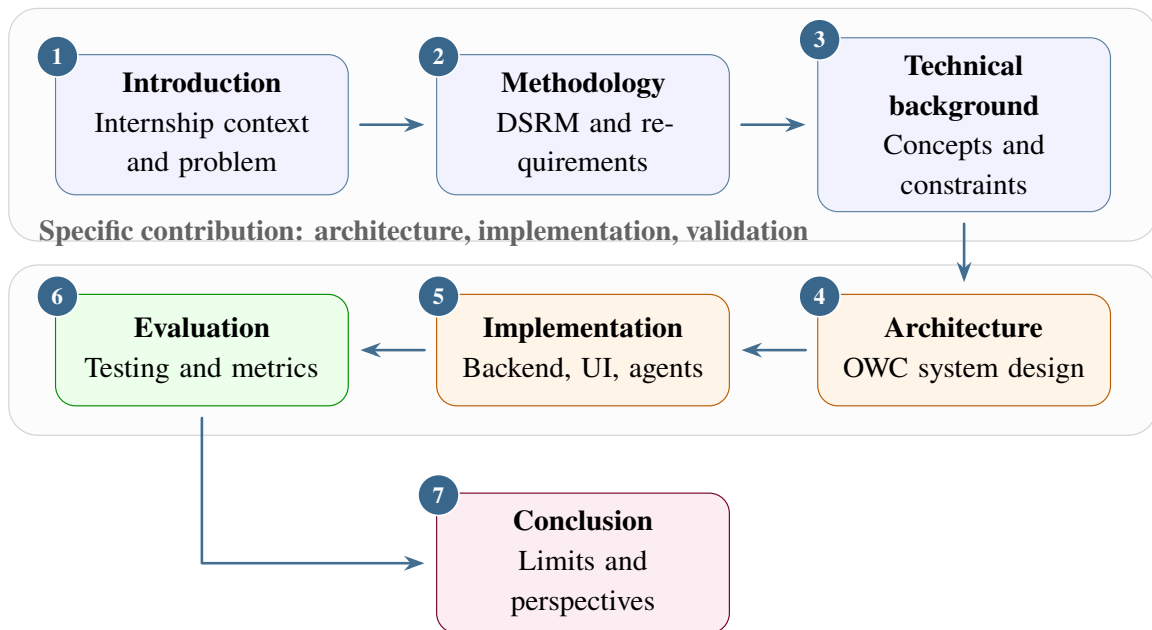
- a clear analysis of why static page collection is insufficient for many dynamic streaming-evidence paths;
- a DSRM-based artifact design centered on Open Web Catcher;
- an n8n landing-agent workflow deployed as an operational backup to the company workflow for dynamic evidence cases;
- a multi-agent architecture that separates classification, landing discovery, hosting interaction, embedded-player inspection, provider enrichment, and review output generation;
- a browser-tool layer with profile-scoped inspection, navigation, interaction, screenshot, media extraction, memory, fingerprint, and proxy controls;
- an operator console and persistence model that make traces, screenshots, model usage, tool calls, costs, channel signals, and failure reasons reviewable.

1.6 Report Structure

The report is organized as follows:

- **Chapter 2** presents DSRM as the methodology and derives the project requirements.
- **Chapter 3** introduces streaming pages, browser automation, media manifests, agent workflows, and evidence traceability.
- **Chapter 4** describes the OWC architecture: agents, browser tooling, channel detection, persistence, and reviewability.
- **Chapter 5** explains the implementation path from the n8n backup to OWC and the company deployment flow.
- **Chapter 6** presents evaluation and testing results.
- **Chapter 7** provides the general conclusion.

General framing: problem, domain, method



The report connects the internship problem, design choices, implementation, evaluation, and final perspectives.

Figure 1.3: Report reading map connecting the internship context, design, implementation, evaluation, and conclusion.

Methodology, Requirements Analysis, and Design Constraints

The internship was organized around Design Science Research Methodology (DSRM). This choice fits the nature of the work. The project was not only about observing a phenomenon from the outside or training a predictive model. It was about designing, building, demonstrating, and evaluating an engineering artifact for a concrete organizational problem.

In this report, that artifact is Open Web Catcher (OWC), an evidence-collection platform built around:

- browser tooling and specialist agents;
- trace storage, screenshots, and stream evidence;
- provider enrichment and reviewable outputs.

DSRM provides the backbone for the chapter structure that follows:

- identify and motivate the problem;
- define the solution objectives;
- design and develop the artifact;
- demonstrate and evaluate the artifact;
- communicate the work through the PFE report and diagrams.

This phase logic follows the design-science view of information-systems artifacts proposed by Hevner et al. [7] and the DSRM process described by Peffers et al. [8].

Chapter focus

Here I explain the method used to organize the project. Design Science Research Methodology connects the operational problem to the artifact that was built, then helps translate the mission into functional requirements, non-functional requirements, design constraints, and an internship timeline.

2.1 DSRM as the Methodological Backbone

DSRM was selected because it gives equal importance to the organizational problem and to the engineered artifact. The operational problem was concrete: useful evidence often appeared only after browser rendering, interaction, iframe traversal, server switching, or media activation. The PFE work needed a methodology that could justify a new artifact, not only describe a work plan.

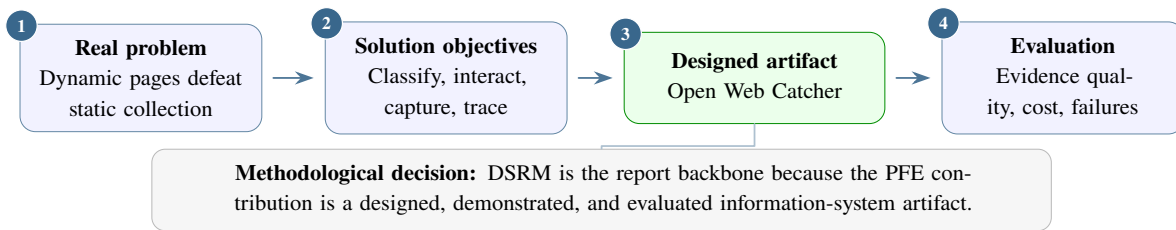


Figure 2.1: DSRM selection logic for an artifact-centered PFE project.

2.1.1 Why DSRM Fits the Internship

The project has three properties that make DSRM suitable:

- **Artifact-centered:** the contribution is a platform and workflow design, not a pure analysis.
- **Use-based evaluation:** the artifact matters only if it can inspect real pages, produce traces, capture screenshots, and return evidence.
- **Organizational relevance:** Soft Stars needed a more adaptable way to handle streaming pages that became too dynamic for static extraction.

2.1.2 DSRM Phases Used in the Report

The six DSRM phases are not treated as abstract theory. Each one corresponds to work carried out during the internship. Figure 2.2 summarizes the adopted backbone.

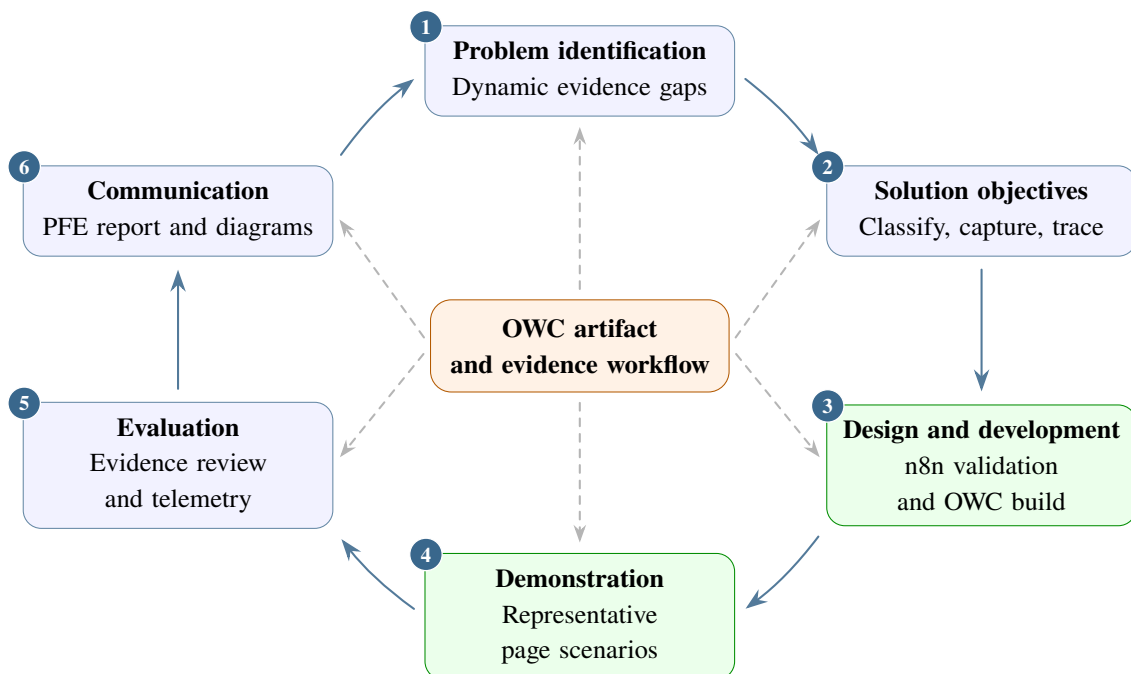


Figure 2.2: DSRM backbone used to structure the OWC problem, artifact, demonstration, evaluation, and communication.

2.1.2.1 Problem identification and motivation

The first phase established the practical problem: suspicious streaming pages could be found, but collecting reviewable evidence was difficult. The difficulty appeared in several common cases:

- pages that exposed only lists of matches or channels;
- pages that required a server-button click before the player loaded;
- pages that hid the player inside an iframe;
- pages that generated short-lived manifest URLs only after playback.

A static HTML fetch can miss these states because it does not perform the same visual and network sequence as a browser user.

2.1.2.2 Objectives of the solution

The second phase translated the problem into objectives. The artifact needed to ingest a URL, classify the page role, follow landing-to-hosting and hosting-to-embedded handoffs, activate players when necessary, collect screenshots, detect stream evidence, preserve channel and provider metadata when available, and store enough trace data for later review. These objectives became the functional and non-functional requirements listed later in this chapter.

2.1.2.3 Design and development

The third phase produced the artifacts. The n8n workflow was used first because it allowed fast operational validation of an AI-assisted landing workflow and could act as a backup path for dynamic evidence cases. OWC then formalized the same ideas in a maintainable platform with FastAPI, Next.js, PostgreSQL, MCP browser tooling, Puppeteer/Chrome, profile-scoped sessions, model telemetry, and specialist agents.

2.1.2.4 Demonstration

The fourth phase demonstrated that the artifact could operate on representative page types: landing pages, hosting pages, embedded player pages, popup-heavy pages, tokenized manifest cases, and pages where channel evidence had to be verified from visible player or screenshot context. Demonstration was not limited to a happy path; it included runs where the system returned explicit failures such as no hosting pages, no streams, or skipped downstream enrichment.

2.1.2.5 Evaluation

The fifth phase evaluated the artifact through trace inspection, tool telemetry, screenshots, token and cost records, stream yield, provider enrichment, and dashboard snapshots. The aim was not to claim that every illegal streaming page can be solved. The aim was to verify whether the artifact made the extraction process more observable, repeatable, and reviewable than a one-shot static fetch or a manually inspected page.

2.1.2.6 Communication

The final phase communicates the artifact. This report presents the problem, methodology, technical background, architecture, implementation, evaluation, and limitations. The diagrams and tables make the artifact understandable as an engineering contribution rather than reproduce internal source files.

2.2 DSRM Mapping to the Internship Work

Table 2.1 maps each DSRM phase to the work performed during the internship and to the concrete outputs used in the rest of the report.

DSRM phase	Application in this project	Main output
Problem identification and motivation	Analyze static-collection limits, dynamic page behavior, stream activation, iframe traversal, channel ambiguity, and the need for reviewable evidence.	Problem statement and motivation for OWC
Objectives of the solution	Define the extraction, traceability, screenshot, channel, provider, cost, and review requirements that the artifact must satisfy.	Functional and non-functional requirements
Design and development	Build the n8n validation workflow, then design OWC as a platform with backend, frontend, database, browser tools, agents, and observability.	n8n workflow and OWC architecture
Demonstration	Run the artifact on representative landing, hosting, embedded, popup, iframe, and tokenized-stream scenarios.	Demonstrated evidence paths and failure paths
Evaluation	Inspect traces, screenshots, stream evidence, provider enrichment, tool behavior, token usage, costs, and dashboard telemetry.	Evaluation metrics, results, and limitations
Communication	Produce the PFE report, diagrams, final PDF, and design explanation for academic and company review.	Written report and final synthesis

Table 2.1: DSRM phase mapping to project activities and outputs.

2.3 Project Evolution Timeline

The timeline follows the DSRM logic but remains concrete. The internship started with evidence needs and static-collection limits, moved to browser-agent experiments, then to the n8n landing-agent backup workflow, and finally to the OWC platform architecture and evaluation.

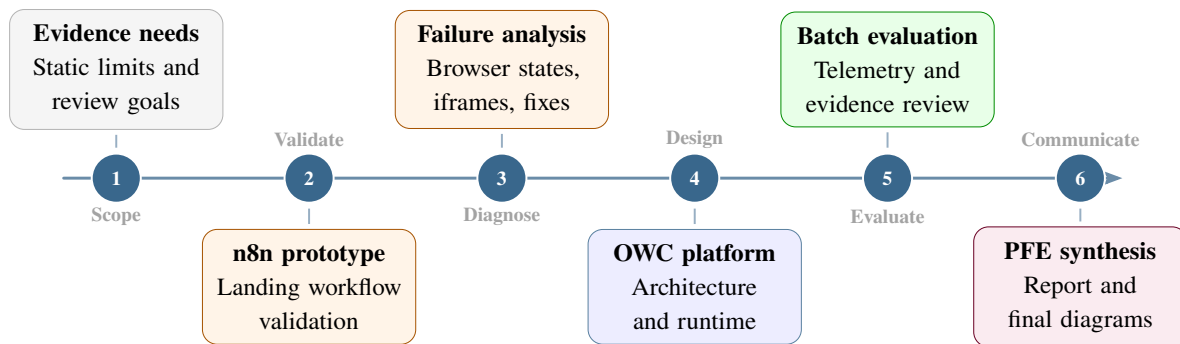


Figure 2.3: Internship timeline from evidence needs analysis to evaluated OWC architecture.

2.4 Functional Requirements

The functional requirements describe what the artifact must do to support the evidence-collection workflow.

ID	Requirement summary	Priority
FR-1	URL ingestion and run lifecycle management	Critical
FR-2	Page-type classification using rendered browser evidence	Critical
FR-3	Orchestrator handoff from classification to landing, hosting, or embedded agents	Critical
FR-4	Landing-page discovery of match, event, channel, and hosting candidates	High
FR-5	Hosting-page interaction with server/source controls, overlays, and play buttons	Critical
FR-6	Embedded-player inspection through frame trees, player state, and media signals	Critical
FR-7	Screenshot capture at important evidence and failure points	High
FR-8	Stream evidence extraction for manifests, media URLs, and embedded fallbacks	Critical
FR-9	Channel detection from landing hints, player context, screenshot/OCR text, and structured agent output	High
FR-10	Provider enrichment and reviewable evidence bundle generation	Medium
FR-11	Operator console views for runs, telemetry, screenshots, tools, settings, and provider results	High

Table 2.2: Functional requirements derived from the evidence-collection workflow.

2.5 Design Constraints and Requirement Traceability

The requirements were not independent checklist items. They came from a small set of constraints observed during the internship: pages were dynamic, players were nested, media URLs could be temporary, browser-agent runs consumed time and tokens, and the final output

had to remain reviewable by a human analyst. Figure 2.4 shows how the operational problem, requirements, artifact response, and review outputs connect.

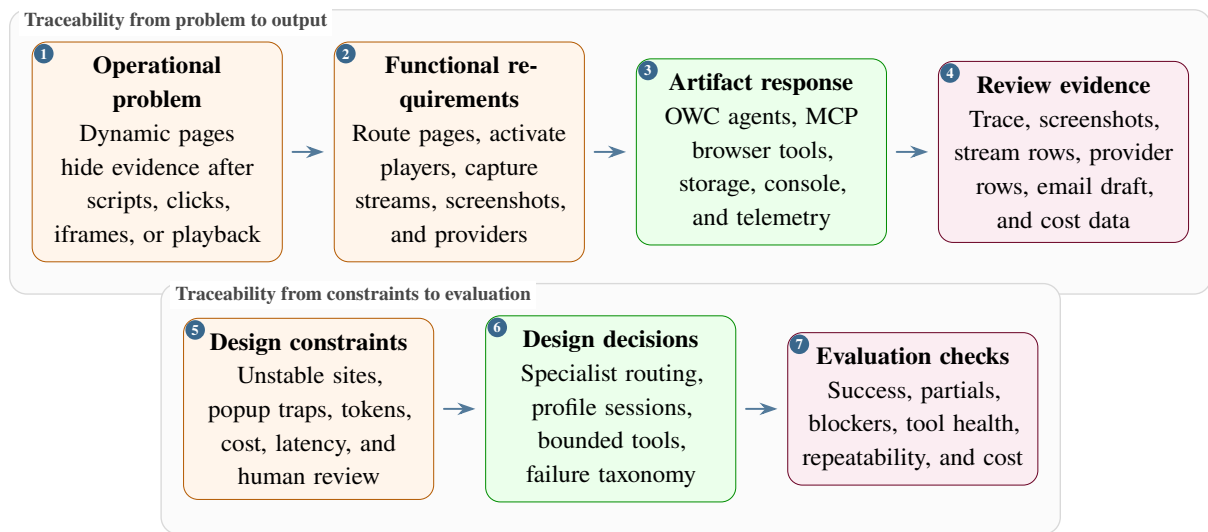


Figure 2.4: Requirement traceability from the operational problem to the implemented artifact and review outputs.

The second traceability view focuses on design pressure. Each constraint forced a specific engineering response rather than a generic automation script. Figure 2.5 summarizes these responses and explains why the artifact uses specialist agents, profile-scoped browser tooling, trace storage, telemetry, and a focused company deployment path.

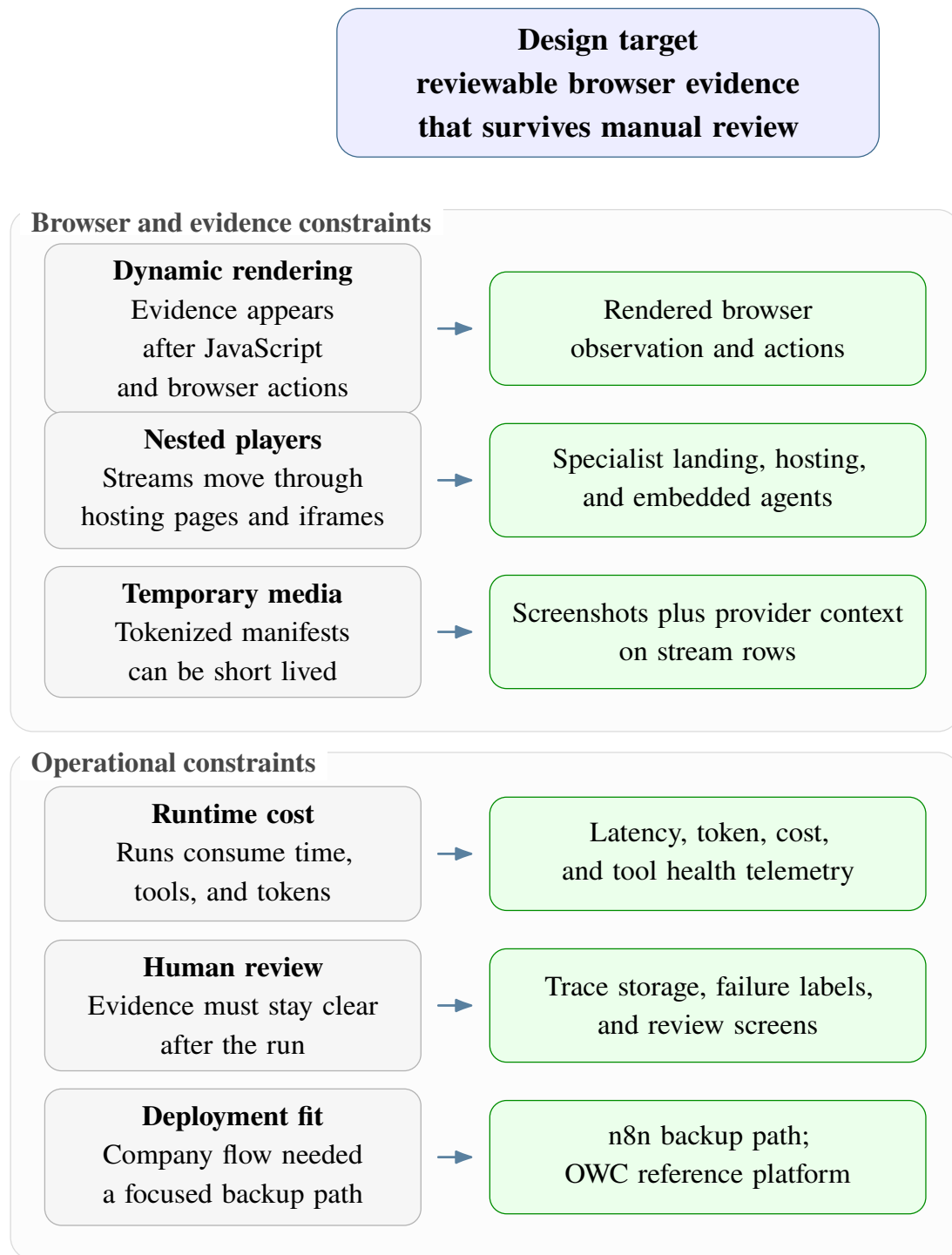


Figure 2.5: Main design constraints and the engineering responses selected for OWC.

2.6 Non-Functional Requirements

The non-functional requirements explain the qualities required for the artifact to be useful after a run finishes.

This DSRM framing defines the rest of the report. Chapter 3 introduces the technical concepts needed to understand the artifact. Chapter 4 describes the OWC architecture, including the agent

ID	Non-functional requirement	How it is addressed
NFR-1	Maintainability	OWC separates backend routes, frontend views, agent contracts, browser tools, runtime settings, and persistence records.
NFR-2	Observability	Runs persist runtime events, agent runs, LLM calls, tool calls, screenshots, streams, provider rows, and final statuses.
NFR-3	Traceability	Each result can be connected back to a URL, agent, tool call, screenshot, stream candidate, and final decision.
NFR-4	Reliability	Profile-scoped sessions, bounded retries, popup recovery, iframe recovery, media verification, and explicit failure states reduce silent errors.
NFR-5	Cost control	Token counters, cached input tracking, model settings, rate-limit awareness, and dashboard cost summaries make model usage visible.
NFR-6	Adaptability	Page-type specialists, browser profile controls, proxy configuration, fingerprint rotation, and memory hints support changing site behavior.
NFR-7	Deployment fit	The company deployment keeps n8n as a backup operational orchestrator and runs the landing agent through queue-driven container jobs.

Table 2.3: Non-functional requirements and implementation responses.

routing, browser tool layer, fingerprint and proxy controls, and channel-detection path. Chapter 5 then explains how the design decisions were implemented.

Technical Background

Before presenting the system architecture and implementation, I first introduce the technical concepts that make the project understandable. I keep this chapter at a general level because the reader needs the web structures, streaming mechanisms, browser automation tools, and agentic concepts before seeing the concrete architecture. The database schema, prompts, and tool contracts are described later.

The chapter moves from the target websites themselves toward the technologies used to analyze them. It starts with page structures and DOM representation, then explains streaming protocols and embedded players, then covers browser automation issues such as ads, cookies, popups, and anti-bot providers. It ends with the agentic concepts used in the project: agents, prompts, tool use, workflow automation, LangChain, MCP servers, model providers, caching, and token usage.

Chapter focus

The goal here is to give the reader the vocabulary needed before the design chapters: streaming-site page roles, DOM structure, embedded players, media manifests, browser automation obstacles, screenshots, network logs, prompts, tool use, LangChain, MCP, provider behavior, caching, and token usage.

3.1 Streaming Website Structures

Illegal streaming websites are not a single page type. They are usually a chain of pages with different responsibilities: one page lists events, another page presents a selected match and server choices, and a final embedded page or iframe hosts the player. Understanding these page roles is necessary before discussing extraction tools, because each role exposes different evidence and requires different interaction.

3.1.1 The normal page flow

Most target websites can be described as a three-step navigation path. A landing page contains match listings. A hosting page focuses on one selected match and usually offers one or more streaming servers. An embedded page contains the actual player, often inside an iframe loaded from another domain.

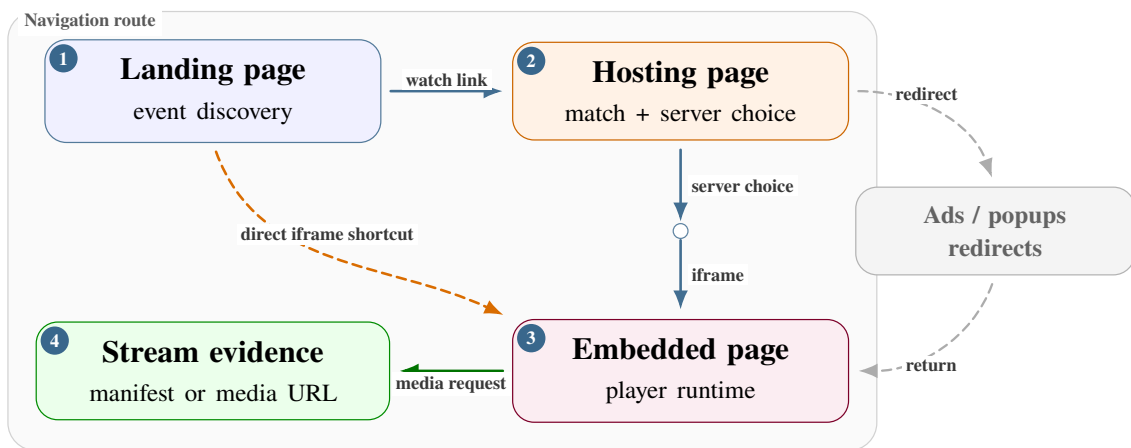


Figure 3.1: Common navigation flow from event listing to hosting page and embedded player.

Figure 3.1 shows the navigation sequence as an abstract route. The next figure uses the same sequence from the browser's point of view: it highlights what each page role exposes to the automation pipeline and what output is expected from that role before the pipeline moves on.

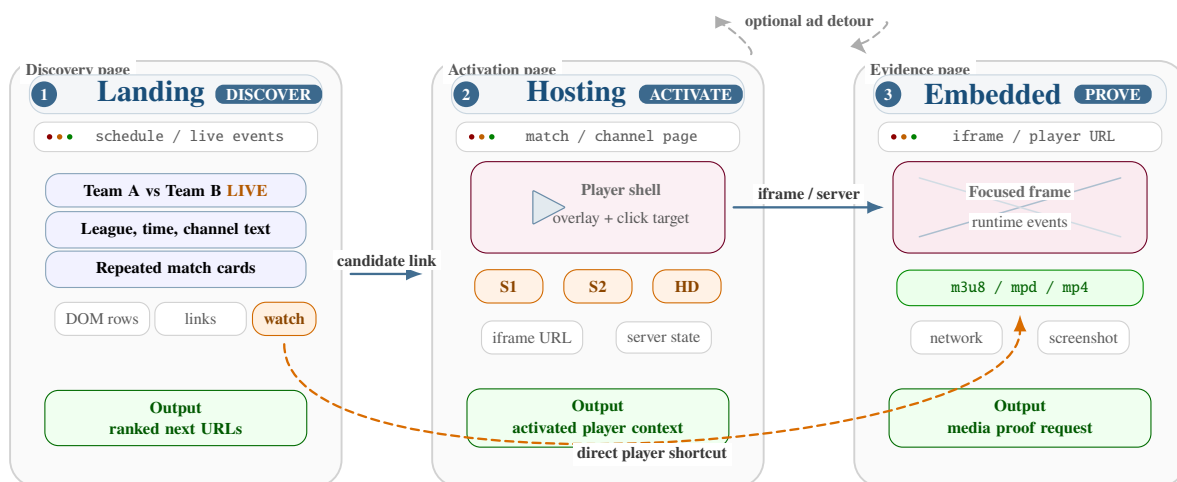


Figure 3.2: Browser-state view of landing, hosting, and embedded page roles.

This flow is not always clean. Some sites skip the hosting page and link directly to an embedded player. Others place several ad pages between the hosting page and the player. Still, this model is useful because it separates discovery, server selection, and media extraction into distinct technical problems.

3.1.2 Landing pages

Landing pages are the discovery layer. They usually contain match cards, schedule rows, league filters, date tabs, search boxes, or category menus. The useful output from a landing page is not the stream itself but a set of candidate hosting links and match metadata.

Common landing-page layouts include:

- **card grids:** repeated blocks with teams, time, league, and a watch link;

- **tables or lists:** one row per event, often with a direct action link;
- **tabbed schedules:** live, today, yesterday, tomorrow, or league-specific tabs loaded through JavaScript;
- **lazy-loaded sections:** match cards that appear only after scrolling or after an XHR request;
- **embedded JSON:** page data stored in `<script type=application/json>` blocks or `data-*` attributes.

The important idea is that landing pages are structurally repetitive. Even when class names change, the page still tends to contain repeated event units. This makes layout recognition more useful than relying on one fixed CSS selector.

3.1.3 Hosting pages

Hosting pages are the decision layer. They normally focus on one match and contain a player container, a server list, tabs such as `Server 1` or `HD`, and sometimes channel names or broadcaster logos. A hosting page may already expose the stream URL, but more often it only exposes an `iframe` or a JavaScript player configuration.

Typical hosting-page signals are:

- one match title or two team names repeated near the player;
- server buttons, mirrors, or quality selectors;
- an `iframe` whose `src` points to a player page;
- player libraries such as `Video.js`, `JWPlayer`, `Clappr`, `hls.js`, or `dash.js`;
- visible state changes after clicking play, selecting a server, or closing an overlay.

Hosting pages are more difficult than landing pages because they require interaction. The same click can activate a real player, open a popup, switch a server, or trigger a redirect. This is why browser automation and runtime observation become necessary.

3.1.4 Embedded pages

Embedded pages are the media layer. They are often minimal pages loaded inside `iframes`, with little content except a player, a play button, and scripts that request the final media manifest. They may belong to another domain, which means the top page and the player page have different DOMs, cookies, and network contexts.

An embedded page is important because the final stream reference is often generated only there. A static request to the hosting page may not show any `.m3u8` or `.mpd` URL, while the embedded player requests the manifest after user activation.

The page taxonomy turns a messy website into a sequence of smaller questions: Where are the matches? Which page hosts the selected match? Which server or `iframe` contains the player? Which runtime request proves that media was loaded?

Page type	Main role	Useful evidence
landing_page	Discover events and candidate match pages	match links, team names, schedule metadata, listing screenshot
hosting_page	Select server and activate player	server labels, iframe URLs, player state, page screenshot
embedded_page	Load the final media runtime	stream manifest, segment requests, player diagnostics
other	Filter noise such as ads, redirects, and unrelated pages	reason for skip or blocked state

Table 3.1: Streaming-site page taxonomy and evidence role.

3.2 DOM Structures and Web Page Representation

The Document Object Model (DOM) is the browser's tree representation of a web page. It turns HTML into nodes such as documents, elements, attributes, text nodes, iframes, buttons, images, and scripts. Browser automation tools operate on this tree when they query selectors, read attributes, click buttons, or remove overlays.

3.2.1 A simplified DOM tree

The diagram below shows a simplified DOM structure. It is intentionally basic, but it captures the idea used throughout the project: a visible page is not just text, it is a tree of nested elements with attributes and event handlers.

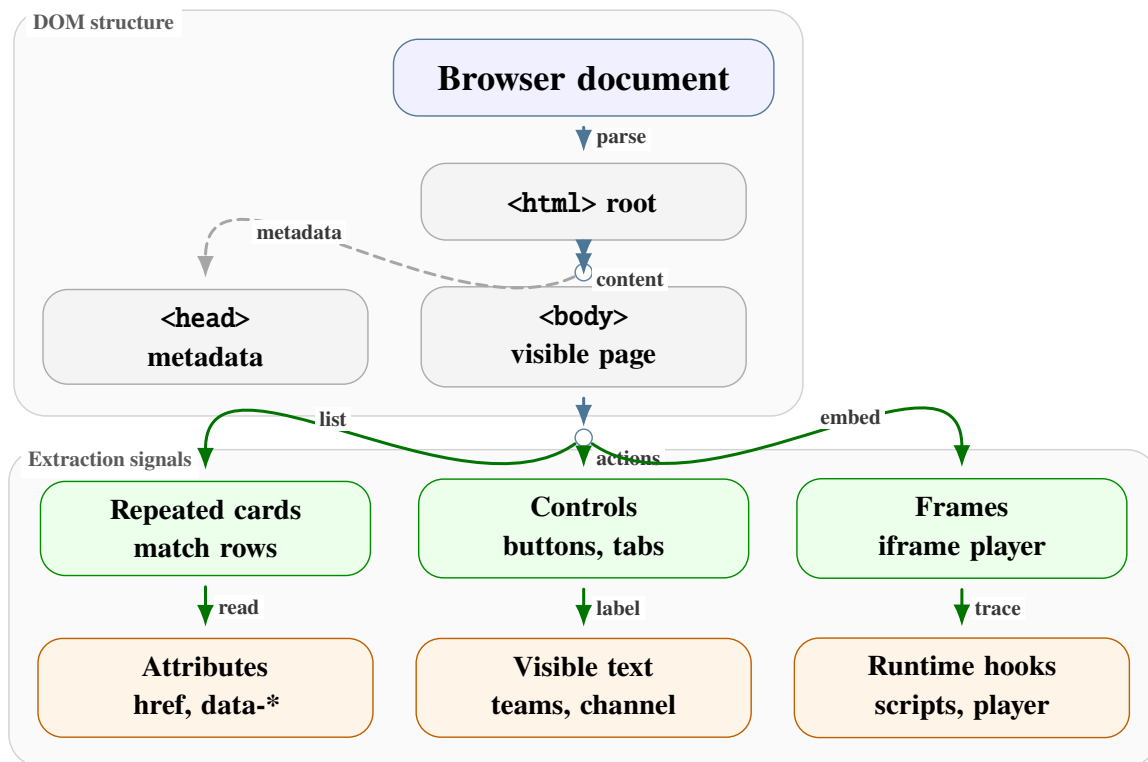


Figure 3.3: Simplified DOM tree showing how a browser represents an HTML document.

For streaming sites, the DOM is useful but incomplete. It can reveal match cards, buttons, iframes, scripts, and player containers, but it may not reveal media URLs that are generated at runtime. This is why DOM inspection is combined with screenshots and network monitoring.

3.2.2 DOM signals on streaming pages

Different page roles expose different DOM signals. Landing pages expose repeated containers. Hosting pages expose player containers and server controls. Embedded pages often expose a reduced tree with one `<video>` element or a JavaScript player root.

DOM signal	Where it appears	Why it matters
Repeated cards or rows	landing pages	indicates match listings and candidate links
<code></code>	landing and hosting pages	provides navigation targets
<code>data-*</code> attributes	landing and dynamic pages	may store event IDs, team names, or hidden URLs
<code><iframe src=...></code>	hosting pages	points to embedded player context
<code><video></code> or <code><source></code>	hosting or embedded pages	may expose direct media source references
script text and JSON blocks	all page types	may contain player configuration or lazy-loaded data

Table 3.2: DOM signals used to understand streaming-site pages.

The conclusion is that DOM analysis is a starting point, not a complete solution. It identifies page structure and interaction targets, while runtime browser evidence confirms what actually happened after the page executed.

3.3 Streaming Protocols and Embedded Players

The final evidence in this domain is usually not a normal web page. It is a media manifest, a player configuration, or a network request made by the browser while the player is active. This section explains the main protocols and player embedding patterns at a general level.

3.3.1 Common embedding patterns

Hosting pages usually embed players in one of three ways. The simplest case is a direct `<video>` element with a `src` or nested `<source>` tag. A common case is an `iframe` that loads a player from another domain. A more dynamic case is a JavaScript player library that inserts the video source after page load.

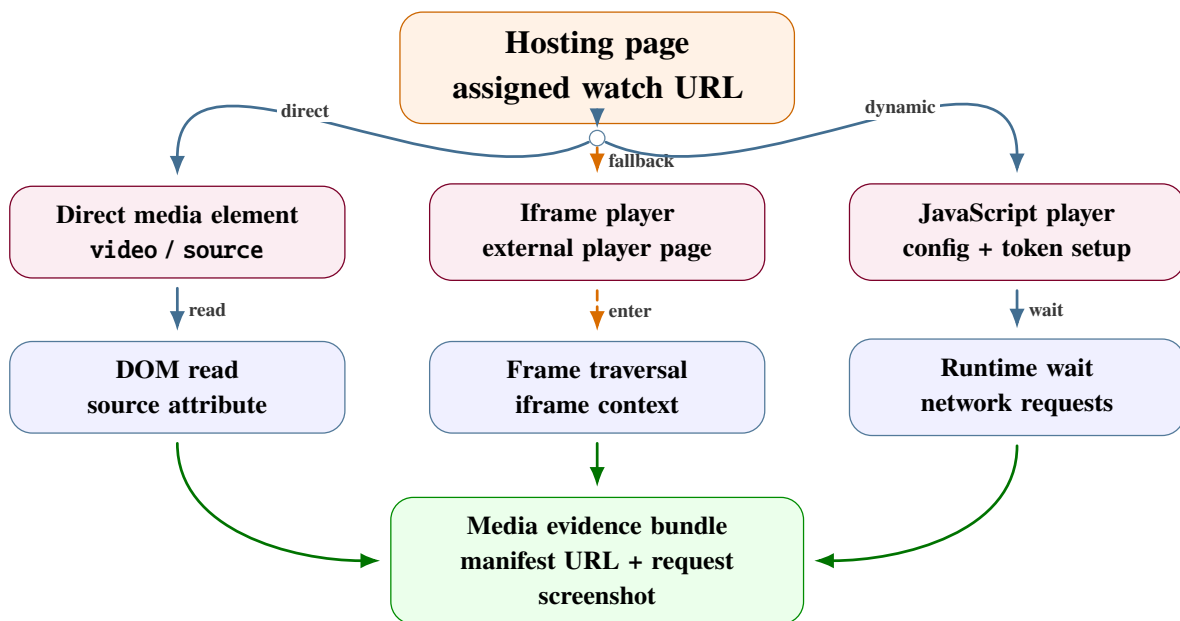


Figure 3.4: Main paths from a hosting page to media evidence.

This distinction matters because each embedding type requires a different observation strategy. Direct video tags can be read from the DOM. Iframes require frame traversal. JavaScript players often require runtime execution and network monitoring.

3.3.2 HLS and MPEG-DASH

HTTP Live Streaming (HLS) is one of the most common protocols for web video streaming. Its entry point is a `.m3u8` playlist that points either to variant playlists or to media segments. MPEG-DASH uses an XML `.mpd` manifest that describes periods, representations, and segment

addressing rules. Both protocols are adaptive: the player can choose different quality levels depending on network conditions [18, 19].

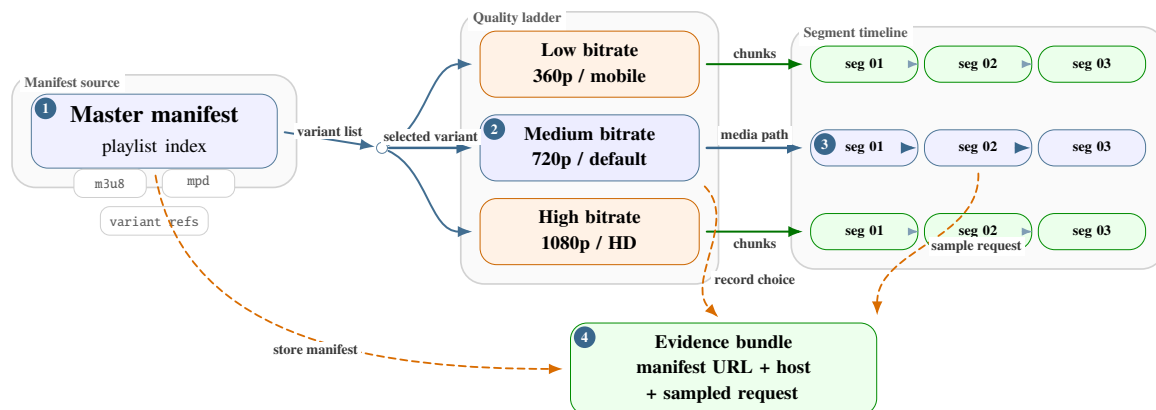


Figure 3.5: Simplified adaptive streaming hierarchy used by HLS and MPEG-DASH.

For evidence collection, the manifest URL is usually more important than downloading media segments. It proves that the page resolved to a playable stream entry point and often contains CDN, token, and path information.

3.3.3 Tokens, cookies, referers, and provider checks

Streaming manifests are frequently protected by short-lived tokens, cookies, referer checks, or provider-side gates. A URL such as `playlist.m3u8?token=...` may work only for a few minutes and only from the browser session that generated it. CDNs and fronting providers such as Cloudflare can also enforce challenge pages, bot checks, cache rules, and origin protection [17].

This is why the browser session matters. A raw HTTP request may fail even when the browser can play the stream, because the browser carries the active cookies, headers, referer, and JavaScript-generated state. For that reason, the technical background includes both media protocols and web-provider behavior.

3.4 Browser Automation, Scraping, and Runtime Obstacles

Static scraping means downloading HTML and parsing it. That is useful for simple pages, but it is not enough for pages that depend on JavaScript, user clicks, iframes, or runtime media requests. Browser automation tools such as Puppeteer and Playwright execute pages in real browsers and expose APIs for clicking, reading the DOM, taking screenshots, and listening to network activity [4, 5, 6].

3.4.1 Static collection versus rendered browser state

Static collection is efficient when the target evidence is present in the fetched HTML or in simple linked pages. Its limitation appears when the evidence is created only after a browser sequence:

- open a collapsed channel group;
- click a server tab;
- activate the player;
- wait for an iframe or player state;
- observe the network requests.

In those cases, the browser state is the evidence surface, not only the page source. This is why OWC adds browser agents to static monitoring instead of treating the first page source as sufficient.

3.4.1.1 Rendered state as evidence

Rendered state includes visible text, player overlays, frame trees, screenshots, media elements, and network requests that appear after interaction. It matters because the final stream or channel signal may not exist at the first page load. A screenshot, media-state record, or tool trace can explain why an agent believed the page exposed unauthorized streaming behavior.

3.4.2 Puppeteer as a browser control layer

Puppeteer provides programmatic control over Chromium. In this project context, the important concept is not only “click a button” but full runtime observation:

- inspect the DOM after JavaScript has executed;
- click buttons, tabs, overlays, server selectors, and play controls;
- traverse frames and iframes;
- capture screenshots for human review;
- observe network requests through the Chrome DevTools Protocol (CDP);
- read browser-side state through page-evaluation calls.

Puppeteer acts as the bridge between a high-level extraction objective and the actual state of the web page. It lets the system test what a user would see and what the browser actually requested.

3.4.3 Ads, cookies, popups, and other annoyances

Illegal streaming websites often monetize through aggressive ad behavior. These behaviors are not just inconvenient; they change the extraction problem because the automation may click the wrong layer, follow an ad redirect, or lose the original player context.

Obstacle	Common symptom	General handling idea
Popups and new tabs	clicking play opens an ad page instead of the player	close extra targets and retry on the original page
Overlay layers	a fake play button covers the real player	detect visible overlays and click or remove them carefully
Cookie banners	controls are blocked until consent is handled	accept, dismiss, or record as blocking evidence
Ad iframes	DOM contains many unrelated frames	rank frames by player-like signals instead of following every iframe
Cloudflare-like challenges	browser sees an interstitial instead of the page	wait, retry with normal pacing, or preserve a blocked-state result

Table 3.3: Browser-level obstacles and handling strategies.

Ad blocking tools such as uBlock Origin can reduce noise by blocking known ad and tracking domains [20]. However, they cannot replace browser reasoning. If a site serves ads and player assets from the same domain, aggressive blocking can break the player itself. The general solution is layered: block obvious noise, keep popup recovery, and preserve evidence when the page remains blocked.

3.4.4 Why screenshots and network logs matter

DOM text alone does not prove that a player actually loaded. A screenshot can show that the player was visible or that a challenge blocked the page. A network log can show that the browser requested a media manifest. Together, DOM, screenshot, and network evidence give a more reliable picture than any one source by itself.

This is also why extraction tools should avoid claiming success only because a keyword or button was found. In this domain, useful evidence is the combination of page role, visible state, player behavior, and media request.

3.5 Agentic Evidence Collection Concepts

The project uses the term *agentic* to describe an extraction approach where a language-model-driven agent observes a page, chooses a tool action, checks the result, and repeats until it has enough evidence or reaches a limit. This differs from a fixed scraper because the next action depends on the current browser state.

3.5.1 Agents and ReAct-style loops

Agentic systems commonly use a loop that combines reasoning and tool use. In a ReAct-style pattern, the agent observes the state, reasons about what to do, acts through a tool, and verifies the result before continuing. In the implementation style used by OWC, this loop is represented

through model messages, tool calls, observations, and structured outputs exposed by the agent framework [11].

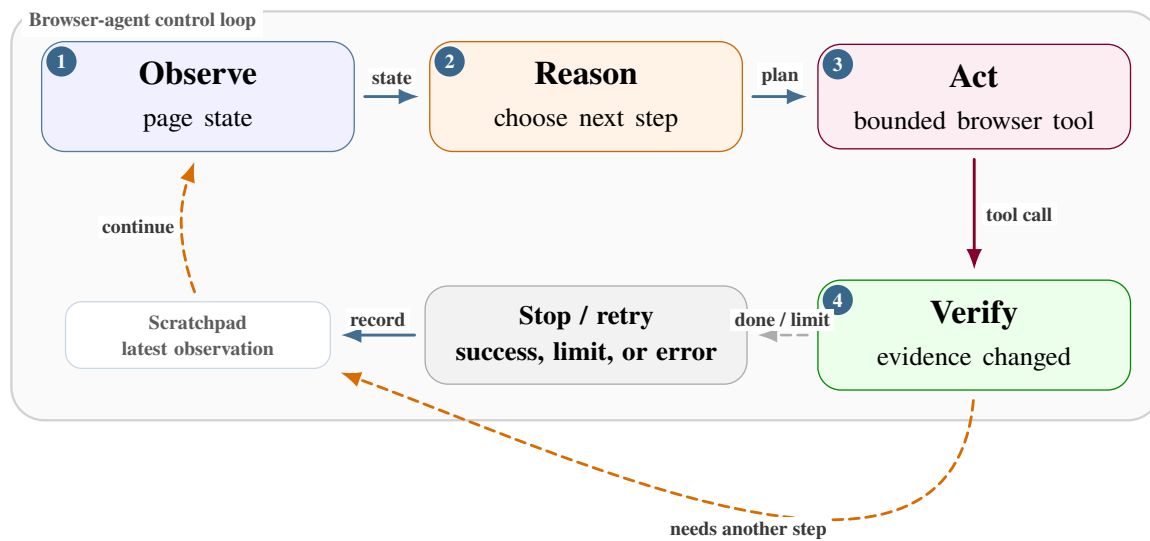


Figure 3.6: Observe, act, and verify loop used by browser agents.

For streaming sites, this loop is useful because the agent can adapt when a server button fails, a popup opens, an iframe appears, or a page turns out to be unrelated. The agent is not expected to know the full path in advance; it uses tools to discover it.

3.5.2 Prompt engineering, tokens, and token usage

Prompt engineering is the process of defining the agent’s role, constraints, tool usage rules, evidence requirements, and output format. In this project domain, a prompt must prevent premature conclusions. It should tell the agent to ground decisions in visible page state, DOM evidence, screenshots, and network observations.

Tokens are the units used by language models to process prompts and outputs. Long page snapshots, repeated tool results, and verbose traces increase token usage and can also increase cost and latency. Token management is part of the technical background because browser agents can easily spend too much context on noisy HTML, ads, or repeated page states.

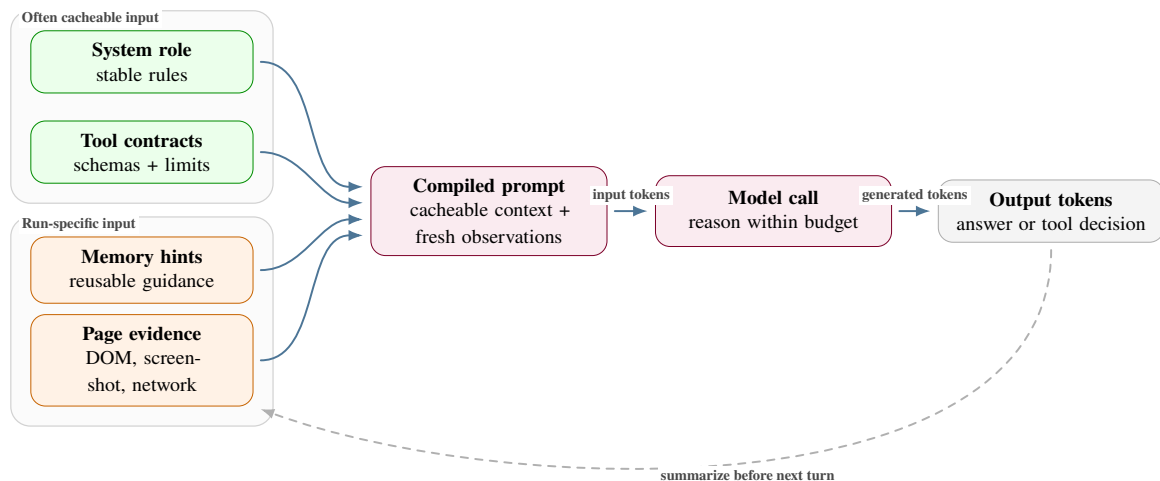


Figure 3.7: Prompt and token-budget composition for an evidence-driven browser agent.

Practical token-control ideas include:

- summarize page observations instead of passing full raw HTML every time;
- keep only evidence-relevant DOM snippets and network URLs;
- stop early when the page role or stream evidence is clear;
- separate input tokens, output tokens, and cached tokens when estimating cost;
- use provider-side caching when stable prompt sections are reused.

LLM providers such as Gemini expose model features that affect cost, context length, streaming behavior, pricing, caching, and rate limits [2, 3, 1]. Provider caching is especially relevant when the same system instructions, tool descriptions, or long context blocks are reused across many runs.

3.5.3 Workflow automation tools

Workflow automation tools let developers connect HTTP requests, browser actions, LLM calls, data transformations, and external services through a visual flow. They are useful for fast validation because a workflow can be assembled and adjusted before a full application is built.

In this report, workflow automation is treated as an orchestration concept. It helps to:

- sequence evidence-collection steps;
- pass data between browser, model, and processing nodes;
- call model providers in a visible workflow;
- make experimental extraction logic easier to inspect.

The specific workflow tool used during implementation is introduced in Chapter 5.

3.5.4 LangChain, tools, and MCP servers

LangChain is a framework for building applications around language models, prompts, tools, memory, and structured chains. At a general level, it helps connect an LLM to external capabilities: a browser tool, a database lookup, a parser, a memory store, or a custom function. This is useful when the model should not only answer textually but take controlled actions [11].

The Model Context Protocol (MCP) is a way to expose external tools and resources to an agent through a consistent interface. In this type of project, an MCP server can expose browser tools such as inspect, click, navigate, screenshot, harvest, or media-state checks. The agent does not need to know how each tool is implemented internally; it needs a clear tool contract, input schema, and output shape [13].

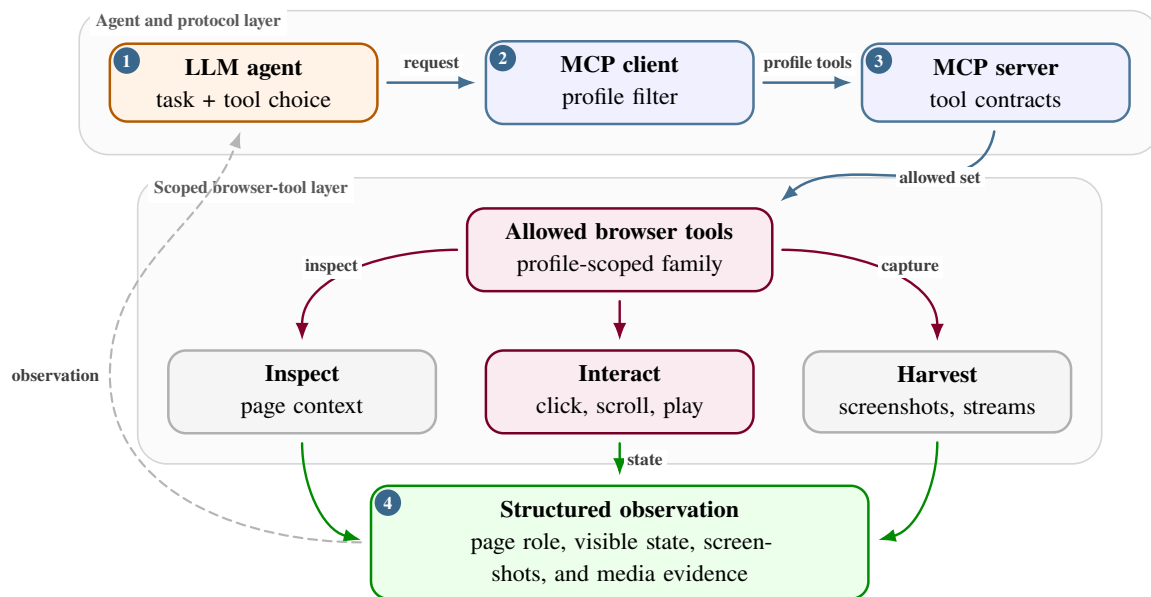


Figure 3.8: General MCP pattern connecting an LLM agent to scoped browser tools.

The important concept is separation of responsibility. The LLM decides what to try next, while tools perform bounded actions and return structured evidence. This keeps the agentic process auditable.

3.6 Chapter Summary

With this background in place, the report can now move from general concepts to the system design. This chapter explained:

- the page flow from landing pages to hosting pages and embedded players;
- the DOM structures used to inspect those pages;
- the streaming protocols and player embeds that carry final evidence;
- the browser obstacles caused by ads, cookies, popups, iframes, and provider checks;
- the agentic concepts used later: ReAct-style loops, prompt engineering, tokens, provider caching, workflow automation, LangChain, and MCP tool servers.

The next chapters use these concepts more concretely. Chapter 4 translates them into architecture, and Chapter 5 explains how the implementation applies them in the actual evidence-collection system.

System Architecture

I present the architecture of Open Web Catcher (OWC) as the evidence-collection platform developed during the internship. The chapter starts with the user-facing system and runtime components, then narrows into the multi-agent pipeline, agent handoffs, browser tooling, and persisted review data.

Here, OWC refers to the reference platform I built during the internship: backend, console, browser tools, specialist agents, runtime telemetry, persistence, and review outputs. Chapter 5 then maps these architectural decisions to the implementation.

Chapter focus

The system is a staged evidence pipeline, not a single unrestricted browser agent. FastAPI starts and records runs. The orchestrator owns the route, handoffs, queues, aggregation, provider analysis, and email drafting. Four browser-facing specialists perform bounded work: classification, landing-page discovery, hosting-page extraction, and embedded-player extraction. Each specialist uses profile-scoped MCP browser tools, returns typed output, and leaves events that explain success or failure.

4.1 System Purpose and Architecture Reading Path

The platform is an evidence-collection workbench for illegal streaming websites. Its roles are separated on purpose:

- the operator launches investigations and reviews evidence;
- the engineer or administrator maintains models, prompts, pricing, datasets, browser settings, and MCP tools;
- external services supply target sites, model inference, screenshot storage, and provider data.

The useful architectural view here is not a generic use-case picture. It is the path from an operator action to the records that make the action reviewable later.

I present the architecture in four layers:

1. **Product layer:** the operator console launches runs, monitors progress, and reviews evidence.
2. **Runtime layer:** FastAPI validates runtime readiness, schedules jobs, resolves settings, and persists trace data.
3. **Agent layer:** the orchestrator routes work through specialized agents instead of asking one prompt to do every task.
4. **Evidence layer:** streams, screenshots, provider rows, emails, events, model calls, and tool calls are normalized for later review.

Console responsibility	Operator intent	Persisted or observable response
Launch investigation	Submit one URL, a single-agent test, or a dataset batch.	Create a traceable run, validate runtime readiness, and schedule execution.
Monitor execution	Follow agent transitions, tool calls, screenshots, events, and failures.	Stream active state and persist each important runtime event.
Review evidence	Inspect streams, screenshots, provider analysis, metrics, and takedown drafts.	Assemble a reviewable evidence bundle with final status and supporting trace data.
Configure system	Tune models, prompts, MCP tools, browser settings, pricing, and datasets.	Persist configuration and expose health or preflight feedback before runs start.

Table 4.1: Operator-facing responsibilities and the runtime records they create.

4.2 Global Runtime Architecture

The runtime separates the operator console, API, agent execution, browser automation, persistence, and external enrichment services. The Next.js console does not access the database directly; it calls FastAPI endpoints. FastAPI owns background-job execution, active trace assembly, settings resolution, and database access. Browser work is delegated to profile-scoped MCP tooling so the agents receive only the tools allowed for their page type.

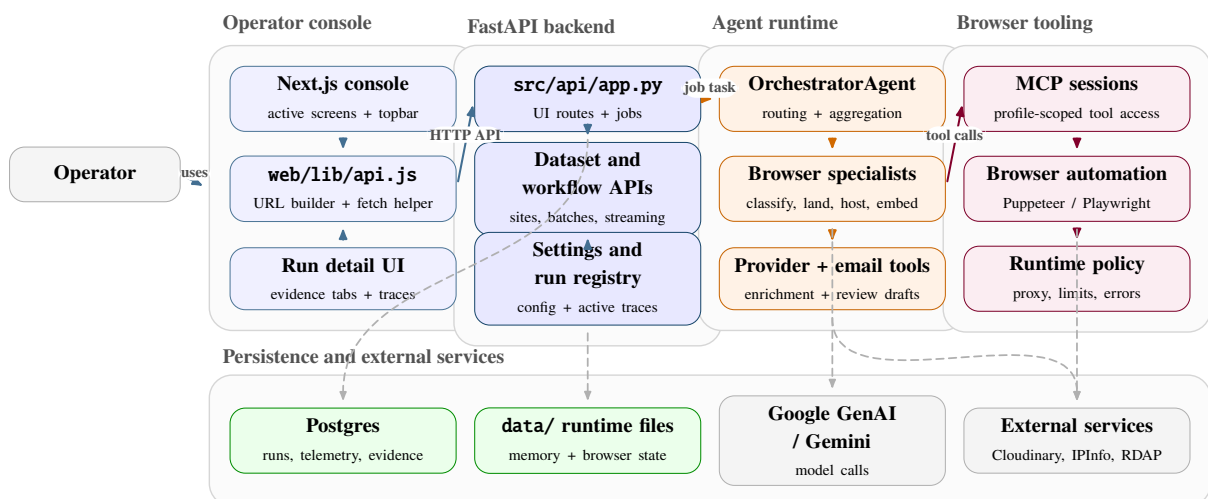


Figure 4.1: System component diagram for the OWC runtime.

The frontend route map is part of the architecture because the console is the operator's entry point into the runtime. The active screens are the dashboard, live pipeline, results, run detail, provider lookup, and settings pages; the /agents path is kept only as a redirect to /live. The route map starts with the shared console shell, then moves to the active screens, and finally shows the FastAPI route groups consumed by each screen.

Runtime area	Main responsibility	Why it is separate
Operator console	Launch workflow runs, inspect run detail, manage datasets, providers, tools, browser settings, and model configuration.	The UI stays thin; it consumes API payloads instead of owning persistence.
FastAPI backend	Validate preflight state, create jobs, run background work, assemble active and persisted traces, and expose review payloads.	Runtime control and database access remain testable from backend routes.
Agent runtime	Execute the orchestrator graph and the specialist model/tool loops.	Routing policy remains explicit instead of hidden inside one prompt.
MCP browser tooling	Provide profile-scoped page, frame, screenshot, interaction, and network tools.	Browser dependencies and page failures are isolated from the API process.
Persistence and services	Store normalized run records and call Gemini, Cloudinary, IPInfo, RDAP, and local runtime files.	The final answer and the failure path can be reviewed after execution.

Table 4.2: Global runtime boundaries used by OWC.

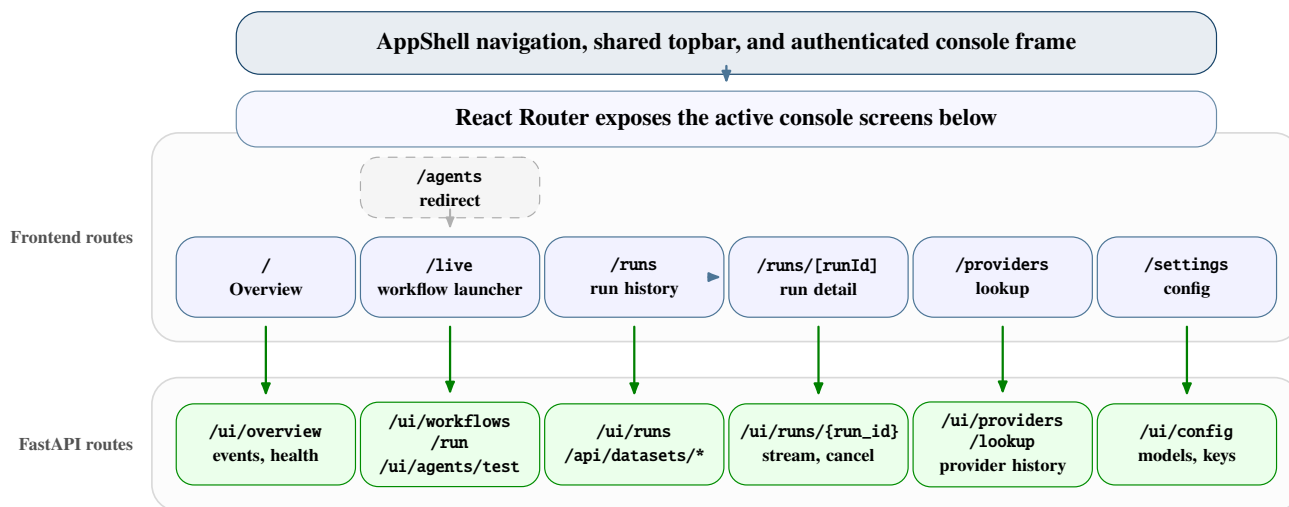


Figure 4.2: Frontend console routes and API route groups.

This separation is practical rather than cosmetic. Browser automation, LLM reasoning, dashboard rendering, and persistence fail in different ways. Keeping their boundaries visible makes failures easier to classify: a browser-tool timeout, a model retry, a missing stream, a skipped provider lookup, and a dashboard payload issue are different problems.

4.3 End-to-End Agent Pipeline

At large scale, the platform behaves as a controlled multi-agent pipeline, not as one unrestricted browser agent. The operator gives the system a seed URL. The API starts a run, creates an active trace, and calls the orchestrator. The orchestrator checks memory as soft guidance, calls classification, routes to the appropriate specialist path, collects evidence, and only then runs provider analysis and email generation.

I use three complementary views for the same run. Figure 4.3 shows the operational path: where the run can block, branch, skip provider/email work, or stop with a no-stream result. Figure 4.4 shows ownership: specialist stages, browser runtime, provider analysis, and final pipeline result. Figure 4.5 shows timing: when the API, orchestrator, specialist agents, tools, observer, and database exchange information during one investigation.

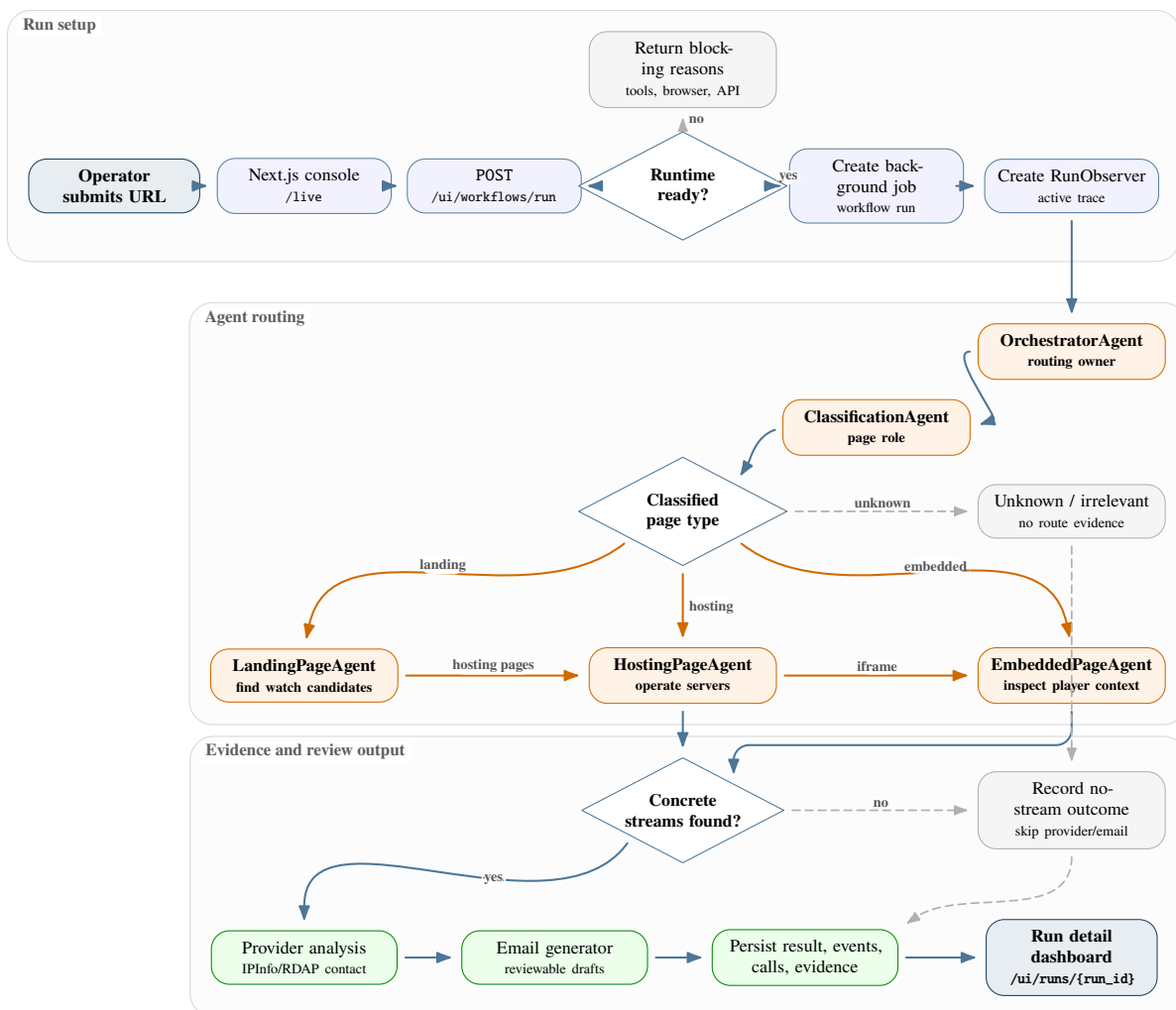


Figure 4.3: End-to-end workflow from launch to persisted evidence.

After the decision path is clear, the next view shifts from “what happens next?” to “who owns each part of the work?” The same run crosses the console, API, orchestrator, browser tools, storage layer, and review outputs, so ownership has to stay visible.

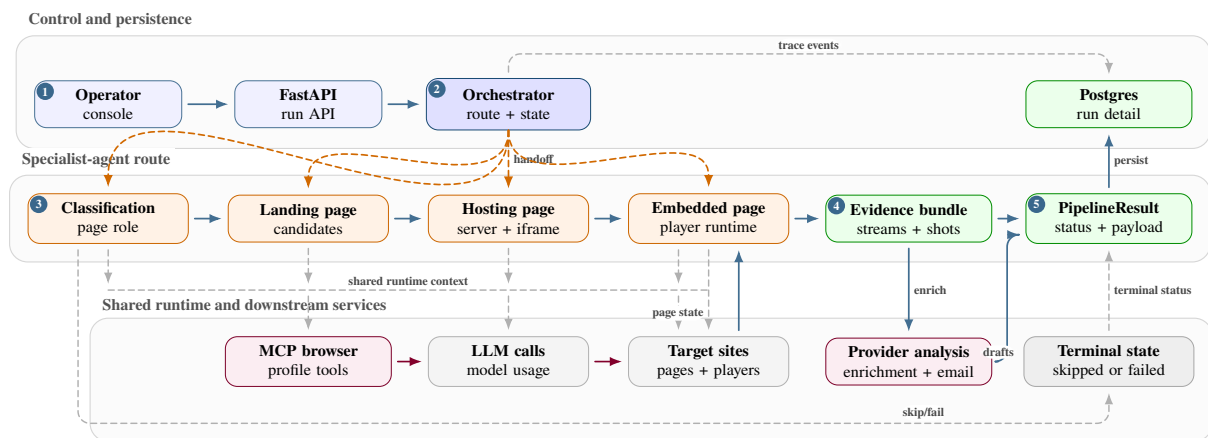


Figure 4.4: End-to-end agent architecture at workflow scale.

The sequence diagram then adds the time dimension. It shows how the same components exchange requests, tool calls, trace events, and final run data during one investigation.

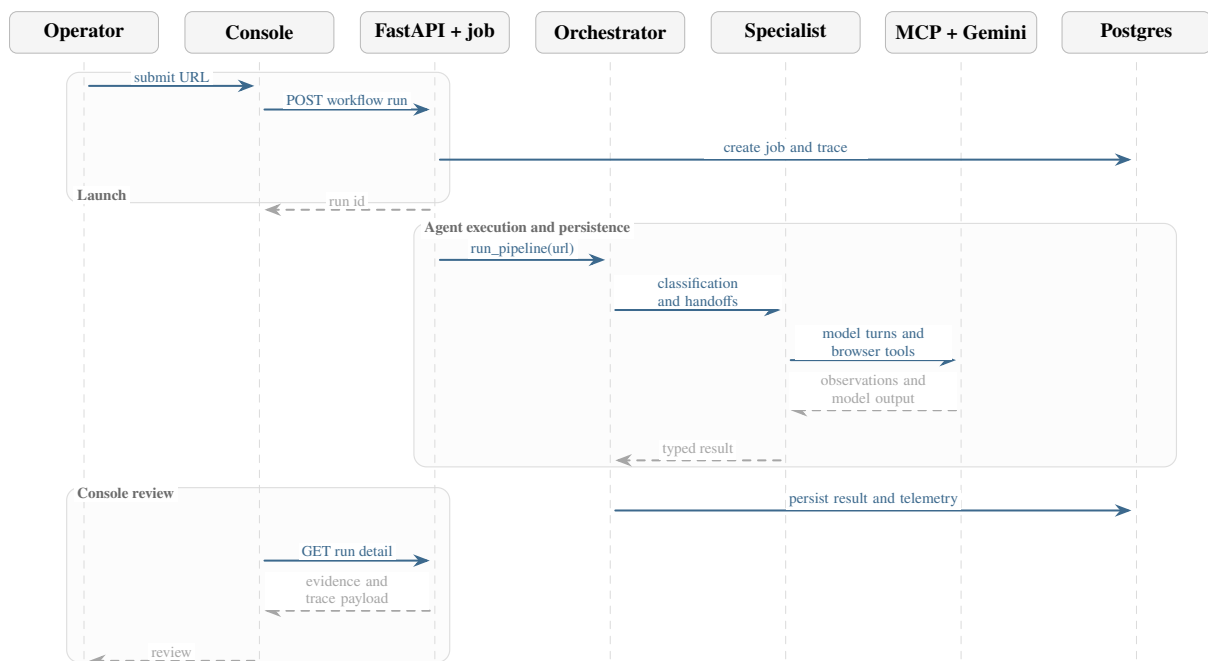


Figure 4.5: Sequence diagram for launching, executing, persisting, and reviewing a workflow run.

The pipeline has three important rules:

- **Agents do not call each other directly:** every specialist returns typed output to the orchestrator.
- **The orchestrator owns queues:** landing can add hosting or embedded targets, hosting can add more hosting or embedded targets, and embedded returns final media evidence.
- **Downstream stages require streams:** provider analysis and email generation are skipped when no stream URL exists.

4.4 Agent Communication and Orchestrator Handoffs

The orchestrator is the routing authority. It builds the LangGraph state machine, checks memory as soft guidance, calls classification, prepares handoffs, executes landing, hosting, and embedded specialists as needed, aggregates structured outputs, then runs provider analysis and email generation only when stream evidence exists. This keeps failures interpretable: a classification timeout, missing hosting candidate, hosting-player failure, embedded-player failure, and provider skip are different operational states.

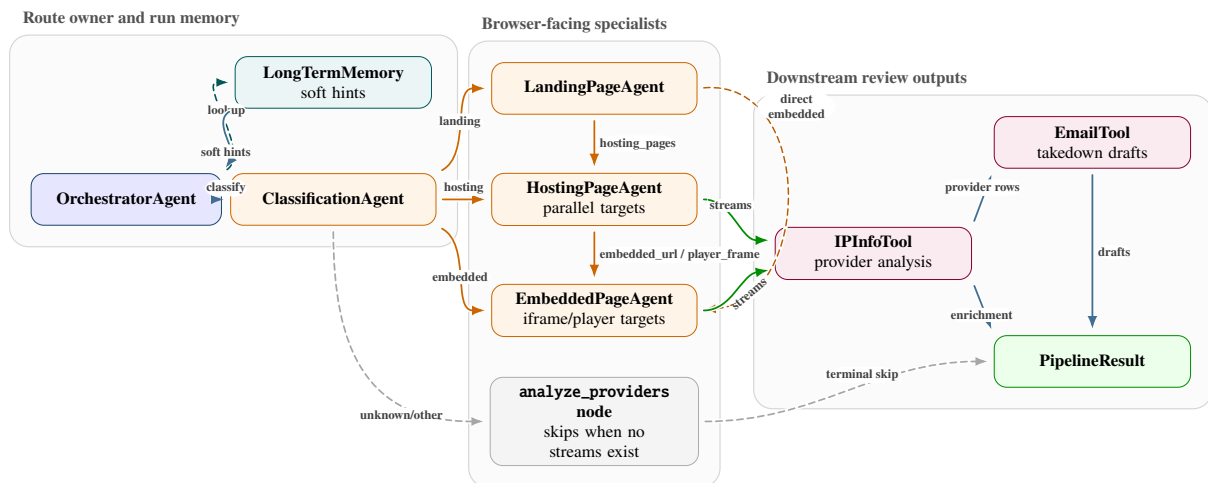


Figure 4.6: Agent interconnection graph showing orchestrator-owned routing and specialist responsibilities.

The handoff is the main communication contract between agents. It contains the root URL, target URL, page type, classification reasoning, route source, memory hints, navigation policy, required evidence, and known pattern context. A specialist receives this context inside its task brief, performs only its bounded job, and returns a schema that the orchestrator can normalize and route.

In this design, communication is visible at three levels:

- route decisions are graph edges, so the next stage is explicit;
- handoff text gives each specialist the local context it needs;
- the observer records agent starts, model calls, tool calls, screenshots, status changes, costs, and final rollups.

That is why the dashboard can explain what happened instead of only showing a final status. Figure 4.28 expands the same contract into the concrete JSON fields used by the current schemas, orchestrator, and prompts.

Prompt compilation is also part of the communication contract. Each browser-facing specialist receives the same basic ingredients but a different stage contract:

- the base policy and agent-specific instructions;
- the URL task brief and runtime context;
- long-memory site hints and short-memory working state.

Stage	Main input	Responsibility	Main output
Orchestrator	Seed URL, settings, observer, and memory hints.	Build the LangGraph route, prepare handoffs, aggregate results, compute final status, and call downstream tools.	<code>PipelineResult</code> , trace events, provider rows, and email drafts.
Classification	URL and profile-scoped browser context tools.	Decide page type without extracting streams or provider data.	Classification result with page type, confidence, and reasoning.
Landing	Landing URL plus orchestrator handoff.	Discover watch or hosting candidates while preserving match, iframe, channel, and player hints.	<code>ExtractionResult</code> with match records and hosting or embedded candidates.
Hosting	Assigned hosting URL plus evidence checklist and navigation policy.	Operate server/source controls, activate playback, collect network evidence, and return explicit embedded handoffs when needed.	Screenshots, server rows, stream URLs, diagnostics, or embedded player URLs.
Embedded	Verified player or iframe URL plus handoff context.	Stay inside the assigned player context, recover from drift, and harvest final stream evidence.	Stream URLs, screenshots, frame diagnostics, and player state.
Provider and email tools	Stream URLs, provider rows, screenshots, and extractions.	Enrich stream hosts and draft reviewable takedown notices.	<code>ProviderInfo</code> records and <code>TakedownEmail</code> drafts.

Table 4.3: Agent and tool responsibility map.

Short memory is run-local. It tracks recent navigation, tool calls, selectors, candidates, streams, server records, screenshots, and the next working objective. Long memory is cross-run site memory. It stores summarized playbooks so a future run can receive hints without trusting them blindly. In both cases, live browser evidence still decides the route.

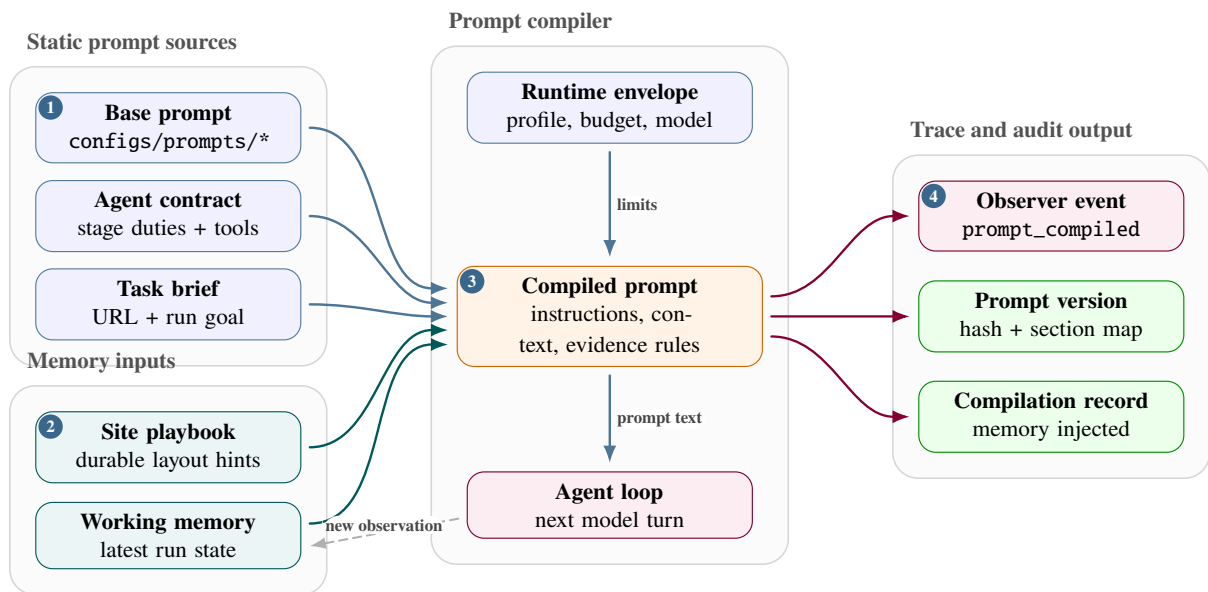


Figure 4.7: Prompt compilation and memory inputs for each browser-facing specialist.

That compilation step explains what each model call receives. The shared loop below focuses on what happens after the prompt is ready: browser inspection, tool use, observation recording, and normalized output.

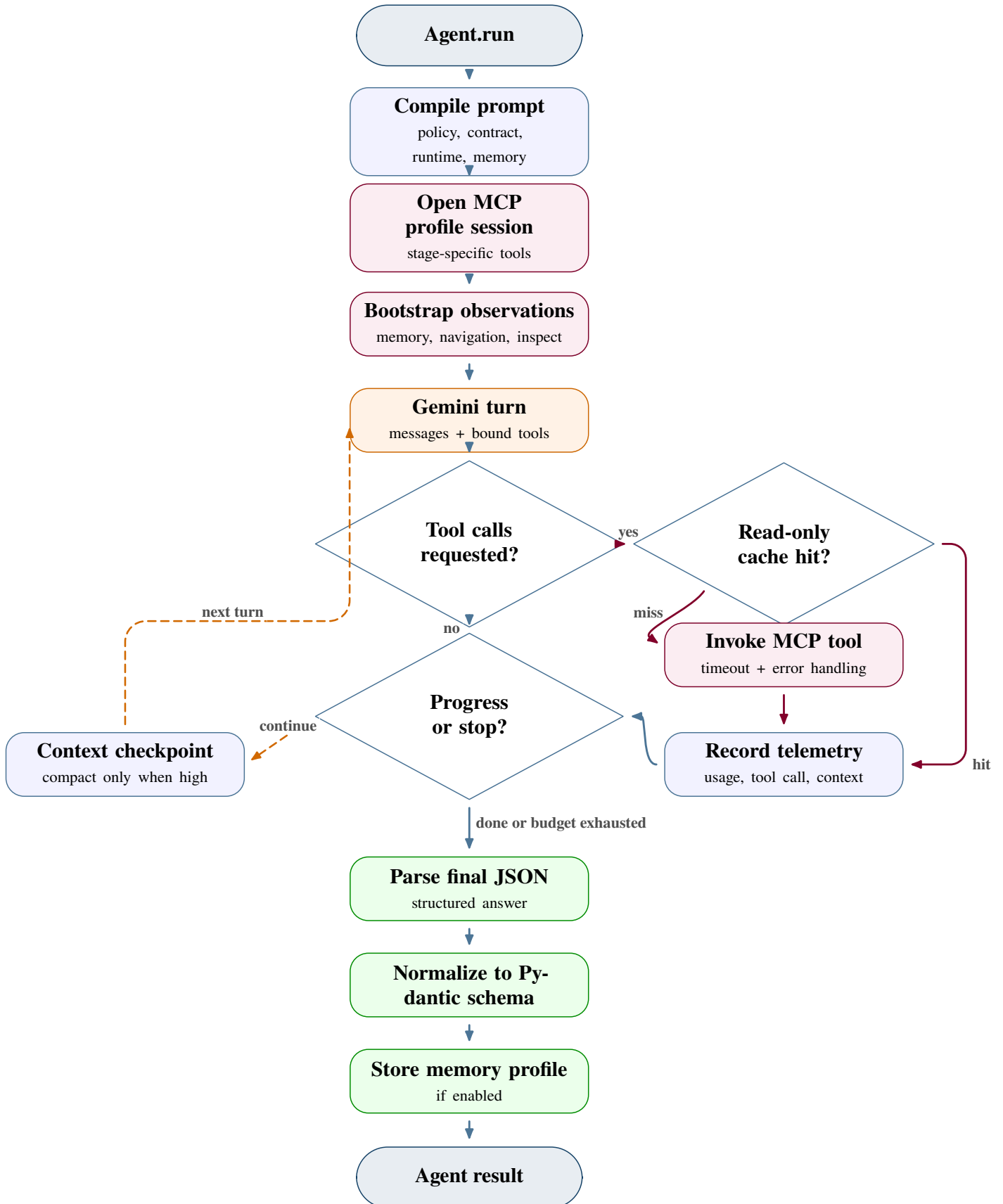


Figure 4.8: Shared activity loop for browser-facing agents.

The browser-facing specialists then reuse the same inner loop. Each one compiles its prompt, opens the correct MCP browser profile, lets the model request tools, records every model and tool turn, controls context growth, and normalizes the final answer into the stage schema.

4.5 Agent-by-Agent Design

4.5.1 Orchestrator Agent

The orchestrator is the control plane for one run. It owns state, queues, handoffs, aggregation, provider and email execution, final status, and trace emission, but it does not inspect pages directly.

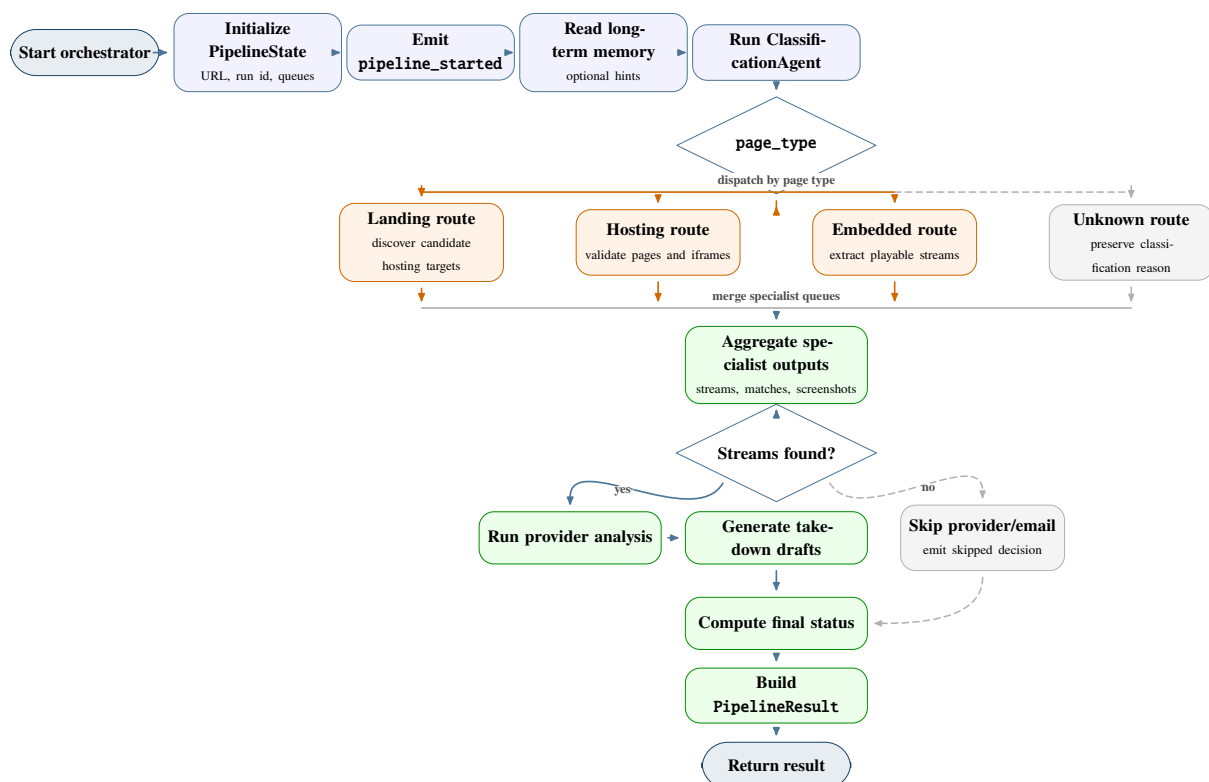


Figure 4.9: Orchestrator activity diagram showing initialization, classification routing, specialist queues, aggregation, provider/email decisions, and final PipelineResult construction.

The activity view is intentionally procedural. The next card summarizes the orchestrator as a component: its inputs, decisions, outputs, and failure signals.

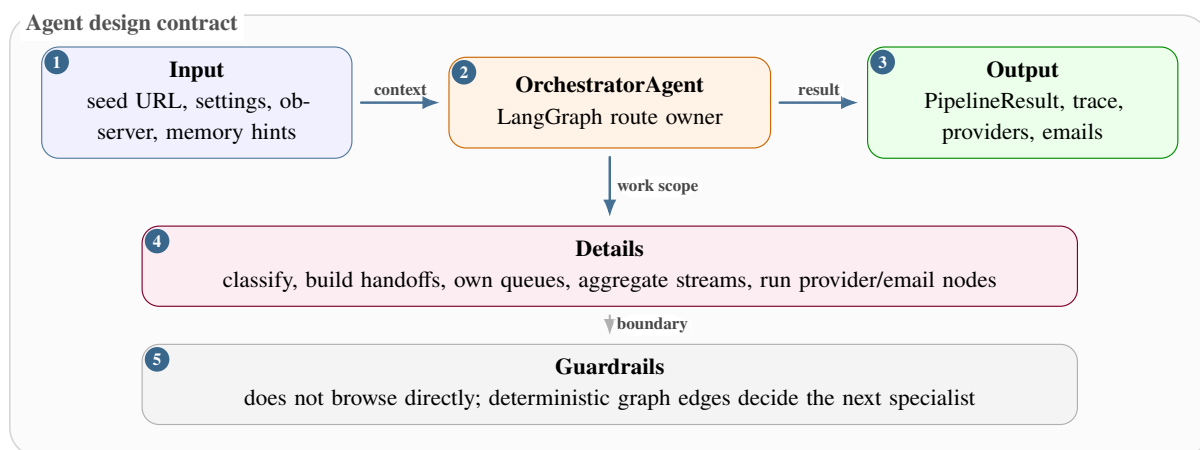


Figure 4.10: Orchestrator-agent design card.

The sequence view closes the orchestrator subsection by showing how those responsibilities appear during a real API-triggered run.

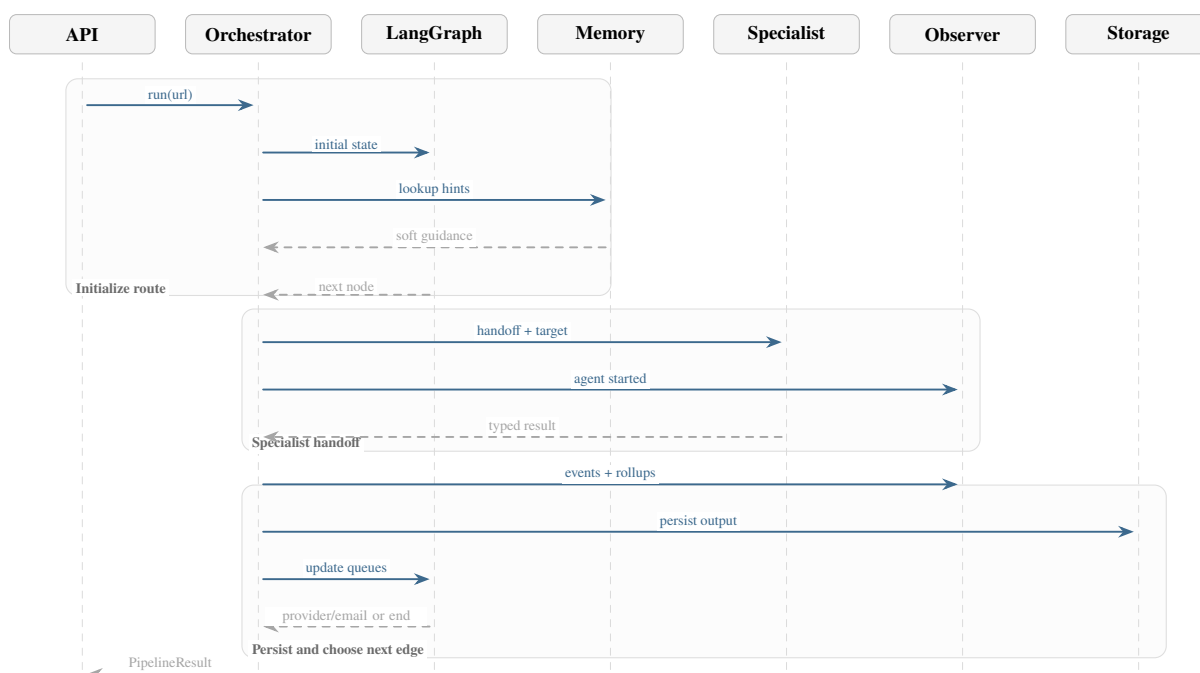


Figure 4.11: Sequence diagram for orchestrator handoff, trace recording, and route updates.

4.5.2 Classification Agent

The classification agent answers one question: what type of page is this URL? It distinguishes landing, hosting, embedded, unknown, and irrelevant pages before extraction begins.

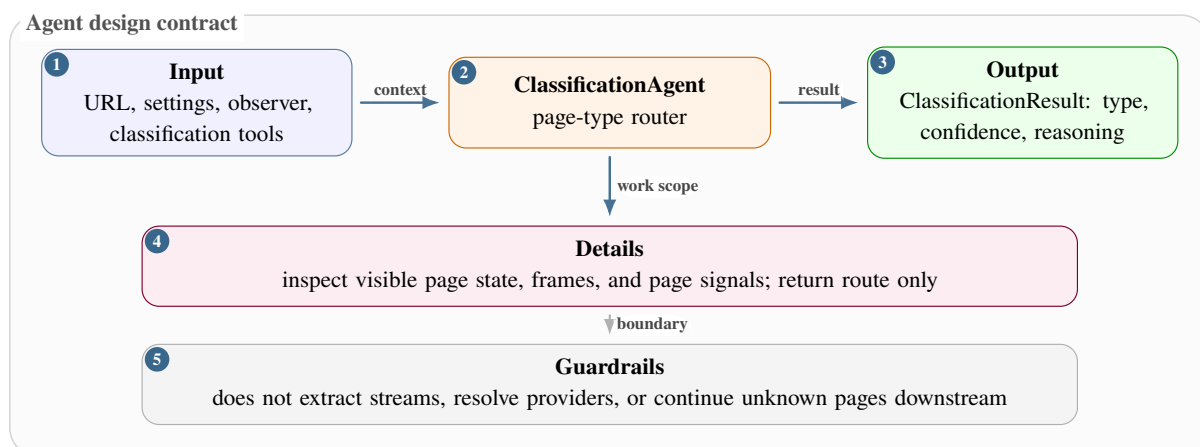


Figure 4.12: Classification-agent design card.

A landing result sends the run to the landing agent. A hosting result queues the root URL for the hosting agent. An embedded result queues the root URL for the embedded agent only when the page looks like a direct player. Unknown or irrelevant pages skip provider and email stages because no stream evidence exists.

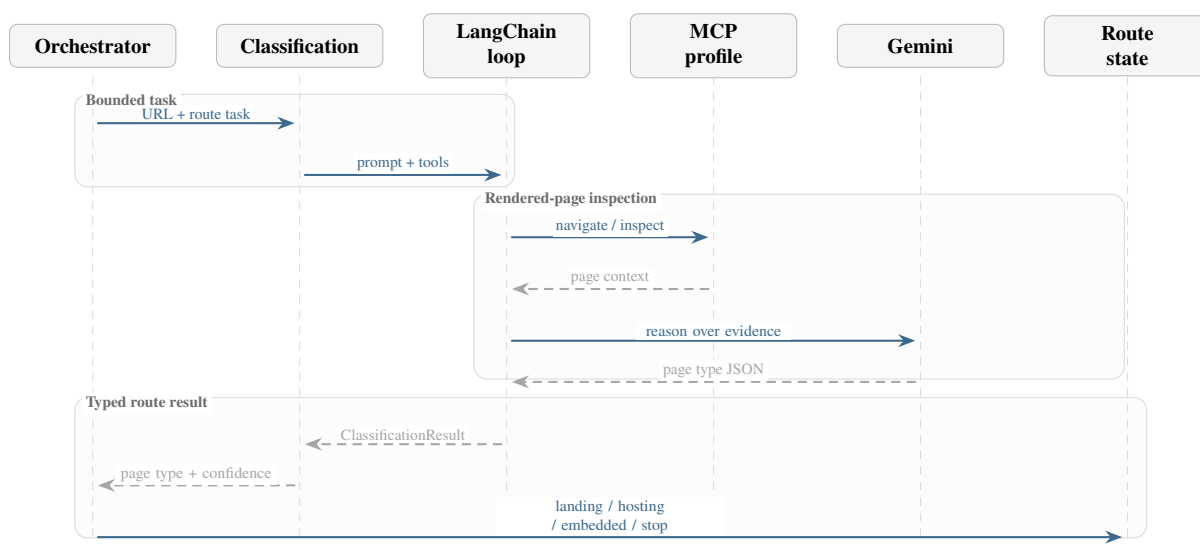


Figure 4.13: Classification-agent sequence: inspect page state and return a routing decision.

4.5.3 Landing-Page Agent

The landing-page agent turns broad site pages into bounded downstream targets. It discovers useful watch, hosting, embedded, or direct-stream candidates from homepages, schedule pages, directory pages, and match listings.

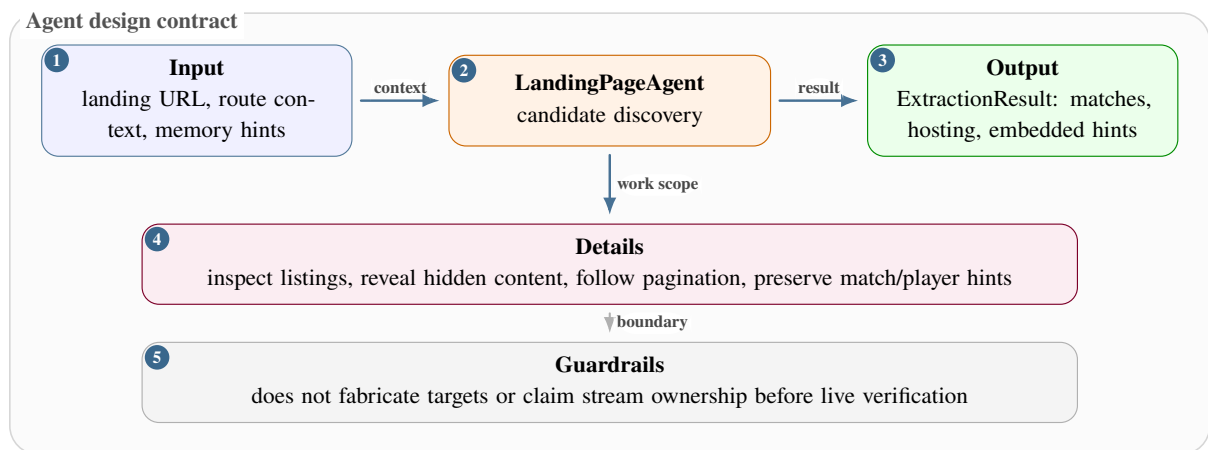


Figure 4.14: Landing-page-agent design card.

The orchestrator deduplicates landing outputs. Normal watch or server pages go to the hosting queue, direct player URLs go to the embedded queue, and provider-like media URLs are preserved as stream evidence.

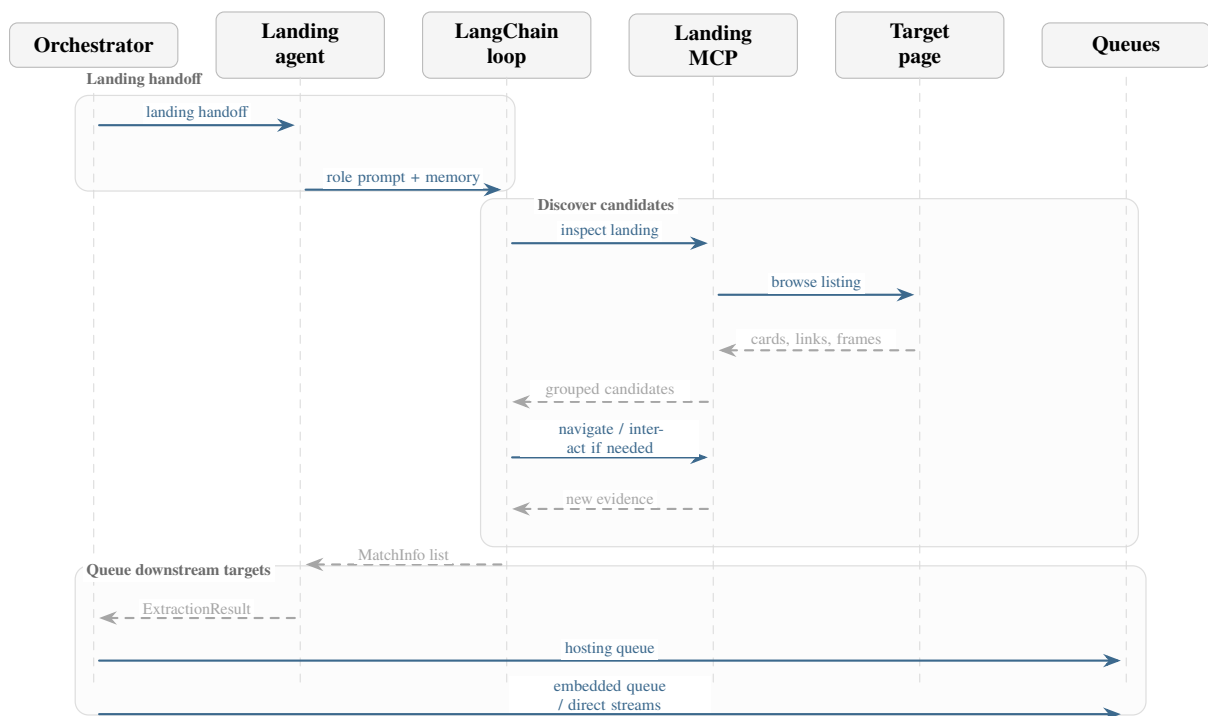


Figure 4.15: Landing-agent sequence: convert listing-page evidence into downstream queues.

4.5.4 Hosting-Page Agent

The hosting-page agent owns watch pages and server/source selection pages. It activates players, operates server controls, handles overlays, captures screenshots, and returns streams or explicit embedded-player handoffs.

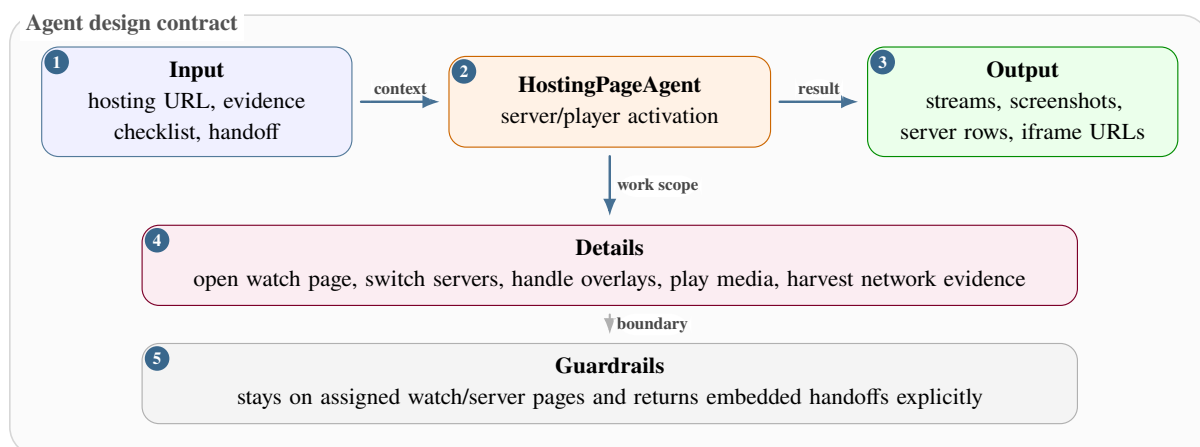


Figure 4.16: Hosting-page-agent design card.

If the page exposes an iframe or direct player URL instead of a stream, the hosting result carries that URL as an embedded handoff. If playback fails, it still returns diagnostics so the run explains why extraction stopped.

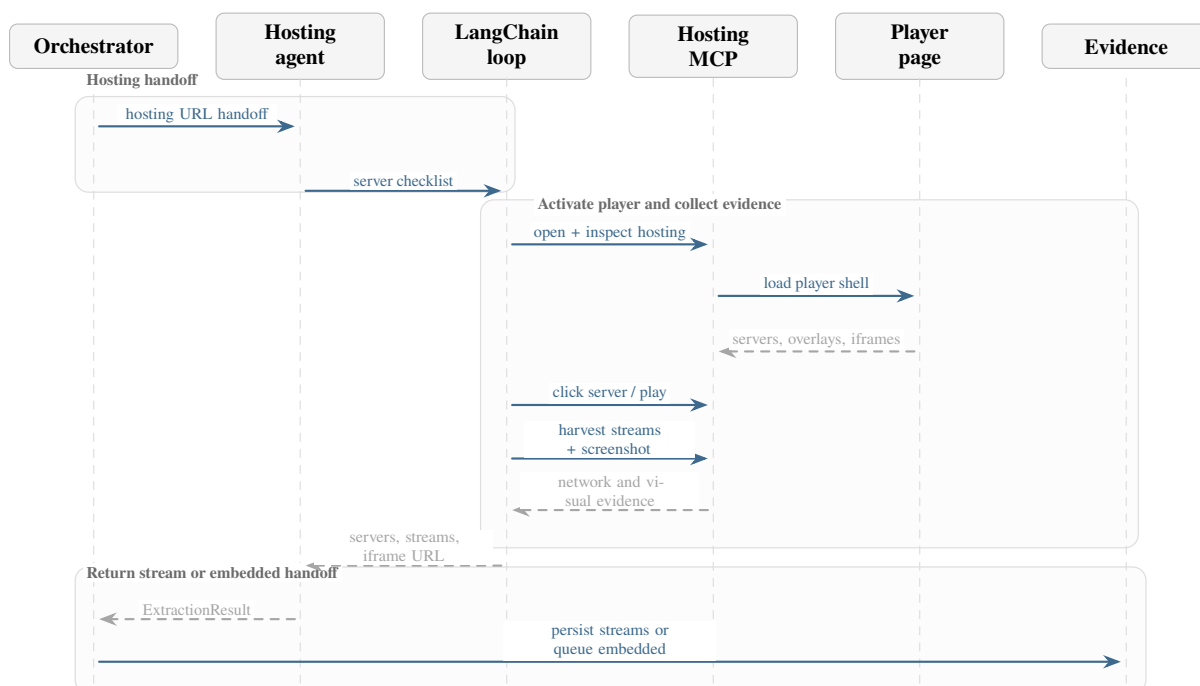


Figure 4.17: Hosting-agent sequence: activate watch pages and return stream or iframe evidence.

4.5.5 Embedded-Page Agent

The embedded-page agent owns direct player or iframe URLs. It stays inside the assigned player context, inspects frames, activates iframe-local controls, recovers from popup or redirect drift, and collects final media evidence.

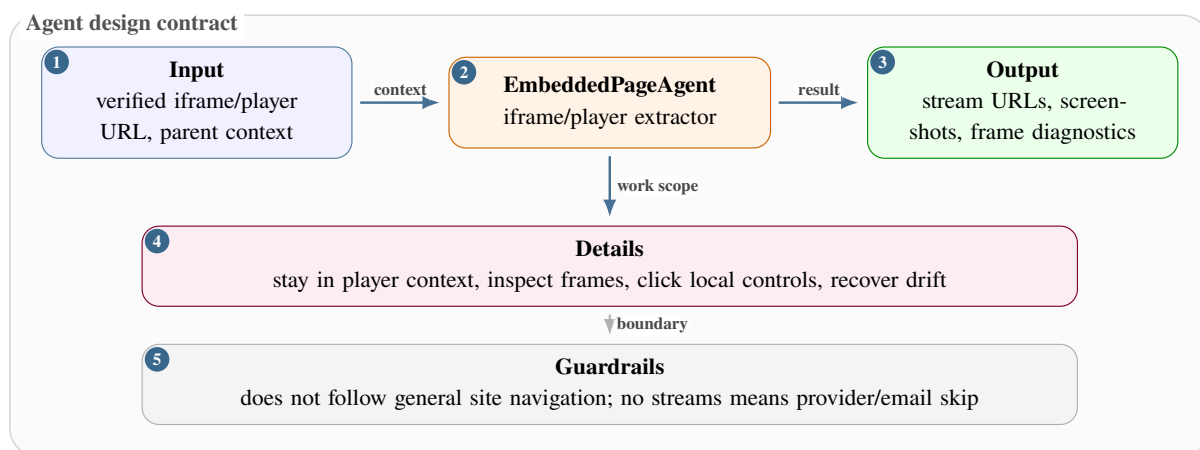


Figure 4.18: Embedded-page-agent design card.

The component card gives the embedded agent’s scope. The sequence diagram then shows the narrower runtime path: open the assigned player context, inspect frames, activate playback, capture evidence, and return the result.

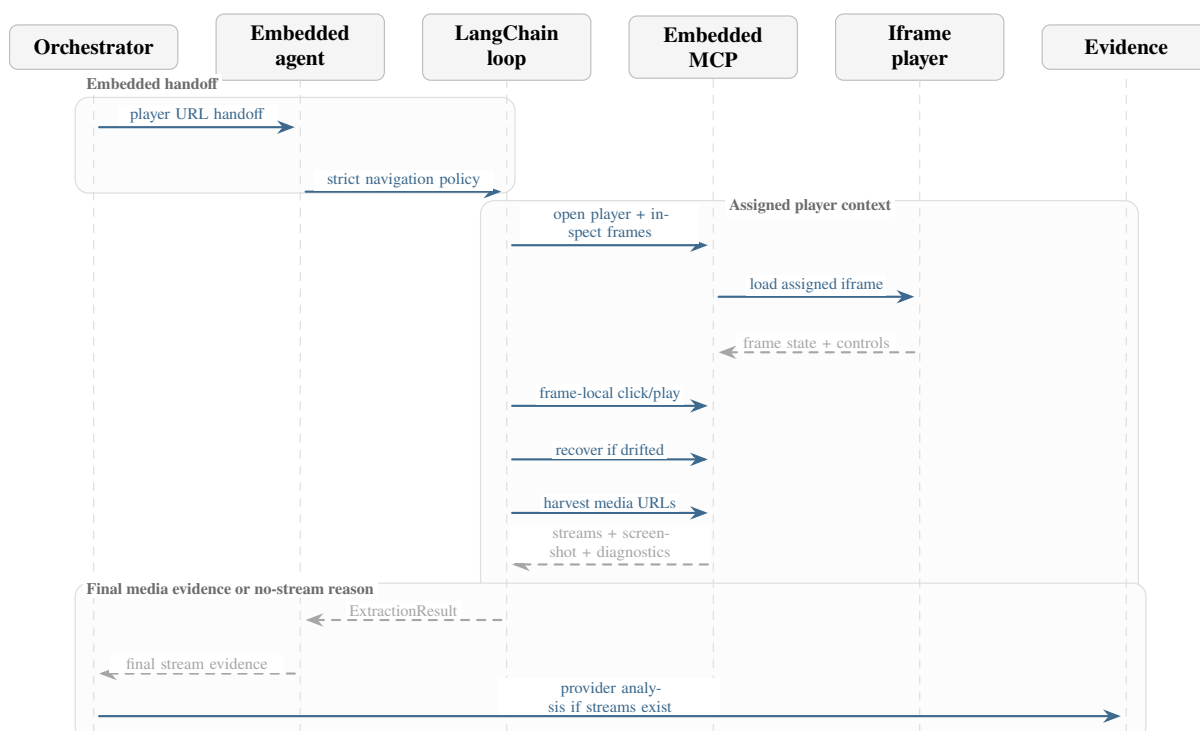


Figure 4.19: Embedded-agent sequence: stay in the player context and harvest final media evidence.

4.5.6 Channel Detection Across Agents

Channel detection is an agent responsibility, not a standalone browser tool. The tools expose page text, DOM structure, frame state, screenshots, server labels, and media requests. The agents decide whether those signals are strong enough to become evidence.

This distinction matters because a channel name printed on a landing page is only a hint. A channel label near an activated player or inside the embedded player context is stronger evidence.

The responsibility is shared across the route:

- classification preserves obvious channel text from the first page observation;
- landing keeps channel and match hints attached to candidate hosting URLs;
- hosting checks whether the selected server or player still matches the same channel;
- embedded confirms, refines, or rejects the signal from iframe-local text, player overlays, and screenshots.

The orchestrator then normalizes these observations as candidates, final channel fields, and confidence notes in the pipeline result.

Agent stage	Channel evidence it can read	How the signal is treated
Classification	Page title, visible headings, URL text, and broad page context.	Preserved as an initial hint, not accepted as final proof.
Landing	Match cards, league labels, event rows, revealed tabs, and candidate hosting links.	Attached to each downstream candidate so the route keeps its sports context.
Hosting	Server labels, player container text, active source state, screenshots, and network-adjacent player metadata.	Used to verify or override landing hints after the watch page is activated.
Embedded	Iframe-local DOM, player overlays, frame title, screenshot text, and final media evidence context.	Treated as the strongest page-level confirmation before provider analysis.

Table 4.4: Channel detection responsibilities across the specialist agents.

4.5.7 Provider Analysis and Email Generation

Provider analysis and email generation are downstream evidence-packaging stages. They are not page-navigation specialists. Provider analysis reads the collected stream URLs, resolves host or IP ownership, and returns provider information and abuse contact data. Email generation groups provider rows and stream evidence into reviewable takedown drafts. These drafts are not sent automatically; they are artifacts for human review.

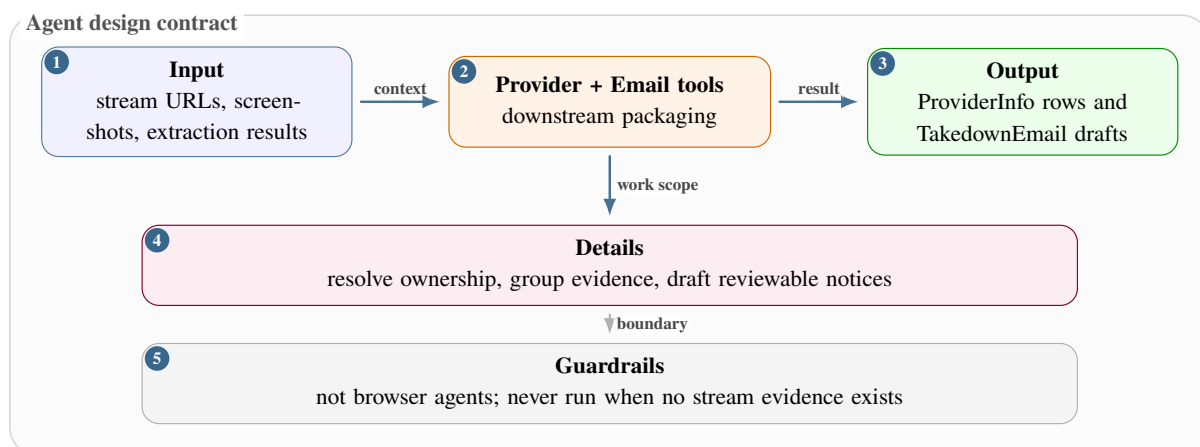


Figure 4.20: Provider-analysis and email-generation design card.

4.6 Browser Tooling and Runtime Controls

After the agent responsibilities are clear, the browser-tool layer explains how the agents observe and change real web pages. OWC does not ask an agent to reason from raw HTML alone. Illegal streaming pages are often:

- rendered by JavaScript;
- split across nested frames;
- guarded by overlays, popups, and server tabs;
- dependent on tokenized URLs or delayed media requests.

For that reason, every browser-facing specialist receives a profile-scoped MCP tool surface backed by Puppeteer and Chrome.

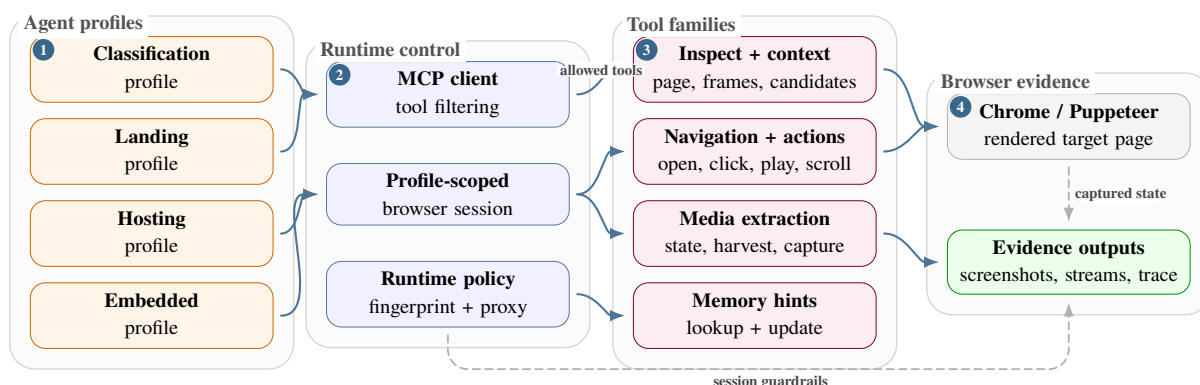


Figure 4.21: Profile-scoped MCP browser tooling and runtime controls used by OWC agents.

4.6.1 Tool Families

The Puppeteer tool layer is divided into tool families so that each agent can ask for the smallest useful observation or action. This keeps context growth under control: the model receives a compact view of the rendered page, then requests a narrower element, frame, media, or screenshot operation when it needs proof.

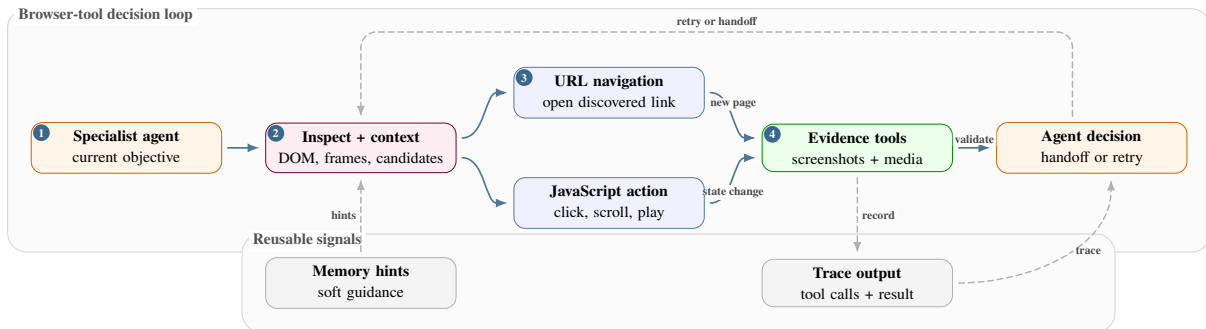


Figure 4.22: Global browser-tool flow.

4.6.1.1 Inspection and context retrieval

Inspection tools are the first read layer. They inspect the rendered Document Object Model (DOM), not only the first server response. Broad inspectors summarize visible text, grouped anchors, buttons, candidate cards, server controls, iframe trees, player containers, media clues, and screenshot links.

Page-type inspectors make that broad idea more precise:

- the landing inspector looks for match cards and hosting candidates;
- the hosting inspector looks for server/source controls and player state;
- the embedded inspector focuses on iframe-local player evidence.

Context tools are narrower. When an agent sees a possible target in a broad observation, it can request element details, a selector-specific subtree, a frame tree, candidate element references, or media state. This avoids repeatedly sending a large DOM summary back to the model. The architectural rule is simple: broad context is used to choose where to look, and scoped context is used before acting.

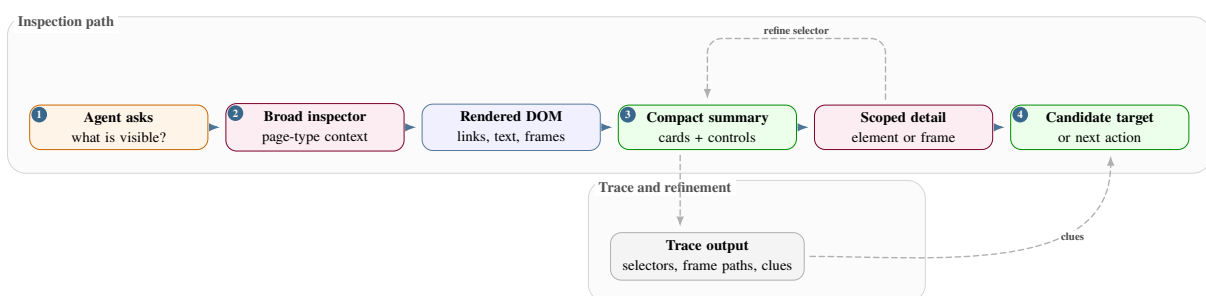


Figure 4.23: Inspection and context-retrieval flow from broad page observation to scoped targets.

4.6.1.2 Navigation and interaction

Navigation has two different meanings in OWC:

- **URL navigation:** open a real link or candidate URL, wait for the document to settle, then inspect the new page.
- **In-page interaction:** stay in the same browser context and click tabs, server buttons, scroll triggers, inputs, selectors, or playback controls.

Both paths must be followed by a new observation. A new document or a changed DOM state can invalidate the previous context.

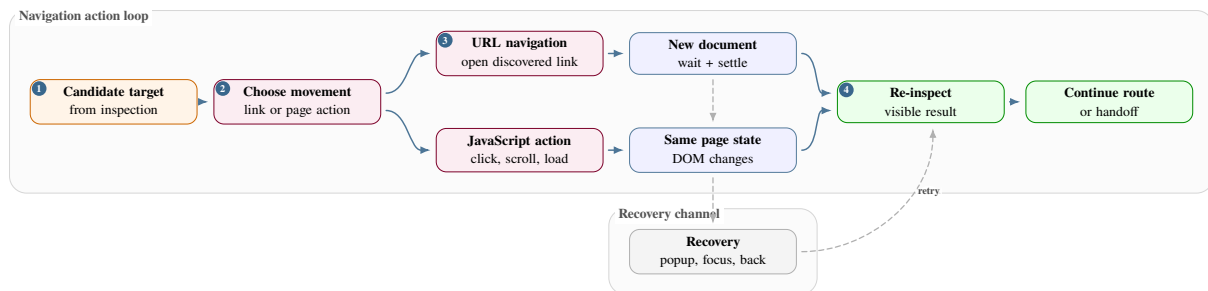


Figure 4.24: Navigation flow split between direct URL navigation and JavaScript-driven page interaction.

4.6.1.3 Evidence and media extraction

Evidence tools convert browser state into reviewable artifacts:

- screenshots show what the page looked like after navigation or interaction;
- media-state tools confirm player presence, playback state, and active frame changes;
- stream-harvesting tools collect m3u8, mpd, mp4, or related candidates from network and browser signals.

The agent still decides whether the candidate is acceptable evidence, but the tools provide the structured facts that make the decision auditable.

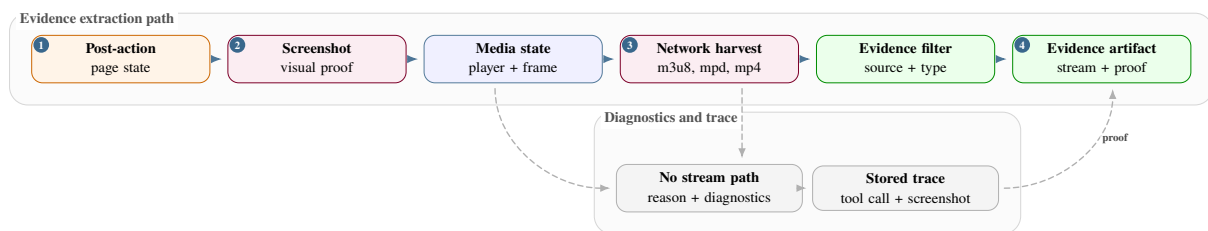


Figure 4.25: Evidence and media-extraction flow from post-action page state to reviewable artifacts.

4.6.1.4 Memory and site hints

Memory tools read and update reusable site hints such as known server tabs, useful selectors, common iframe patterns, and previous failure causes. They do not replace live browser evidence. A memory hint can tell an agent where to start, but the agent must still inspect the current page,

act through Puppeteer if needed, and validate the result with screenshots, player state, or stream evidence.

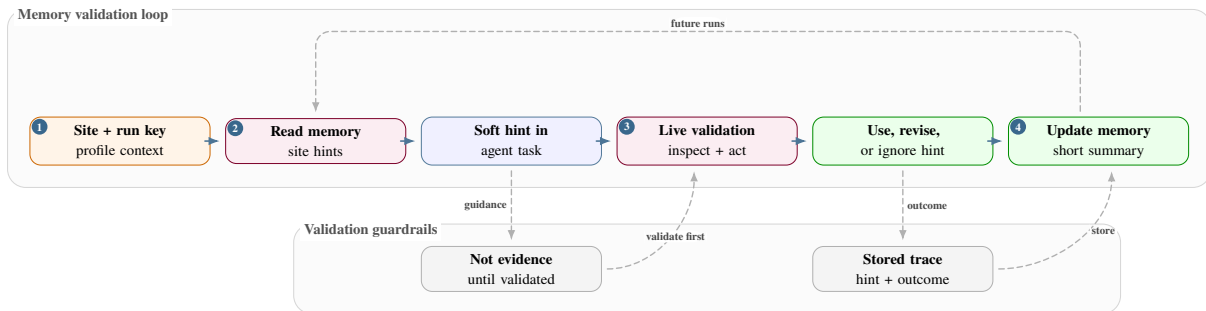


Figure 4.26: Memory-tool flow showing hints as guidance that must be validated live.

4.6.2 Fingerprint Handling and Proxy Rotation

OWC treats browser identity as a runtime setting rather than as prompt text. Browser profiles keep cookies, cache, and session state scoped to the current run or agent profile. Fingerprint handling controls properties such as browser identity, navigator surface, viewport-related behavior, and profile fallback so that repeated browser sessions do not expose an obviously inconsistent automation profile.

Proxy rotation is handled at runtime as a validated sticky strategy:

- candidate proxies are collected from configured remote or custom sources;
- each candidate is validated against a test endpoint;
- the selected proxy stays with the current browser profile while healthy;
- rotation happens after failure, not after every click.

Changing network identity mid-run can break cookies, session-bound player URLs, and tokenized media links. If no proxy candidate works, the runtime can fall back to direct browsing instead of incorrectly reporting the target site as unreachable.

Runtime concern	Architectural rule	Reviewable output
Tool scope	Each specialist receives only the tools needed for its page role.	Tool calls and profile identifiers attached to the run trace.
Browser profile	State is scoped by browser/profile so cookies, cache, and session-bound pages stay coherent during a run.	Profile, runtime settings, and browser events.
Fingerprint handling	Browser identity is hardened and rotated according to runtime policy, not improvised by the agent.	Runtime configuration and browser diagnostics.
Proxy rotation	Proxies are validated and kept sticky until failure; direct fallback avoids false site-down conclusions.	Proxy strategy, failures, and final navigation status.
Context retrieval	Agents request broad DOM summaries first, then scoped element, frame, or media details before acting.	Inspector output, selectors, frame paths, and observation events.

Table 4.5: Browser runtime controls that support reliable and reviewable evidence collection.

4.7 Data, Trace, and Reviewability View

The reviewability view is stored as normalized database records, not only as one large final JSON object. The central class is `PipelineRunRecord`. It stores:

- stable run identity and root URL;
- final status and high-level classification fields;
- aggregate evidence counts;
- token totals and cost totals.

Background jobs can exist before a pipeline row is finalized. Once the run is persisted, agent, telemetry, prompt, memory, evidence, provider, screenshot, and email records attach back to `pipeline_runs`.

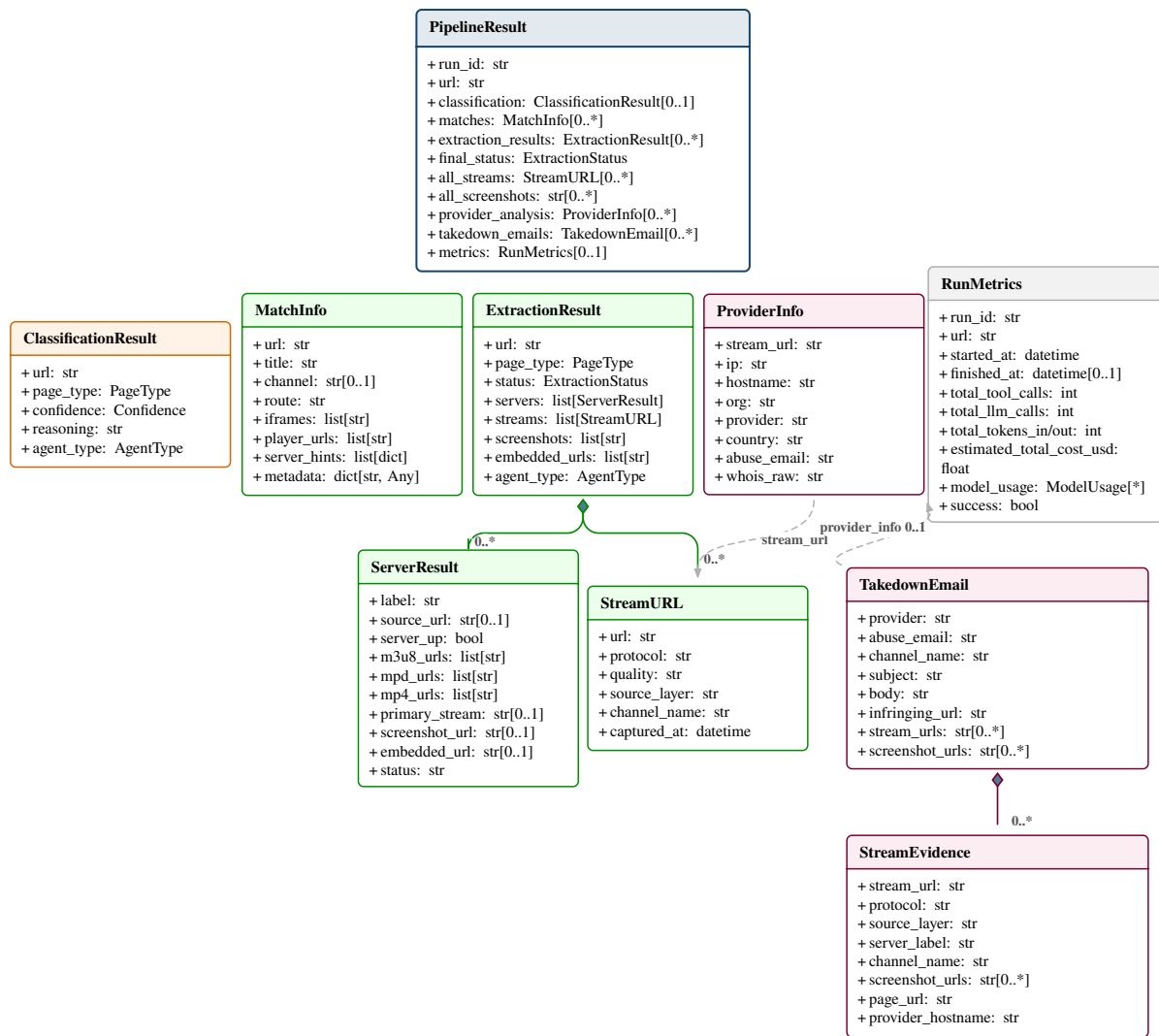
This is the database shape behind the run-detail console. The dashboard can show the final answer and the failure path because agent runs, model calls, browser tool calls, runtime events, prompt compilations, memory hints, streams, screenshots, providers, and email drafts are queryable as separate rows.

`PipelineResult` is the aggregate runtime object returned by the orchestrator. It groups classification, matches, extraction results, streams, screenshots, provider analysis, emails, final status, and metrics. In the Python runtime this object is convenient, but the report also needs to show why the final result is split into database tables that answer different review questions.

Figure 4.27 shows the runtime result as a class diagram. The diagram uses composition for objects owned by the aggregate result and a dashed dependency where an email draft references provider enrichment data. It is a runtime class diagram, not a database entity-relationship diagram.

Database table family	Representative tables	Architectural reason
Run identity and job state	background_jobs, pipeline_runs, run_snapshots, legacy_runs.	Keeps a durable job/run handle from launch to terminal status. The run row stores the root URL, page type, success flag, failure mode, counts, duration, token totals, and cost totals; the snapshot preserves a full fallback payload.
Agent timeline	agent_runs, agent_outputs, runtime_events.	Explains which actor ran, which target URL it handled, how many model and tool turns it used, what status it reached, and which event sequence led to the final decision.
Model and prompt telemetry	llm_calls, run_model_usage, prompt_versions, prompt_compilations, pricing_configs.	Makes model behavior auditable: provider, model, prompt hash/version, cache mode, token split, context-window use, and estimated cost are not hidden inside the final answer.
Browser and tool evidence	tool_calls, tool_playground_calls, run_screenshots.	Connects browser actions to reviewable evidence. Tool rows keep arguments, status, previews, errors, and timing; screenshot rows attribute images to the agent, invocation, tool, target URL, and event sequence that produced them.
Extracted evidence and review output	run_streams, provider_analyses, provider_lookup_checks, takedown_emails.	Separates concrete stream evidence from provider enrichment and draft notices. This lets a reviewer trace an email back to specific stream URLs, screenshots, server labels, and abuse-contact resolution.
Human review, evaluation, and memory	run_decisions, run_tasks, dataset_sites, dataset_batches, dataset_site_runs, memory_entries, memory_hints_used.	Supports manual follow-up, batch evaluation, and cross-run learning without letting memory or reviewer notes replace live browser evidence.

Table 4.6: Database table families used to turn an agent run into reviewable records.



PipelineResult lists top-level aggregate fields. Diamonds show nested ownership; dashed arrows show references.

Figure 4.27: Class diagram of the runtime result centered on PipelineResult.

The table structure is intentionally normalized because different reviewers ask different questions:

- “What stream was found?” points to `run_streams`.
- Hosting-stop questions point to `agent_runs`, `runtime_events`, and `tool_calls`.
- “What did this run cost?” points to `run_model_usage` and `llm_calls`.

This separation lets the dashboard show product evidence and operational traceability without treating one JSON blob as the only source of truth.

The contract between agents is intentionally simple. The model can inspect the page, use tools, and reason through a difficult browser state, but the value that crosses a stage boundary must be structured. This prevents the next agent from depending on vague prose such as “the second server looked good”.

Instead, the orchestrator passes a bounded `HandoffContext` containing:

- the target URL, page role, and route source;
- known match or channel context;
- navigation policy and required evidence checklist;
- memory hints that still need browser verification.

The summary below is a chain of contracts, not a chat between agents:

- classification produces a small routing object;
- landing turns a listing page into candidate watch targets;
- hosting captures streams or hands an explicit player URL to embedded;
- embedded works inside that player context and returns final media, screenshot, or blocker evidence;
- provider and email tools run only after stream rows exist.

This is why the final `PipelineResult` can point back to the screenshots and stream hosts that justify the review output.

A useful way to read the contract is to ask what the next stage is allowed to trust. It can trust typed fields such as `page_type`, `hosting_pages[]`, `embedded_urls_for_processing[]`, and `all_stream_urls[]`.

It should not trust:

- the previous model's private chain of thought;
- temporary browser impressions;
- a long natural-language summary that cannot be checked later.

This is why the handoff object carries both the evidence already collected and the checklist that still has to be proved.

This contract also makes failed runs easier to review:

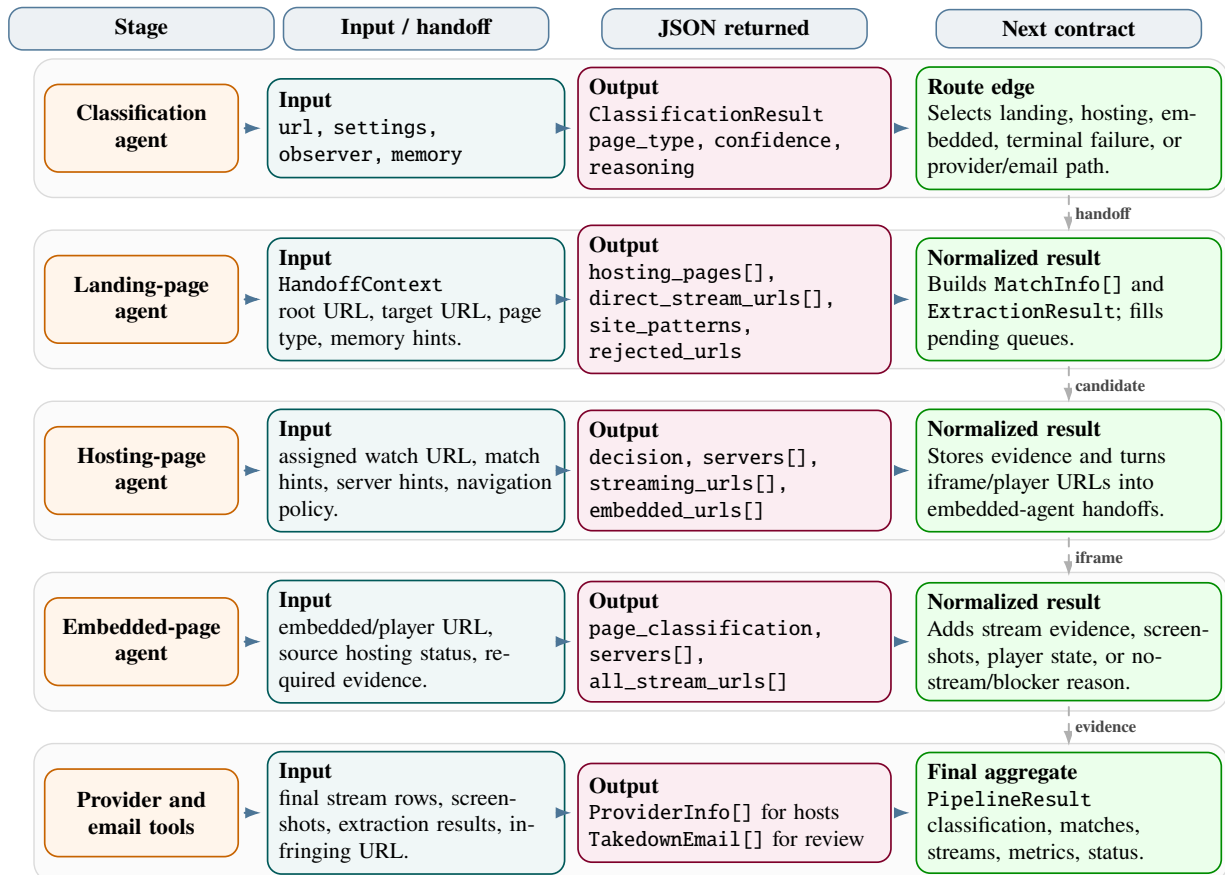
- if landing found schedule rows but no valid hosting target, the run stops with that specific reason;
- if hosting found an iframe but no stream URL, embedded receives the iframe as a new bounded task;
- if provider and email steps run, they start from persisted stream and screenshot rows, not from a vague claim that a stream existed.

The same structure is what makes the dashboard understandable to a reviewer. Instead of one large log blob, the run can be read as a sequence: route decision, landing candidates, hosting evidence, embedded-player evidence, provider identification, and draft email. Each step has an input object, an output object, and a reason for either continuing or stopping.

Stage	JSON input it receives	JSON output it must return	Why the boundary matters
Classification	Seed URL, runtime settings, observer, and memory hints.	<code>ClassificationResult</code> with <code>url</code> , <code>page_type</code> , <code>confidence</code> , <code>reasoning</code> , and <code>agent_type</code> .	Converts the first rendered browser state into a route decision instead of letting later agents guess the page role.
Landing	<code>HandoffContext</code> with root URL, target URL, upstream page type, classification reasoning, and memory hints.	Landing JSON with hosting pages, direct stream URLs, site patterns, and rejected URLs, normalized into <code>MatchInfo[]</code> and <code>ExtractionResult</code> .	Preserves match rows, schedule context, hosting links, and rejected candidates before the workflow moves to player pages.
Hosting	<code>HandoffContext</code> with assigned watch URL, match/channel hints, server hints, navigation policy, and required evidence checklist.	Hosting JSON with decision, server rows, stream URL rows, embedded-player handoff URLs, and channel metadata.	Separates server/player interaction from landing discovery and creates explicit embedded handoffs when the player is inside an iframe.
Embedded	<code>HandoffContext</code> with embedded/player URL, source hosting status, navigation policy, and evidence checklist.	Embedded JSON with <code>page_classification</code> , <code>servers[]</code> , <code>all_stream_urls[]</code> , <code>failed_servers</code> , <code>session_summary</code> , and channel fields.	Keeps iframe/player inspection focused; it can return stream evidence or a clear no-stream/blocker explanation without restarting discovery.
Provider and email	Final stream rows, screenshots, extraction results, and the original infringing URL.	<code>ProviderInfo[]</code> and <code>TakedownEmail[]</code> , then the final <code>PipelineResult</code> .	Turns browser evidence into review material while keeping the email a draft for human validation.

Table 4.7: Agent JSON contract summary used by the orchestrator.

Figure 4.28 expands the same chain into a full visual contract map. It makes the handoff rule explicit: each specialist receives a bounded JSON task, returns a typed JSON result, and leaves the orchestrator to normalize that result before another agent is called.



Boundary rule: each specialist reasons internally, but every handoff is a typed JSON contract. The orchestrator normalizes raw specialist JSON into the next bounded task and finally into PipelineResult.

Figure 4.28: Agent JSON contracts and orchestrator handoffs.

Architectural element	Boundary	Reviewable output
Orchestrator state	Current URL, classification, pending hosting and embedded queues, extraction results, provider rows, emails, and final error state.	Route decisions and final <code>PipelineResult</code> .
Handoff context	Root URL, target URL, page type, route source, evidence checklist, navigation policy, and memory hints.	Bounded specialist task context.
Runtime observer	Agent lifecycle, model calls, tool calls, screenshots, status changes, cancellation, and cost counters.	Events, rollups, traces, and metrics.
Persistence layer	Background jobs, run records, agent runs, evidence rows, screenshots, streams, provider rows, and email drafts.	The data behind <code>GET /ui/runs/{run_id}</code> .

Table 4.8: Runtime data boundaries and persisted review outputs.

4.8 Chapter Summary

The architecture can be read as a staged, reviewable evidence pipeline:

- the operator starts and reviews runs through the console;
- FastAPI controls execution, persistence, settings, and run-detail APIs;
- the orchestrator uses LangGraph to route between agents and preserve handoff state [11, 12];
- specialist agents use LangChain/LangGraph loops and profile-scoped MCP browser tools for rendered DOM context, frame inspection, screenshots, media state, and network evidence;
- browser runtime controls handle profile state, fingerprint behavior, proxy selection, popup recovery, iframe recovery, and screenshot capture;
- provider and email tools run only after streams exist;
- storage preserves both the final result and the failure path.

The next chapter maps these architecture boundaries to the concrete implementation.

Implementation

The implementation progressed in three major parts:

- **n8n prototype agents:** fast exploration of the four browser responsibilities: classification, landing, hosting, and embedded-player extraction.
- **Open Web Catcher:** the reference platform built around FastAPI, Next.js, MCP browser-tool containers, specialist agents, model/runtime management, and PostgreSQL observability.
- **Deployment direction:** the company kept its existing operational workflow and deployed only the n8n landing agent as a focused backup cloud job.

This split matters because the systems did not have the same purpose. The n8n phase tested four page-role agents: classification, landing, hosting, and embedded. OWC was built after those lessons, with cleaner engineering boundaries, persistent traces, configurable runtime settings, and a better foundation for evaluation. The deployment work then narrowed the cloud target to the part the company wanted to operationalize during the internship: the landing agent.

Chapter focus

Here I turn the architecture into concrete implementation work. It first explains the n8n prototype used to validate the browser-agent idea, then presents the OWC platform across FastAPI, Next.js, MCP browser containers, Puppeteer, specialist agents, prompts, model telemetry, memory, PostgreSQL observability, and the focused deployment path chosen for the company environment.

5.1 Part 1: n8n Workflow Prototype

5.1.1 Purpose of the n8n phase

The n8n implementation acted as the project playground [14]. It made it possible to test the core question quickly: can an LLM-driven browser agent inspect illegal streaming websites, navigate through landing and hosting pages, activate players, and recover useful evidence? At this stage, the priority was not a clean software architecture. The priority was proving feasibility on real, unstable websites.

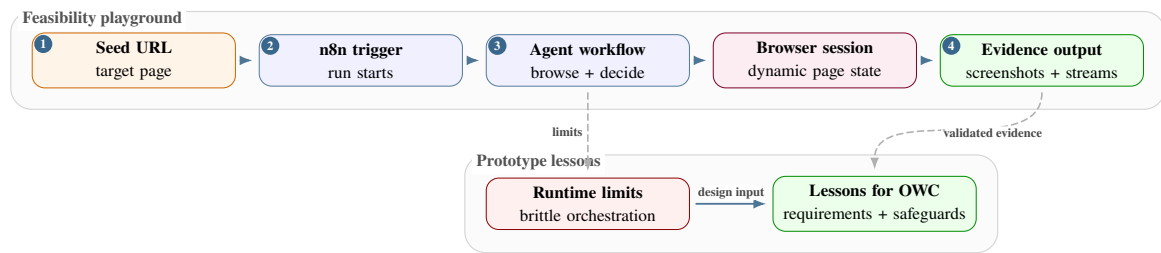
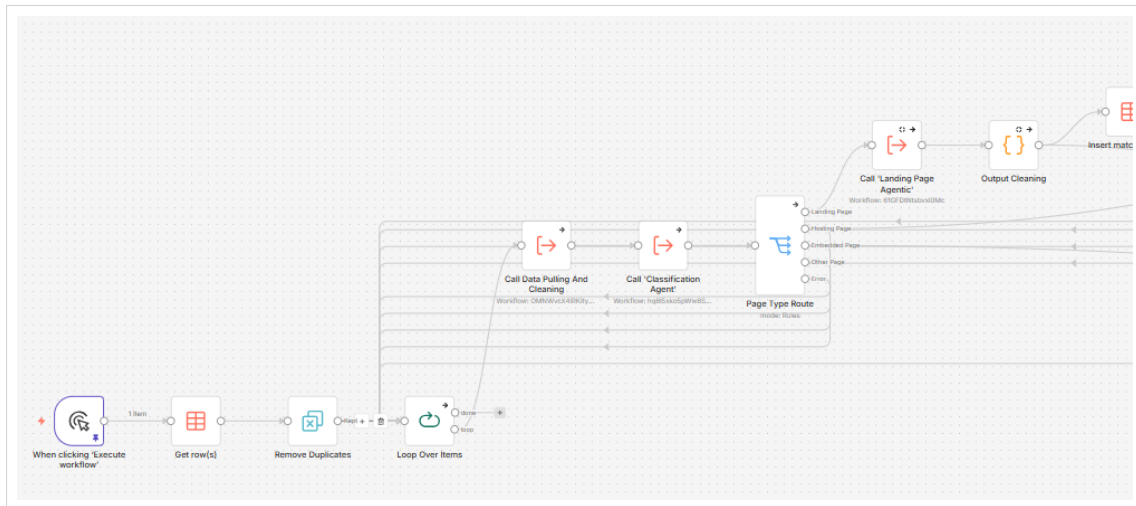


Figure 5.1: Role of the n8n workflow as a feasibility playground for browser-agent evidence collection.

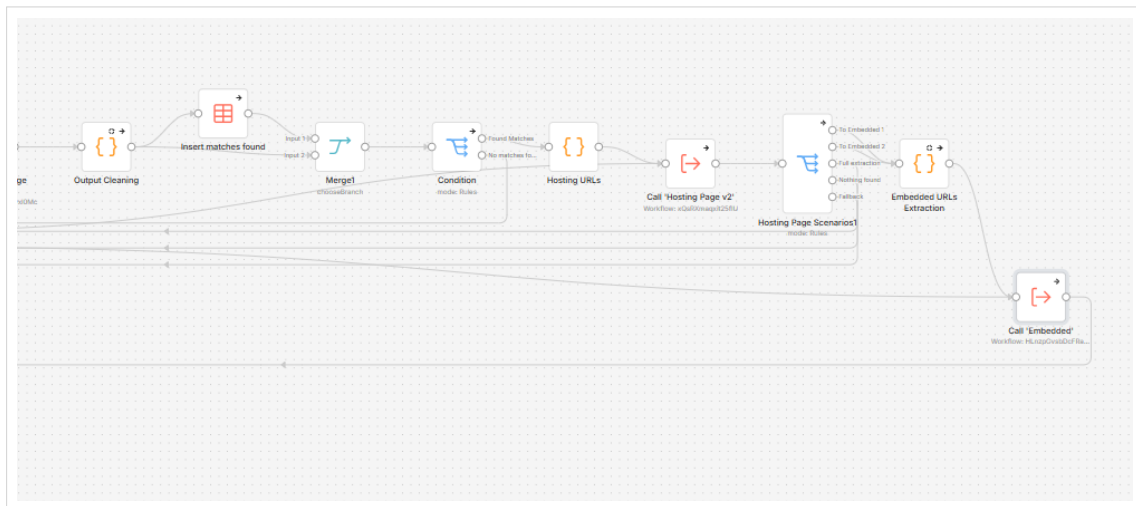
The workflow helped test agent behavior before investing in a full backend, database, and operator console. It also exposed which parts of the task were hard: browser state continuity, popup handling, stream activation, memory, runtime length, and evidence validation.

5.1.2 n8n prototype-agent screenshots

The n8n workflow used four agent-like branches to test the same page-type responsibilities that later became specialist agents: classification, landing, hosting, and embedded-player extraction. Figure 5.2 shows the full sequential workflow used during the prototype phase. The following figures give the prototype-agent screenshots in normal stacked layouts.



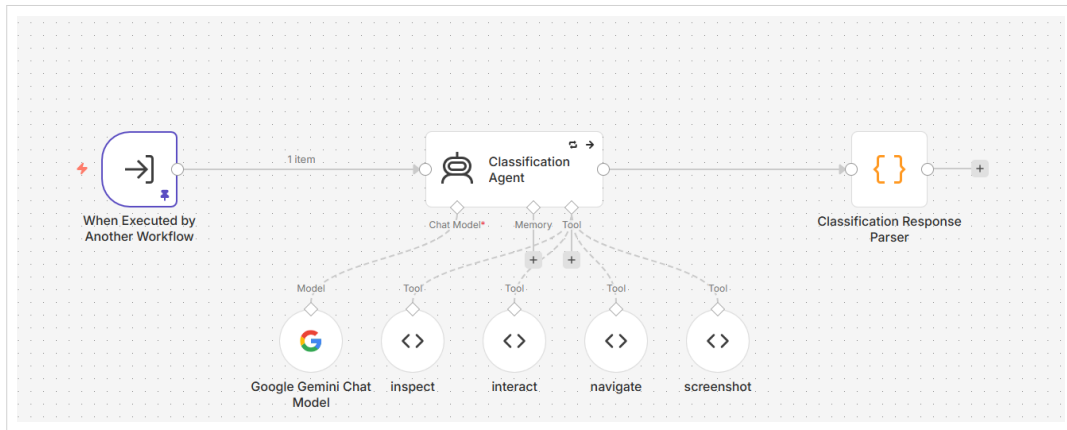
Workflow overview, first half



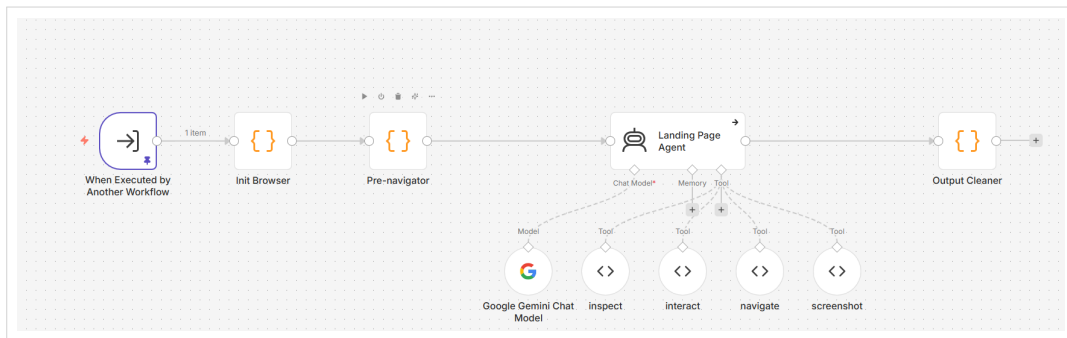
Workflow overview, second half

Figure 5.2: Full n8n prototype workflow.

The full workflow view gives the overall sequence, but it is too dense to explain each agent responsibility on its own. The next figure separates the first two prototype responsibilities: classification and landing-page exploration. The landing branch is the one later deployed as the operational backup; the other three remained prototype work that informed OWC.



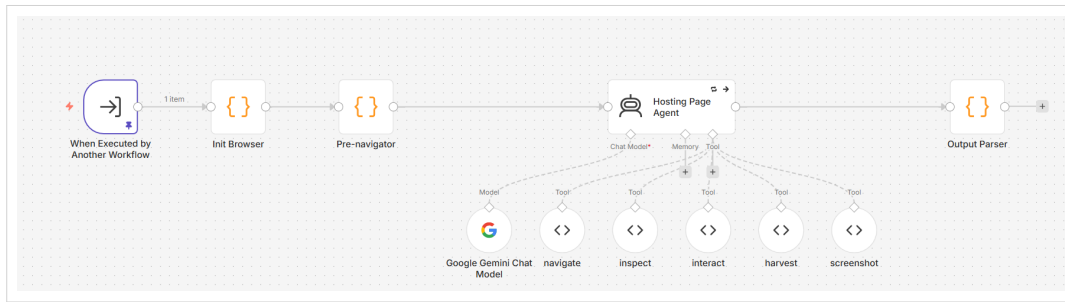
Classification workflow



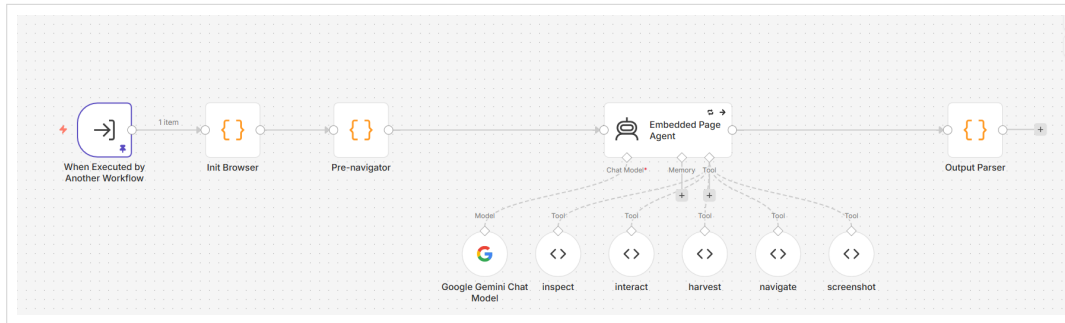
Landing-page workflow

Figure 5.3: n8n classification and landing workflow screenshots.

The prototype then moved from discovery into browser work. The hosting and embedded screenshots cover four checks: player activation, server handling, iframe inspection, and stream harvesting.



Hosting-page workflow



Embedded-player workflow

Figure 5.4: n8n hosting and embedded-player workflow screenshots.

n8n agent role	What it tested	Lesson learned
Classification	Whether a page could be categorized as landing, hosting, embedded, or other from browser evidence.	Routing must be explicit and fail-closed; wrong classification wastes the whole run.
Landing	Whether listing pages could expose match cards, tabs, and hosting links.	Generic DOM inspection was not enough; landing pages needed page-type-specific extraction.
Hosting	Whether server buttons and play controls could be activated.	Stream evidence appears only after interaction; screenshot and media-state validation are necessary.
Embedded	Whether iframe and player pages could reveal final manifests.	Nested frames require a dedicated extraction path and should not be mixed with landing-page exploration.

Table 5.1: n8n prototype agent roles and implementation lessons.

5.1.3 Limitations of the n8n implementation

The n8n workflow was valuable, but it reached clear limits when the task became larger than a single experimental run. These limits explain why the second implementation was necessary.

Limitation	What happened in practice	Consequence
No real evaluation layer	Runs could be inspected manually, but there was no structured evaluation harness for repeatable scoring.	Prompt changes were hard to compare objectively.
Weak memory model	The workflow did not provide a durable, domain-aware memory layer for reusable site knowledge.	Agents repeated failed actions and could not reliably reuse successful selectors or strategies.
Long runtimes	Large pages and multi-server hosting pages created long blocking executions.	Debugging became slow and failures were expensive to reproduce.
Sequential execution	The workflow behaved mostly as a sequential chain rather than parallel specialist agents.	Runs were slower than necessary, especially when multiple hosting or embedded candidates existed.
Limited development environment	n8n is a workflow editor, not a normal source controlled development environment.	Workflows had to be exported as JSON files to preserve progress and compare versions.
Fragile Puppeteer integration	Puppeteer code inside n8n nodes was brittle and tools frequently broke when browser state changed.	Browser automation needed a dedicated tool service with stable sessions.
Isolated code execution	Code executions were isolated, so browser instances could be lost between steps. A <code>puppeteer.connect</code> workaround was needed to reconnect to a running browser.	Browser continuity became a workaround instead of a clean runtime contract.
High memory usage	Long browser sessions, page artifacts, and workflow state created large memory pressure.	The prototype was not comfortable for long or batch-oriented extraction runs.
Hard failure analysis	Errors were visible, but not stored as normalized events, LLM calls, tool calls, screenshots, and costs.	Post-mortem review required manual reconstruction.

Table 5.2: n8n prototype limitations and engineering consequences.

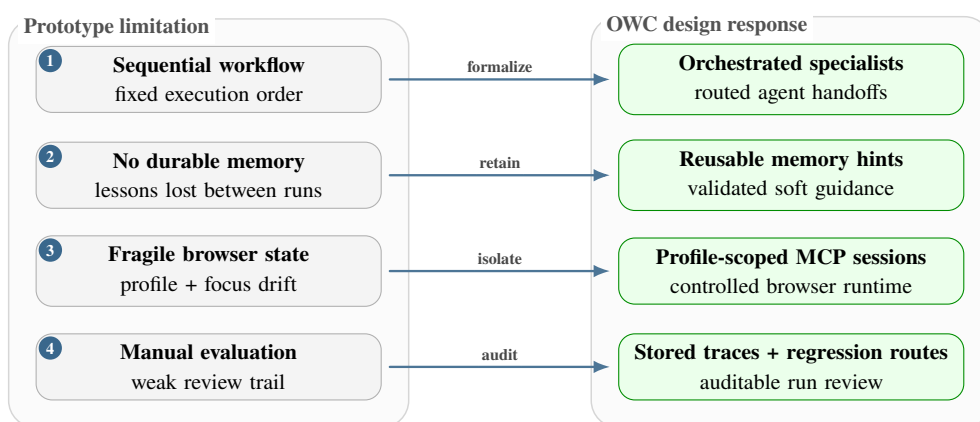


Figure 5.5: How n8n limitations shaped the Open Web Catcher implementation.

5.2 Part 2: Open Web Catcher Platform

OWC keeps the useful extraction logic from the n8n phase but moves it into explicit software components. It is organized into five main implementation areas: FastAPI, Next.js, MCP browser containers, agents and model runtime, and PostgreSQL.

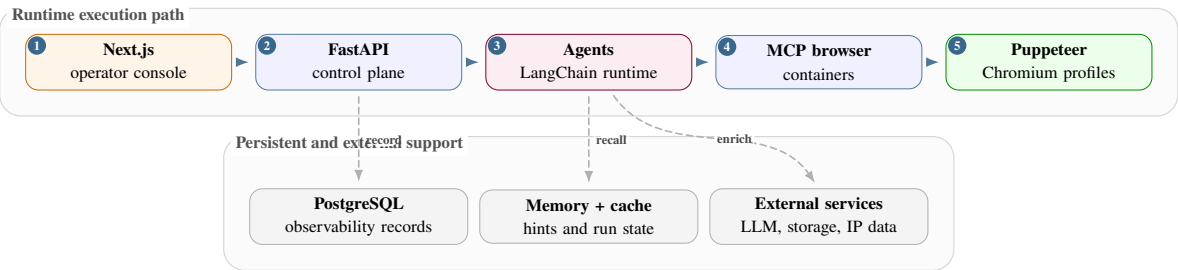


Figure 5.6: Five implementation areas of the Open Web Catcher platform.

5.2.1 FastAPI Backend

The FastAPI backend is the control plane of OWC [9]. It starts runs, checks runtime readiness, streams or returns run state, exposes settings, stores observability records, and gives the frontend a stable API surface.

Backend area	Responsibility	Why it matters
Runtime health	Check browser, MCP, settings, and launch readiness.	Failed preflight is clearer than a silently broken agent run.
Run execution	Start single URL runs and dataset-backed runs.	Keeps workflow launch separate from UI rendering.
Run inspection	Return trace, screenshots, streams, provider data, model usage, and errors.	Makes each run auditable after execution.
Configuration	Persist model, provider, pricing, browser, and tool settings.	Allows controlled experimentation without editing code every time.
MCP tool diagnostics	Expose profile-specific tool status and logged tool calls.	Separates browser/tool failures from prompt failures.
Evaluation routes	Run or review batches and reliability checks.	Provides the foundation that the n8n prototype lacked.

Table 5.3: Backend API responsibilities in OWC.

The backend also uses lightweight caching for frequently polled read endpoints. This reduces repeated database work when the frontend refreshes dashboards or run summaries.

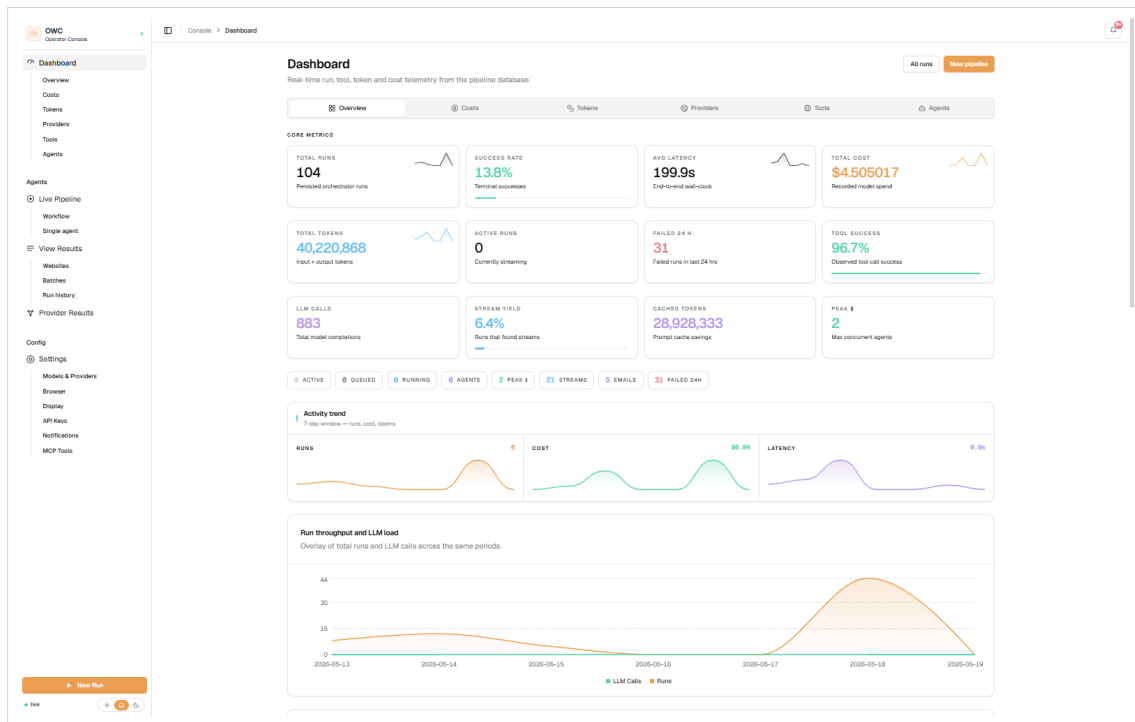
5.2.2 Next.js Operator Console

The Next.js frontend is not just a dashboard [10]. It is the OWC operator console for launching runs, following execution, reviewing evidence, and changing runtime configuration.

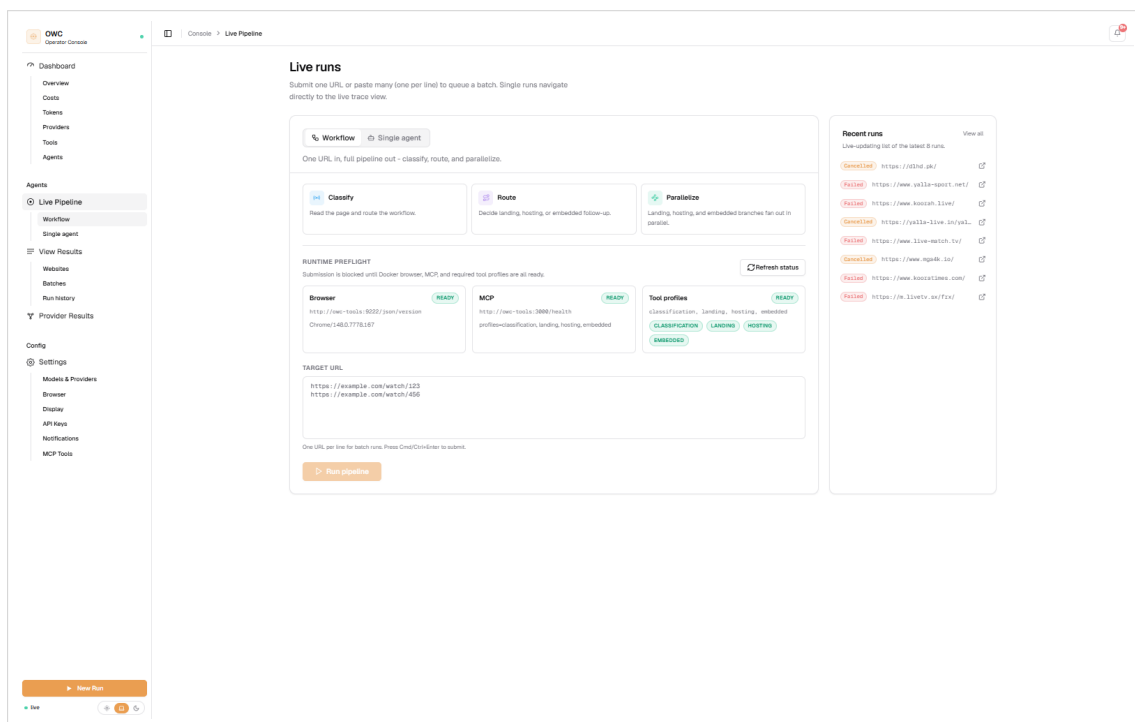
Frontend surface	What it shows or controls	Main backend dependency
Dashboard	Overview, cost, token, provider, tool, and agent telemetry.	Dashboard aggregation endpoints and persisted observability tables.
Live Pipeline	Workflow and single-agent launch with runtime preflight.	Workflow launch, health, and browser/MCP readiness endpoints.
View Results	Website dataset, batch execution, per-site status, and run history.	Dataset, batch, and run repositories.
Provider Results	Stream URL enrichment, IP resolution, provider distribution, country distribution, and manual lookup.	Provider-analysis records and lookup services.
Settings	Gemini model assignments, cache/runtime controls, browser settings, display preferences, API key status, notifications, and MCP tool enablement.	Configuration endpoints and runtime settings persistence.

Table 5.4: Current Next.js operator-console surfaces and backend dependencies.

The design goal was operational clarity. A failed launch should show why it failed. A completed run should show what the agent actually did, which tools it called, what evidence it found, and where the model or browser runtime failed.



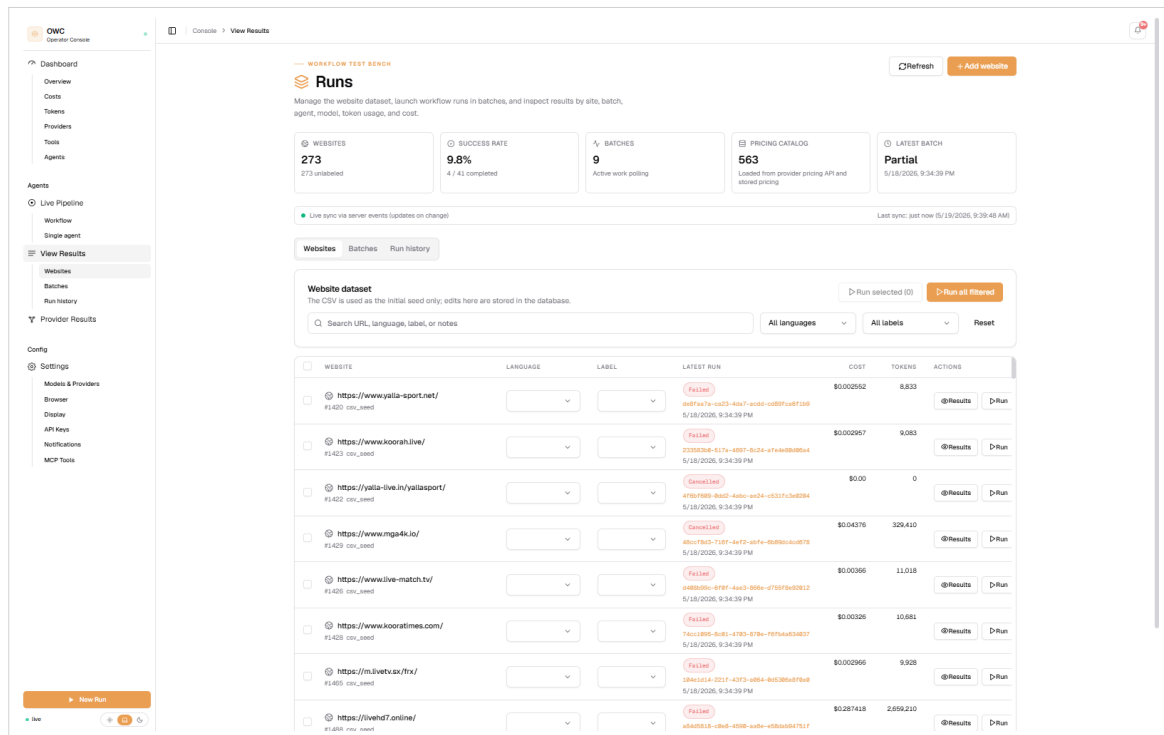
Dashboard telemetry



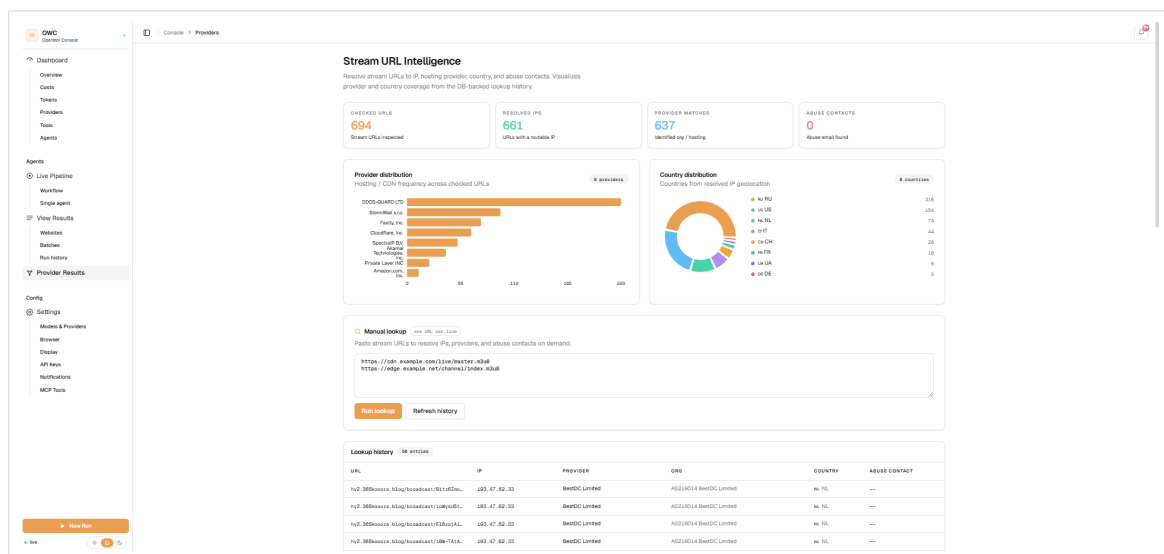
Live workflow launcher

Figure 5.7: OWC dashboard and live-run console screens.

The run detail and dashboard views make completed executions inspectable. The next console views focus on review and configuration screens, which are needed after a run has produced evidence.



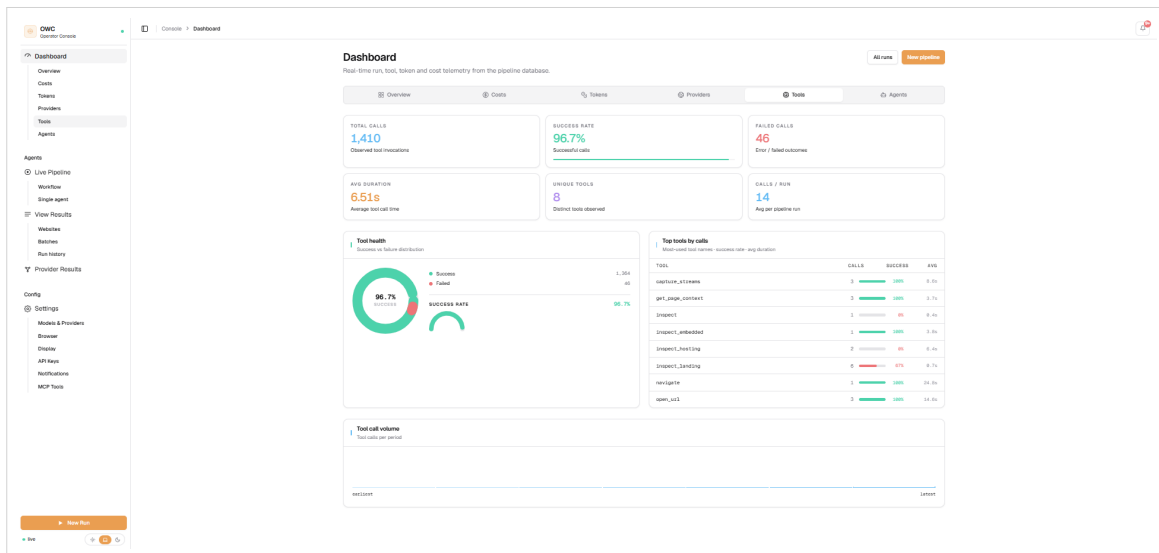
Run and website review



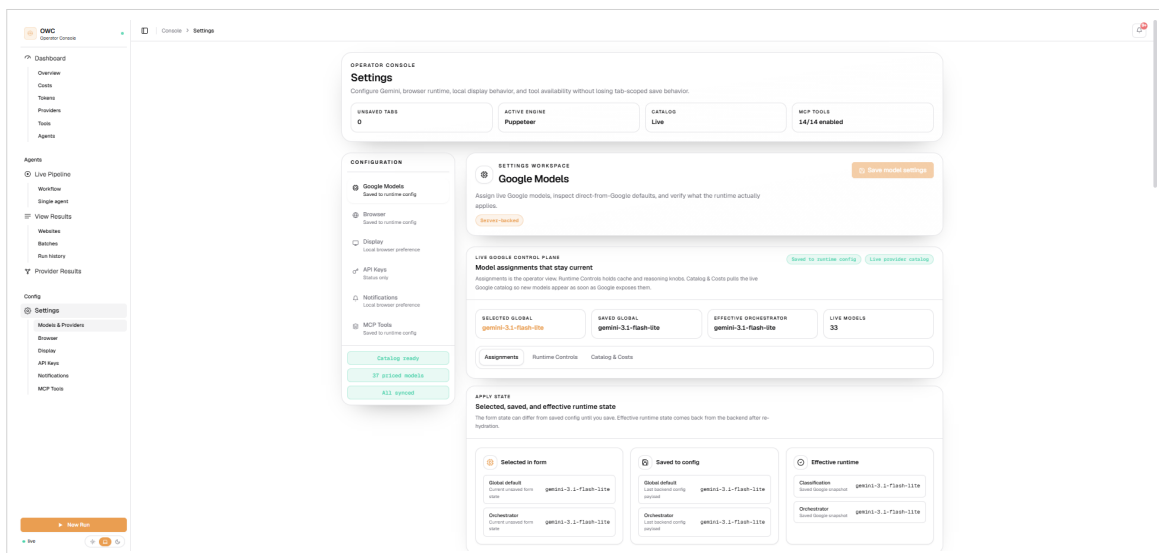
Provider enrichment review

Figure 5.8: OWC review screens for run results and provider enrichment, enlarged so tables remain readable in print.

The review and settings views expose the user-facing controls. The final console screenshot group focuses on browser-tool diagnostics, which are useful when a run fails because of tool availability or profile configuration.



Tool telemetry



Model and runtime settings

Figure 5.9: OWC tool telemetry and runtime configuration screens, enlarged for printed review.

5.2.3 MCP Browser Containers and Puppeteer Tools

The browser tooling was extracted from the n8n code-node style into dedicated MCP browser services [13]. Each agent profile receives a filtered tool surface through an MCP endpoint. This reduces tool-selection noise and keeps browser session ownership outside the LLM prompt. The engine comparison below is based on the official Playwright and Puppeteer references and the project’s integration constraints [5, 4, 6].

5.2.3.1 Tool registry and profile filtering

The MCP server exposes a larger internal tool registry than any one agent should use. During a run, OWC binds only the subset required by the current profile:

Browser engine	Strengths observed	Project decision
Playwright	Strong context isolation, good frame and browser-context APIs, useful for comparing behavior and validating browser policies.	Tested and kept as a useful reference path, but not the primary runtime.
Puppeteer	Direct Chrome DevTools Protocol (CDP) fit, simpler integration with the existing tool registry, stable enough for the profiled MCP tool server.	Chosen as the main browser engine for the implemented agent prompts and tool runtime.

Table 5.5: Browser automation engine comparison for the MCP tool layer.

Tool group	Purpose
Navigation tools	Open URLs, go back, wait for page state, and recover from navigation drift.
Inspection tools	Read page context, landing cards, hosting server controls, embedded player state, DOM elements, and frame trees.
Interaction tools	Click by selector, text, XPath, checkbox/radio/select, coordinates, play media, type input, scroll, and swipe.
Evidence tools	Capture screenshots, media state, stream requests, player signals, and harvested m3u8/mpd/mp4 candidates.
Memory tools	Read or update reusable site hints from the memory layer.

Table 5.6: Puppeteer MCP tool groups and responsibilities.

- **Classification:** broad page inspection and screenshot access.
- **Landing:** listing-page context, candidate extraction, reveal actions, scrolling, and element details.
- **Hosting:** server controls, playback activation, popup recovery, media state, and stream harvesting.
- **Embedded:** frame-focused tools that keep the run inside the assigned iframe or player URL.

This filtering prevents a landing task from drifting into provider analysis or a hosting task from rewriting the whole route.

5.2.3.2 Rendered DOM context retrieval

The implementation uses Puppeteer because the useful evidence is usually in the rendered browser state rather than the initial HTML response. Context tools execute inside the active Chrome page. They read visible text and DOM structure, group anchors and buttons, inspect iframe trees, and return compact JSON observations.

The agent follows a two-step pattern: broad page context first, then scoped element or frame detail before acting. This reduced repeated full-page dumps and made long runs easier to review because the trace shows why a selector, server tab, iframe, or candidate URL was chosen.

5.2.3.3 Puppeteer actions and evidence feedback

Action tools are implemented as browser-state changes followed by explicit evidence checks. A click, scroll, text input, server switch, play action, or popup recovery is not considered useful by itself. After acting, the agent must verify the result by:

- inspecting the page again;
- capturing a screenshot when the visual state matters;
- reading media state when a player is involved;
- harvesting network candidates after playback.

This is why the tool trace alternates between observation, action, and verification instead of showing one large scripted Puppeteer procedure.

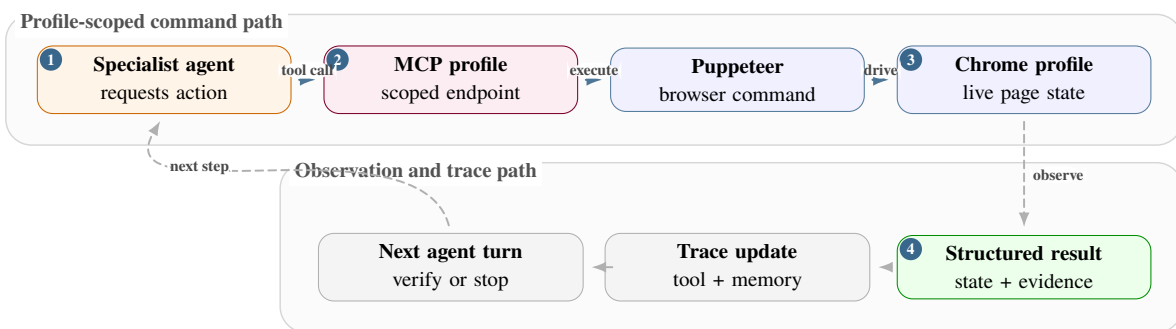


Figure 5.10: Browser tool execution loop through MCP and Puppeteer.

5.2.4 Agents, Models, Memory, and Prompts

OWC uses specialist agents rather than one general browser agent. Each specialist receives a role-specific prompt, a profile-specific tool set, and a bounded execution budget.

Agent	Objective	Main tools used
Classification	Decide whether the seed is landing, hosting, embedded, or other, while preserving obvious channel hints.	Generic inspection, screenshot, frame tree, memory lookup.
Landing	Discover match cards, listing tabs, pagination, and hosting candidates with match and channel context.	Landing inspect, element queries, click/scroll, screenshot, memory tools.
Hosting	Activate server controls, verify player state, and extract streams or embedded fallbacks while checking channel/player labels.	Hosting inspect, click/play tools, media state, frame tree, harvest, screenshot.
Embedded	Work only on embedded/player URLs and harvest final stream evidence.	Embedded inspect, play/media tools, harvest, screenshot, frame diagnostics, iframe-local text checks.

Table 5.7: Specialist agent objectives and assigned tool surfaces.

5.2.4.1 Design constraints implemented in the agent runtime

The agent runtime was shaped by constraints that became clear during prototyping. These constraints are implementation concerns, not only methodology notes, because each one changed how the agents, prompts, tools, memory, and stopping rules were built.

Constraint	Runtime effect	Implementation response
Changing layouts	Selectors and visible labels differ across languages, clones, and match-list layouts.	Agents use broad DOM context first, then scoped element details and page-type-specific tools.
Nested iframe players	The playable media source may not exist in the top-level page.	Hosting agents can hand off embedded URLs; embedded agents inspect frame trees and player-only pages.
Popups and ad traps	Clicks can open new tabs, steal focus, or hide the original player.	Browser tools restore focus, close popup tabs, and retry with safer JavaScript interactions.
Delayed stream generation	Stream manifests often appear only after a play click or server switch.	Prompts require media-state or screenshot verification before harvesting network evidence.
Cost and context growth	Repeated DOM observations, screenshots, and retries can make runs expensive.	The runtime tracks token usage, uses caching, and enforces bounded tool budgets.
Evidence isolation	The reference system must not depend on company production data.	Persistence is limited to traces, metrics, screenshots, tool calls, model usage, and structured outputs.

Table 5.8: Runtime constraints and their implementation responses.

5.2.4.2 Prompt evolution in the implementation

The prompts evolved from linear intent prompts into evidence-driven loops. The early prompt asked the model to solve the page in one pass. That worked on simple pages, but it failed on ambiguous layouts because the agent would stop before opening tabs, activating players, or verifying stream generation. The implemented prompt pattern instead forces each specialist to observe, reason, act, and validate before returning a result.

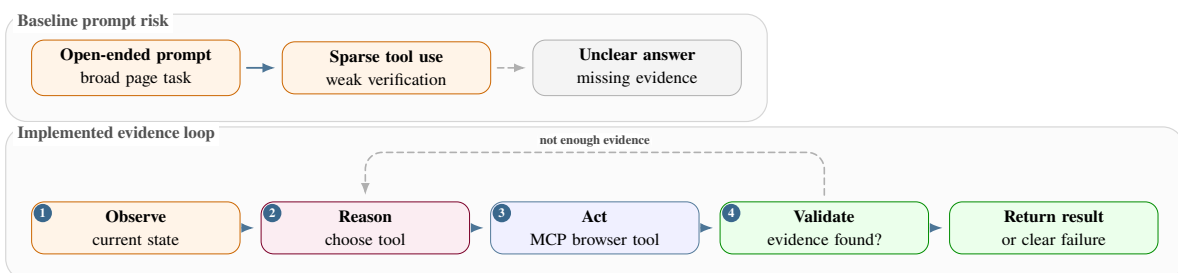


Figure 5.11: Prompt design contrast between an open-ended browser task and the evidence-driven agent loop used in OWC.

5.2.4.3 Agent design principles

The resulting agent design follows seven practical principles:

- classify the page role before deep extraction;
- separate operational n8n validation from the OWC platform architecture;
- ground every conclusion in browser evidence rather than textual guessing;
- preserve traces, screenshots, and structured outputs for each critical step;
- use specialist agents and explicit fallbacks instead of one unconstrained agent;
- optimize reliability and cost together through caching, budgets, and model routing;
- keep deployment loosely coupled across API services, browser workers, observability storage, and queue-based execution.

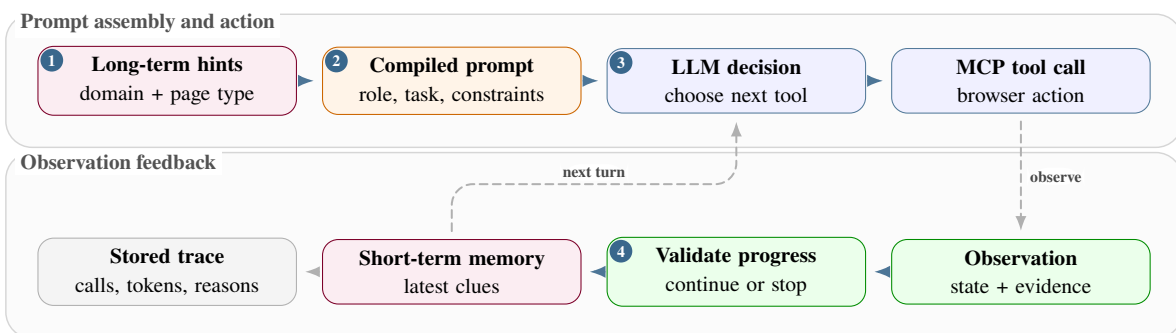


Figure 5.12: Agent reasoning loop with memory and tool feedback.

LangChain is used as the tool-binding and message-loop layer [11]. The implementation does not let the model freely access every browser capability. Instead, the MCP client loads only the tool profile for the current agent. The shared loop records tool calls, model calls, token usage, cache behavior, and stop reasons.

5.2.4.4 Model behavior, token usage, and rate limits

The model layer had to be treated as an operational dependency, not only as a prompting detail. Long browser pages, repeated DOM observations, screenshots, tool traces, and failed retries can grow the context quickly. The implementation tracks context windows, input tokens, output tokens, cached tokens, cache writes, and estimated cost.

Gemini rate limits are evaluated across RPM, TPM, and RPD. Google's current provider reference defines these as requests per minute, input tokens per minute, and requests per day, and states that limits depend on model and usage tier and can change in AI Studio [1]. The implementation treats rate limits as configuration and observability concerns rather than hard-coded constants.

5.2.4.5 Prompt engineering and stopping behavior

The prompts evolved from broad goals into operational procedures. Each prompt defines the objective, the evidence required, which page ownership rules must be respected, and when the

Model concern	Implementation handling	Why it matters
Context window	Store and display model context-window metadata where available.	Prevents long runs from silently approaching context exhaustion.
Token usage	Track input, output, cached input, and cache-write tokens.	Makes cost and prompt growth visible per run.
Tool calling	Bind only profile-specific tools through LangChain and MCP.	Reduces incorrect tool choice and keeps browser actions controlled.
Rate limits	Treat RPM, TPM, and RPD as provider limits to monitor and respect.	Avoids confusing quota failures with browser or prompt failures.
Retries	Apply bounded LLM retry attempts and delays per agent profile.	Recovers transient provider errors without infinite loops.
Caching	Use prompt/provider cache and tool-result cache where safe.	Reduces repeated tokens and repeated browser observations.

Table 5.9: Model-runtime concerns handled by the agent platform.

agent should stop. The strongest prompt pattern was an evidence-based loop: observe, state the current hypothesis, choose one action, verify the result, then either continue or return a structured failure.

Prompt design rule	Implementation effect
Explicit page ownership	Hosting agents stay on hosting pages; embedded agents stay on embedded/player URLs.
Evidence before harvest	Agents confirm player state or screenshot state before claiming stream extraction.
Bounded tool budget	Long runs stop with a clear partial result instead of looping indefinitely.
Structured output contract	Results can be stored, evaluated, and rendered in the UI without manual parsing.
Memory hints as soft guidance	Prior site knowledge helps, but does not replace current evidence.

Table 5.10: Prompt engineering rules enforced by the specialist agents.

5.2.5 PostgreSQL Database and Observability

The PostgreSQL database is used mainly for observability, review, evaluation, and configuration. It is not the product database of the external organization. Its purpose in this project is to preserve what happened during a run so that failures and successes can be inspected later.

Record group	Tables / records	Status in the project
Core run trace	Pipeline runs, agent runs, runtime events, LLM calls, tool calls.	Central to reviewing and debugging runs.
Evidence	Run streams, screenshots, provider analyses, takedown emails.	Central to the evidence bundle.
Model and prompt metadata	Prompt versions, prompt compilations, model usage, pricing configs.	Important for cost, prompt iteration, and reproducibility.
Memory	Memory entries and memory hints used.	Used to connect cross-run knowledge with current agent decisions.
Datasets and batches	Dataset sites, dataset batches, dataset site runs.	Useful for evaluation/batch execution, but not always used in every run.
Operational support	Run decisions, run tasks, tool-call diagnostics, snapshots.	Support/diagnostic records; some remain secondary rather than core report outputs.

Table 5.11: PostgreSQL observability records and their implementation role.

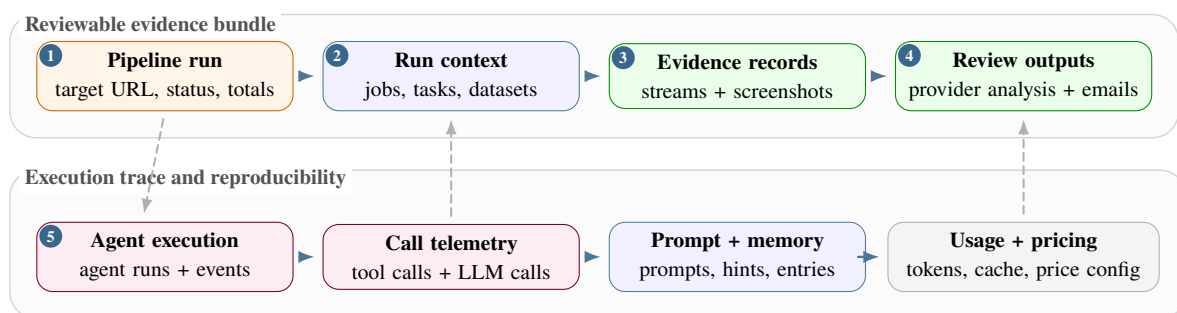


Figure 5.13: Summarized PostgreSQL persistence model derived from the detailed database table schema visualization.

5.3 Part 3: Deployment

The company deployment decision was intentionally focused. The existing company workflow remained the primary operational system, and the deployment did not attempt to replace it with the complete OWC platform or with all four n8n prototype agents. Technically, the internship produced classification, landing, hosting, and embedded n8n agents, but the deployed unit was only the validated n8n landing-agent backup: a browser-capable agent used when the primary flow cannot extract enough evidence from a dynamic landing page.

The deployment shape follows that practical boundary:

- the target URL and job identifiers enter the `agent_tasks` Azure Service Bus (ASB) queue;
- a focused Azure Container Apps Job runs the landing-agent worker with its Puppeteer MCP endpoint and Chrome browser runtime;

- the agent opens the rendered landing page and inspects visible match, channel, pagination, and candidate-link signals;
- the result returns through the configured results queue [15, 16].

The returned payload is meant for operational review and integration rather than automatic enforcement. It can include:

- matched URLs, match or channel metadata, and screenshots;
- rejected URLs with reasons and extraction summary values;
- reasoning logs, raw JSON for debugging, and failure information.

This keeps the company workflow, queue interface, and browser-agent backup loosely coupled. The primary process continues normal discovery, while the landing agent handles rendered-page work only for cases that need it.

This deployment choice matches the internship scope. OWC shows how a complete platform could be structured, but the company-side deployment remained focused on the backup landing agent as a cloud-executable single-container unit.

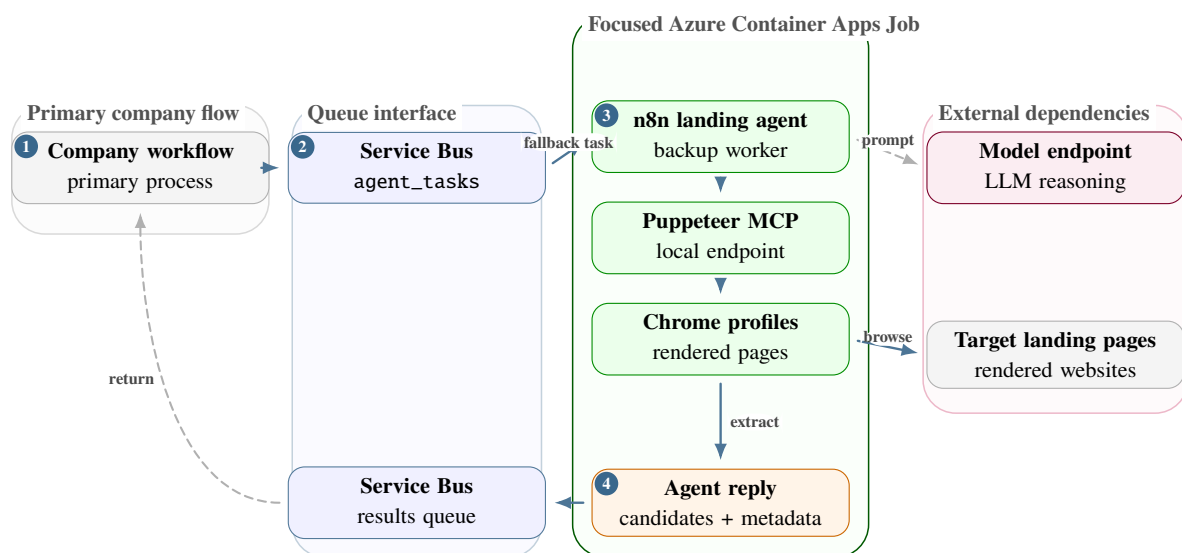


Figure 5.14: Company deployment flow using agent_tasks, a focused browser-agent container, and a configured results queue.

Deployment element	Role	Scope during internship
n8n landing-agent backup	Handles dynamic landing pages when the primary company workflow needs browser reasoning.	Deployed as the focused operational backup.
Azure Service Bus (ASB)	Decouples fallback submission from browser execution.	Used to pass landing-page jobs to the container worker and return results.
Container Apps Job	Runs one containerized agent unit when a message arrives.	Single image for the landing-agent worker, Puppeteer MCP, and Chrome.
Puppeteer runtime	Opens the page and extracts match/host candidates through the local MCP endpoint.	Runs inside the same job container as the landing agent.
Output queue	Publishes matched URLs, metadata, screenshots, rejected URLs, logs, and failure reasons to the configured results queue.	Used for review and integration back into the workflow.

Table 5.12: Landing-agent deployment scope and responsibilities.

5.4 Implementation Summary

The implementation moved from a useful but limited n8n prototype to the structured OWC platform. The n8n phase validated the feasibility of agentic browser extraction and exposed the main runtime problems. OWC addressed those problems with explicit backend routes, a reviewable frontend, profiled MCP browser tools, specialist agents, memory and caching, model-usage tracking, and PostgreSQL observability.

The result is not only an extraction script. It is a reviewable agent platform where failures can be diagnosed, prompts can be evaluated, tool behavior can be tested independently, and evidence can be inspected after the browser run ends.

Evaluation and Testing

I evaluate Open Web Catcher (OWC) as an evidence-production system for rights-protection review. I do not treat it as production-grade piracy detection or universal illegal-streaming coverage. The goal is narrower and more useful for this internship: check whether a suspected streaming page can produce a traceable evidence package.

That package is evaluated through:

- the route decision and specialist-agent activity;
- screenshots and stream evidence;
- provider intelligence and email readiness;
- model, tool, token, and operating-cost telemetry.

The evaluation follows the real review workflow. It starts with the business question, then moves through batch design, status taxonomy, specialist-agent checks, one concrete run, dashboard telemetry, cost metrics, and limitations.

This structure matters because final status alone is too weak. A website can fail because the agent behaved badly, but it can also fail because the site is dead, a hosting page is inaccessible, a stream server is down, or an embedded player exposes no media during the capture window.

Chapter focus

Here I evaluate OWC as a review-oriented evidence system. The chapter defines the business question, acceptance criteria, status taxonomy, KPI formulas, batch snapshot, agent-level checks, one detailed run case study, dashboard telemetry, cost evidence, operational limits, and the interpretation of the current results.

6.1 Evaluation Objective and Business Context

6.1.1 Business Issue

The practical business issue is manual uncertainty. A suspected live-sports streaming website rarely exposes all useful evidence on the first page:

- a landing page may only list matches;
- a hosting page may hide the player behind server buttons, popups, or delayed activation;
- an embedded page may be reachable only through an iframe or a selected hosting server;
- a playlist URL still needs screenshots, provider metadata, and a notice draft before a reviewer can use it.

In a beIN-context rights-protection workflow, the analyst needs an evidence bundle, not only a classification label. Soft Stars provides the internship engineering context, while beIN MEDIA GROUP provides the media-rights context in which live-sports piracy is treated as an operational protection problem.

6.1.2 Evaluation Question

The central evaluation question is:

Does OWC reduce manual uncertainty by producing a traceable, screenshot-supported, and provider-enriched evidence package that a human reviewer can inspect?

This question is more suitable than a generic accuracy score because OWC is an agentic browser system. The reviewer needs to know what the agents did, which page state was observed, which server path was attempted, what evidence was captured, and what remained uncertain.

6.1.3 Reviewable Evidence Bundle

For this report, a run is considered reviewable when it can provide a meaningful subset of the following evidence:

- seed URL and persisted run identifier;
- page-route decision and specialist-agent handoff;
- landing, hosting, or embedded-player navigation evidence;
- player activation, server switching, iframe inspection, or documented blocker evidence;
- stream evidence such as manifest, embedded, media, or playlist URLs;
- screenshots that validate the page, player state, server path, or failure condition;
- provider enrichment for the stream or playlist host;
- abuse-contact and takedown-email draft fields when available;
- trace, tool-call, token, cache, latency, and cost telemetry.

6.1.4 Evidence-First Evaluation

The chapter treats metrics as support for evidence review, not as a replacement for it. A strict success rate shows how many runs reached the strongest terminal status, but it does not explain partial evidence, website availability, server availability, or agent-level behavior. For this reason, the evaluation separates strict success, productive evidence, external blockers, agent/runtime failures, repeatability, and cost.

6.2 Evaluation Protocol and KPI Definitions

6.2.1 Batch Test Design

6.2.1.1 Input Set and Repeated Runs

The batch evaluation is organized around websites. A batch contains suspected streaming URLs with different page structures: landing pages, hosting pages, embedded players, redirecting pages, inactive pages, and pages whose server buttons change over time. Each URL is launched through the same pipeline: classification, specialist routing, evidence extraction, provider enrichment, and review-output generation.

The repeated-run idea is important for this domain. A website that works once may stop working later because a player server goes offline, a page redirects, the site changes its anti-bot behavior, or the stream is available only during a live event. Repeated repeated runs test not only initial success, but also whether evidence-producing sites remain evidence-producing after prompt, tool, browser, or website changes.

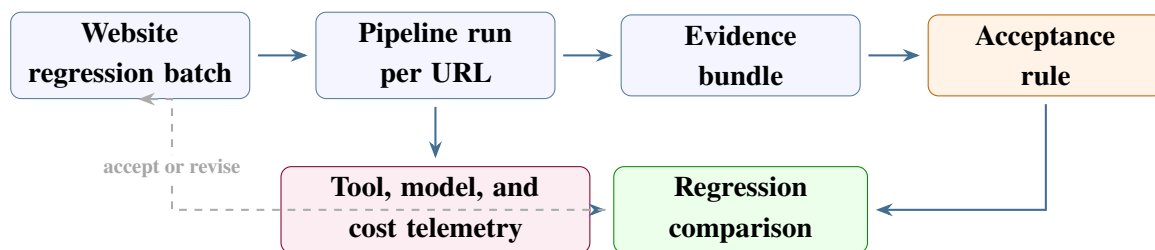


Figure 6.1: Evaluation protocol used for website-batch regression testing.

6.2.1.2 Manual Review After Automated Status

After each batch run, the automated status is reviewed together with the trace, screenshots, stream records, provider rows, and email records. This manual review step is necessary because the same terminal status can hide different causes. For example, a no-stream result may happen because the agent failed to activate the player, but it may also happen because all available servers were down during the run.

6.2.1.3 Website-Level Acceptance Rule

A website is accepted only when the run produces evidence that can be inspected. This is stricter than simply reaching a terminal state.

6.2.2 Failure Taxonomy

6.2.2.1 Why Failure Categories Matter

The dashboard strict success rate is useful, but it must be interpreted carefully. A low strict success rate may indicate weak agent behavior, but it may also reflect a difficult batch containing dead sites, inaccessible hosting pages, inactive streaming servers, and embedded pages that no

Checkpoint	Website is accepted when
Page classification	The page is routed to landing, hosting, embedded, or terminal handling according to observable browser behavior.
Landing discovery	Landing pages return relevant match-detail or hosting candidates, or the absence of candidates is explained.
Navigation	The agent reaches the next useful page instead of stopping at the first listing, redirect, advertisement state, or unsupported page state.
Player activation	Hosting or embedded agents interact with play controls, iframe contexts, or server buttons until a player state, media state, or blocker is observed.
Server switching	When server controls exist, the agent tests server paths in a reasonable order and records which path produced evidence or failed.
Network evidence	The run returns at least one useful manifest, embedded URL, media URL, or stream candidate when playback evidence exists.
Screenshot evidence	Screenshots support the final claim by showing the player, activation state, source page, server path, or failure condition.
Provider/email evidence	Stream or playlist evidence is enriched with provider metadata and, when available, abuse-contact or notice-draft fields.

Table 6.1: Website-level acceptance criteria used during batch evaluation.

longer load. These are still important operational results, but they are not the same as a bad classification or an incorrect browser-tool sequence.

Bucket	Status values	Count in snapshot	Interpretation
Productive evidence	success, partial	33 terminal runs	Full or partial evidence was produced. Partials should be reviewed separately instead of being hidden inside failure.
External or expected blocker	page_inaccessible, site_dead, no_hosting_pages, no_streams	22 terminal runs	The page, hosting route, embedded route, player, or stream server did not provide a usable path during the run.
Agent or runtime failure	failed, timeout, redirect	80 terminal runs	The run may have failed because of route choice, tool order, timeout, browser drift, model behavior, or runtime instability.
Excluded from quality score	cancelled	10 terminal runs	The run was stopped, so it should not be used to judge agent quality.
Active	queued, running	4 non-terminal runs	The run is still in progress and is excluded from terminal metrics.

Table 6.2: Failure taxonomy used to separate productive evidence, external blockers, agent/runtime failures, cancelled runs, and active runs.

6.2.2.2 External Blockers Are Not Automatically Agent Faults

This distinction is especially important for live-streaming pages. The trace may show external blockers rather than an agent fault:

- the site is down;
- the match page exists but hosting pages were removed;
- the hosting page is reachable but the selected server is down;
- the embedded player loads visually but produces no manifest request.

These cases still reduce operational yield. They should be recorded as availability or evidence-path blockers unless the trace shows that the agent skipped an available action.

6.2.3 KPI Definitions

6.2.3.1 Rates Used in This Chapter

The evaluation keeps several rates side by side. This is more transparent than forcing all outcomes into one success metric.

KPI	Formula and interpretation
Strict success rate	$success_runs/terminal_runs$. This is the dashboard's strict success view.
Partial rate	$partial_runs/terminal_runs$. This reports useful but incomplete evidence separately.
Productive evidence rate	$(success_runs + partial_runs)/terminal_runs$. This is the rate of runs that produced full or partial reviewable evidence.
External blocker rate	$external_blocker_runs/terminal_runs$. This identifies rows where website, hosting, embedded, player, or stream availability limited the result.
Agent-adjusted non-failure rate	$(success_runs + partial_runs + external_blocker_runs)/terminal_non_cancelled_runs$. This is not a success rate; it indicates how many non-cancelled runs did not end as agent/runtime failures.
Evidence extraction rate	$runs_with_streams/terminal_runs$. This measures technical stream evidence production.
Email yield rate	$runs_with_emails/terminal_runs$. This measures whether provider and notice fields reached email-draft readiness.
Tool success rate	$successful_tool_calls/observed_tool_calls$. This monitors browser-tool reliability separately from agent judgment.
Regression retention	$previously_accepted_still_accepted/previously_accepted$. This checks that prompt or tool changes do not break earlier working sites.

Table 6.3: KPI definitions used to interpret batch results without hiding partials or external blockers.

6.2.3.2 Current Batch Snapshot

The current dashboard/API snapshot contains 149 total runs, 145 terminal runs, and 4 running runs. The terminal status breakdown is 25 successful runs, 8 partial runs, 80 failed runs, 10 cancelled runs, 14 no-hosting-page runs, 6 no-stream runs, and 2 page-inaccessible runs.

I interpret the current snapshot in layers:

- strict success rate: 17.24%;
- partial rate: 5.52%;
- productive evidence rate: 22.76%;
- external blocker count: 22 terminal runs, or 15.17%;
- agent-adjusted non-failure rate without cancelled runs: $55/135 = 40.74\%$.

The last value is not an evidence success rate. It is a diagnostic rate showing that part of the raw failure picture comes from site and server availability rather than only from agent/runtime failure.

Metric group	Current value	Interpretation
Run volume	149 total runs, 145 terminal runs, 4 running runs.	Active runs are kept outside terminal-quality interpretation.
Strict success	25 successful runs, 17.24% strict success.	This is the strongest terminal-success view, but it excludes partial evidence.
Productive evidence	25 success and 8 partial runs, 22.76% productive evidence.	Partials are incomplete but still useful for review and debugging.
External blockers	2 page-inaccessible, 14 no-hosting-pages, and 6 no-streams.	These rows can come from source-site, hosting-page, embedded-page, player, or stream-server availability.
Agent/runtime failures	80 failed runs.	These rows need trace-level diagnosis because they may indicate route, tool-order, timeout, model, or runtime problems.
Cancelled runs	10 cancelled runs.	Cancelled runs are excluded from agent-quality judgment.
Latency	763.632 seconds average latency.	Browser-agent runs are expensive in time because they navigate, interact, wait, and verify unstable pages.

Table 6.4: Current batch outcome snapshot used for Chapter 6 interpretation.

6.3 Agent-by-Agent Behavior Evaluation

6.3.1 Agent Test Matrix

6.3.1.1 Why Agent-Level Evaluation Is Required

The agent pipeline is made of specialist responsibilities. Because of that, a whole-run label is not detailed enough. The selected run used in this chapter demonstrates the point: the embedded-page agent failed, but hosting-page agents still produced valid stream, screenshot, provider, and email evidence. The evaluation checks the behavior owned by each agent instead of treating the whole workflow as one black box.

Agent	Tests performed	Passing evidence
Classification	Route correctness, confidence, redirect/blocker recognition, and handoff to the right specialist.	The chosen route matches visible page behavior and the next agent receives the correct task.
Landing	Match-card discovery, schedule/list extraction, duplicate-link filtering, host-candidate discovery, and no-hosting-page handling.	Relevant hosting or match links are returned, or the absence of a hosting route is clearly recorded.
Hosting	Player activation, popup survival, server detection, server switching order, screenshot timing, iframe handoff, and stream capture.	A player path, screenshot, stream candidate, iframe target, or clear server/player blocker is returned.
Embedded	Iframe access, direct player inspection, delayed media harvest, embedded-page inaccessible handling, and no-stream outcomes.	Media evidence is harvested, or the embedded blocker is explained without hiding it as a generic failure.
Provider and email	Playlist-host attribution, IP/provider lookup, country and organization enrichment, abuse-contact lookup, and notice-draft readiness.	A provider row and reviewable email fields are attached to concrete stream evidence.

Table 6.5: Agent-specific tests used during manual and batch evaluation.

6.3.2 Classification and Landing Tests

6.3.2.1 Classification Tests

Classification is tested first because it decides the rest of the workflow. A correct classification result should identify whether the seed page behaves like a landing page, hosting page, embedded page, redirect, inaccessible page, or unsupported state. The review checks the route, the confidence, the visible evidence used for the decision, and whether the next agent receives the correct handoff.

6.3.2.2 Landing Tests

Landing-page tests check whether the agent can move beyond a listing page. The reviewer looks for:

- match cards, event links, and watch buttons;
- schedule sections and pagination;
- host candidates or other downstream routes.

A landing page is not considered a failure just because it does not contain a stream. It becomes an agent problem only if useful hosting candidates were visible and missed. If the page exposes no usable downstream route, `no_hosting_pages` is the more accurate status.

6.3.3 Hosting Tests

6.3.3.1 Player Activation

Hosting-page tests focus on interaction. The reviewer checks whether the agent clicks the correct player controls, survives popups, returns to the player when focus is stolen, and waits long enough for the page to generate media activity. The screenshot must be taken after a meaningful state change, not before the player has been activated.

6.3.3.2 Server Detection and Switching

Many streaming pages expose several server choices. The hosting agent is expected to detect server controls, try them in a reasonable order, and record which server path was used. If server 1 is down but server 2 works, the agent should continue instead of stopping too early. If all servers are down, the result is a server/player availability blocker rather than automatically an agent failure.

6.3.3.3 Stream Capture Order

Stream capture is evaluated after activation and server switching. This order matters: harvesting network traffic too early can produce empty results even when the page would generate a playlist after a play action. A passing hosting result connects the browser action, player state, screenshot, and stream evidence in the trace.

6.3.4 Embedded Tests

6.3.4.1 Iframe and Player Inspection

Embedded-page tests check whether the agent can inspect iframe/player contexts without mixing them with landing-page exploration. The agent should identify the embedded URL, inspect the frame or player state, attempt playback where appropriate, and harvest media requests after the page has had time to load.

6.3.4.2 Embedded-Page Inaccessible Cases

An embedded page can be inaccessible even when the hosting page is reachable. It can also load visually but never expose a playlist during the capture window. The evaluation records these as embedded availability or no-stream outcomes unless the trace shows that the agent skipped an obvious available action.

6.3.5 Provider and Email Tests

6.3.5.1 Provider Intelligence Criteria

Provider analysis starts from concrete stream evidence. Playlist URLs are useful because their host identifies the infrastructure that can be used for provider lookup and abuse contact discovery. The provider test checks host extraction, DNS/IP resolution when available, autonomous-system or organization enrichment, country fields, and whether the result is attached to the correct run and stream record.

6.3.5.2 Email Readiness Criteria

The generated email is not treated as an automatic enforcement action. It is evaluated as a draft. A draft is review-ready when it contains the provider/contact, infringing page, playlist host, stream evidence, screenshot references, and a clear need for human verification before sending.

6.3.6 Agent Dashboard Evidence

The agents dashboard supports behavior-level evaluation by separating agent runs, tool calls, LLM calls, context tracking, and status by agent type. It helps locate where a batch is struggling: classification, landing discovery, hosting activation, embedded inspection, or provider/email enrichment. It does not replace manual review, but it turns manual review into a more targeted process.

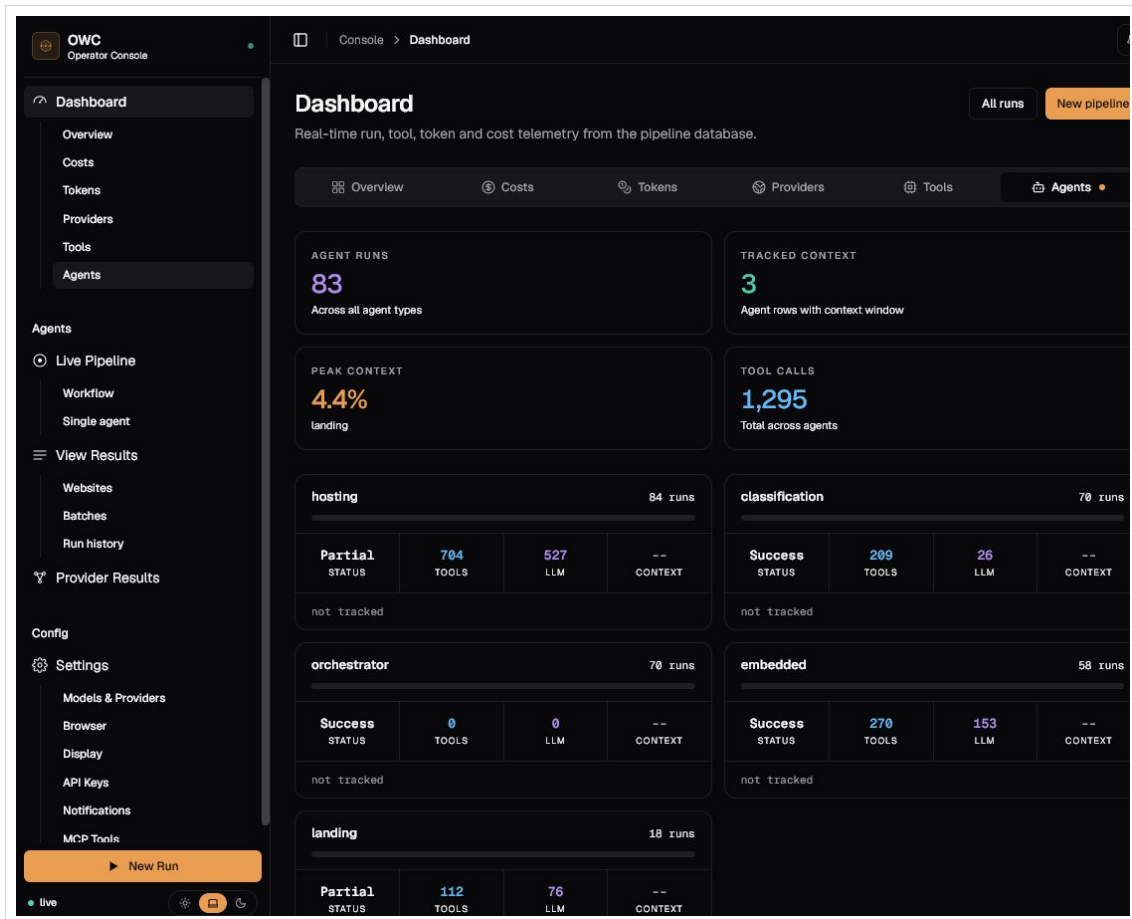


Figure 6.2: Agents dashboard for behavior evaluation.

6.4 Run-Based Evidence Case Study

6.4.1 Selected Run Summary

The main case study uses run 9566a9be-12ba-434a-afa0-f05f529d2403. It was available from the local backend and operator console during evidence capture, so no fallback run was used.

The run summary screenshot validates that the selected case is a concrete persisted run, not only a manually written example. It shows the final status, run identity, timing, model/tool profile, and evidence counters in the operator console.

Field	Observed value
Seed URL	go4kora.life
Run date	May 23, 2026
Duration	302.91 seconds, approximately 5 minutes and 3 seconds
Final status	success
Top-level page type	landing_page
Agent runs	6 agent runs: orchestrator, classification, landing, two hosting attempts, and embedded
LLM and tool footprint	15 LLM calls and 25 tool calls
Evidence produced	2 HLS playlist URLs, 2 screenshots, 2 provider-analysis rows, and 1 email draft
Token footprint	503,127 input tokens, including 430,132 new input and 72,995 cached input, plus 9,636 output tokens
Estimated cost	\$0.12381188

Table 6.6: Selected-run metadata used as the main Chapter 6 evidence case study.

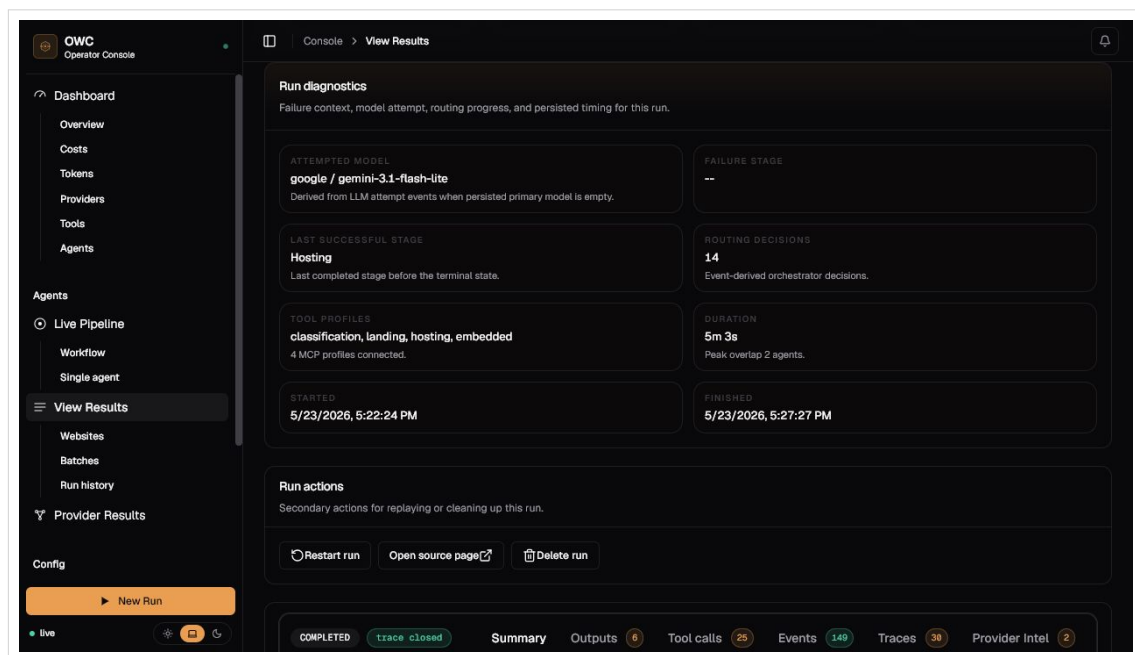


Figure 6.3: Selected run summary and status.

6.4.2 Trace and Agent Activity

The trace view is used to reconstruct the path from seed URL to final evidence. In this case, the trace shows the landing-page route, downstream discovery, hosting attempts, and persisted specialist-agent activity. This is also where the embedded-agent nuance is visible: the embedded path did not succeed, but the hosting path still produced enough evidence for the run to be useful.

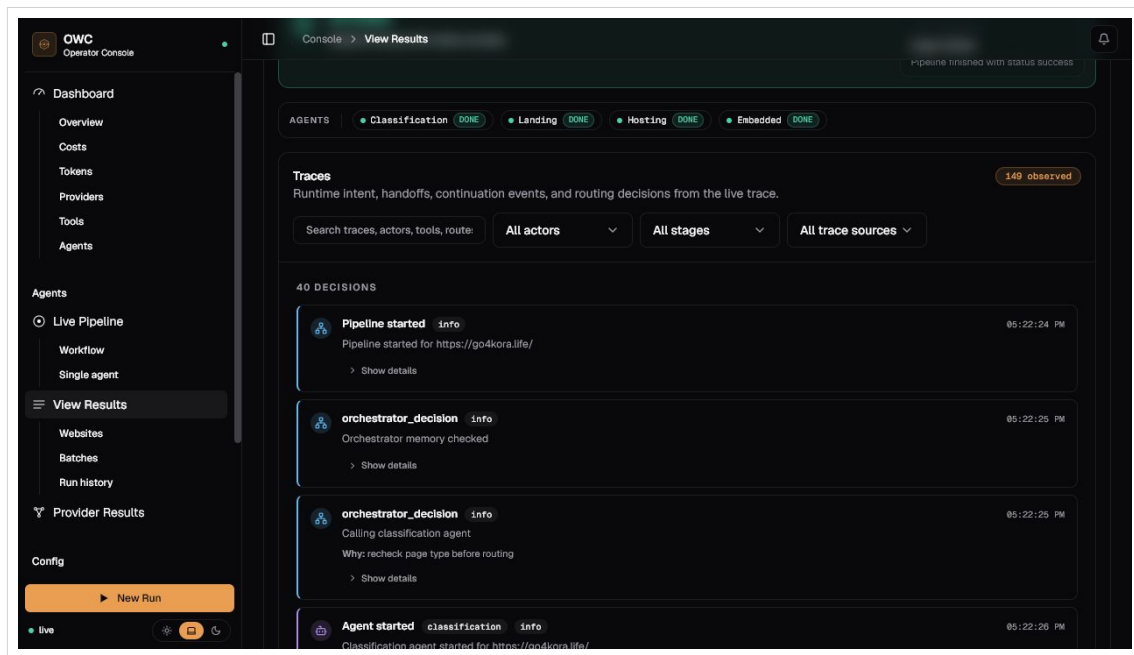


Figure 6.4: Trace rows and agent activity for the selected run.

6.4.3 Landing Evidence

The landing screenshot explains why specialist routing was needed: the first page is only a match/listing context. The relevant test is whether the landing agent discovers a useful hosting or player path instead of stopping at the surface-level page label.



Figure 6.5: Landing-page evidence for the selected run.

6.4.4 Hosting and Player Evidence

The first hosting screenshot validates player activation. It shows that the workflow reached a sports video/player state on the discovered path. This supports the hosting checks for navigation, activation, screenshot timing, and media-harvest context.



Figure 6.6: Hosting/player evidence for the selected run.

The second hosting screenshot records another captured player state from the same evidence path. Server behavior can vary during a run, so multiple screenshots make it easier to review whether a server was active, stalled, blocked, or unstable.



Figure 6.7: Additional hosting evidence for the selected run.

6.4.5 Stream, Provider, and Email Evidence

The streams/provider view confirms that stream evidence was persisted together with source labels and screenshot support. This connection is important: the playlist URL is not an expiring token claim in this report; it is persistent review evidence because the playlist host can be used for provider attribution and email preparation.

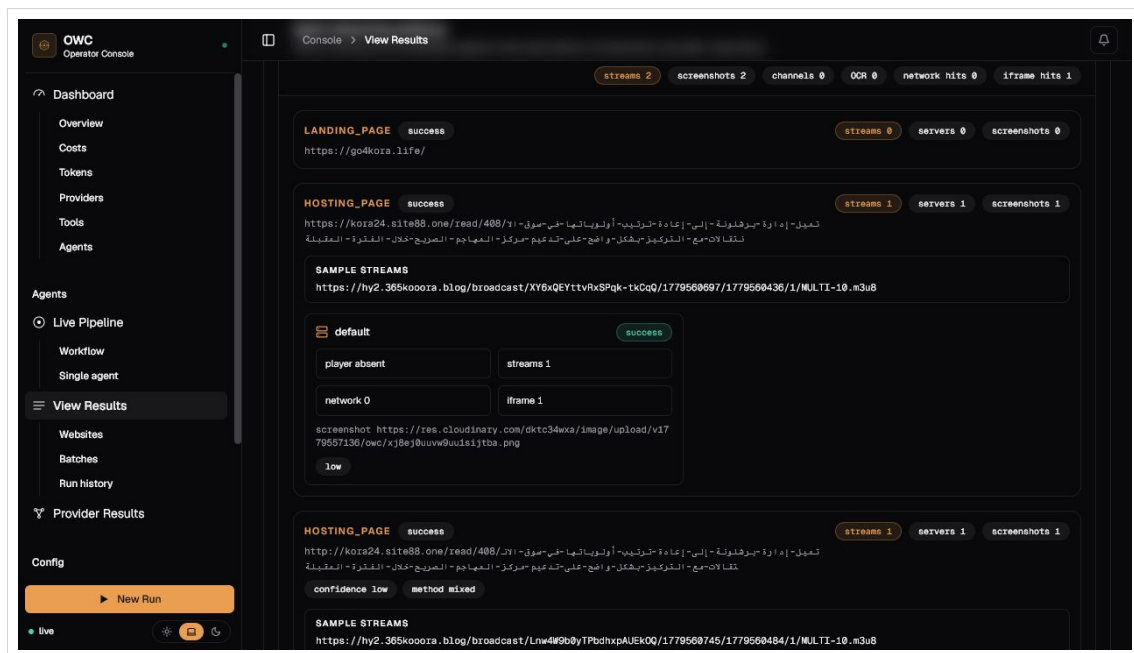


Figure 6.8: Streams and provider evidence for the selected run.

The provider-intelligence view shows the enrichment step. In this run, the playlist host `hy2.365kooora.blog` was resolved to BestDC Limited, organization AS216014 BestDC Limited, country NL, with an abuse-contact field. This turns stream discovery into provider-intelligence evidence.

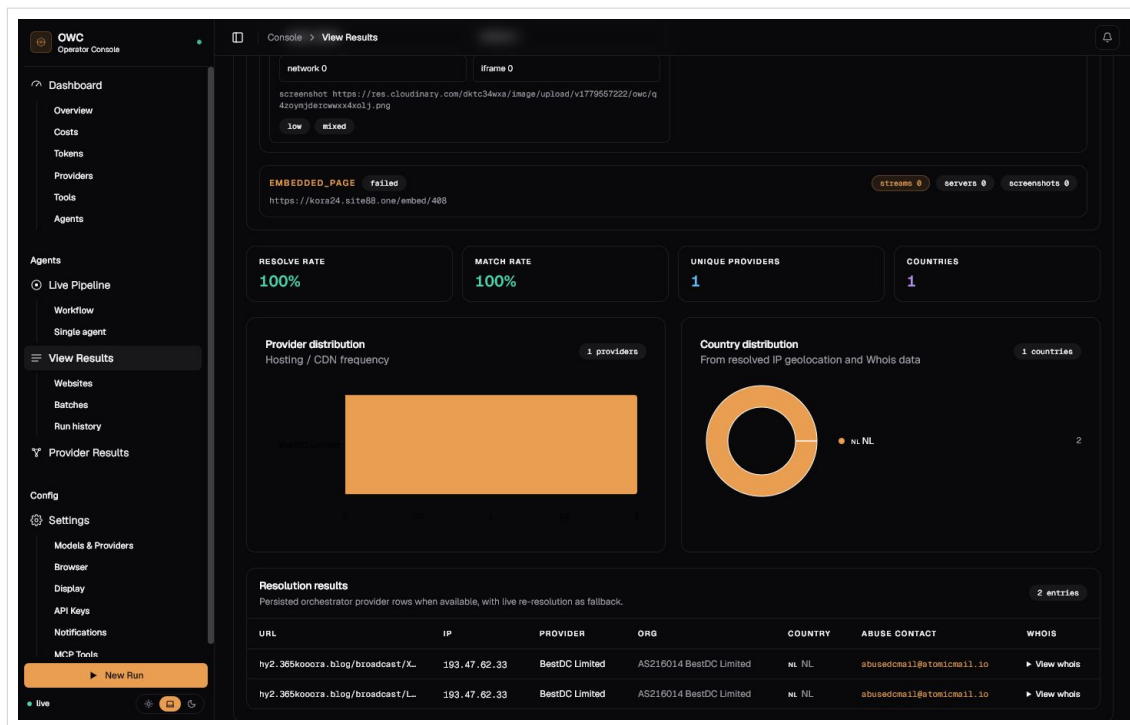


Figure 6.9: Provider-intelligence evidence for the selected run.

Generated email example from run 9566a9be-12ba-434a-afa0-f05f529d2403

Subject: DMCA Notice: unauthorized streaming hosted by BestDC Limited

Provider/contact: BestDC Limited, abusedcm1@atomicmail.io

Infringing page: <https://go4kora.life/>

Playlist host: `hy2.365kooora.blog`

Evidence fields:

- two m3u8 playlist URLs under `https://hy2.365kooora.blog/.../MULTI-10.m3u8`;
- protocol recorded as m3u8;
- server/source recorded as default;
- two screenshot evidence links attached to the stream records.

Review note: the email is an automatically prepared draft. A human reviewer must verify rights ownership, jurisdiction, contact accuracy, and wording before sending.

6.5 Dashboard Telemetry and Cost Review

6.5.1 Overview Metrics

The overview dashboard gives the operational context for the batch. It shows that OWC is measurable: runs, success rate, cost, token volume, latency, active runs, and tool success are visible from the same console. They are useful only when read through the failure taxonomy, not as one generic model-accuracy claim.

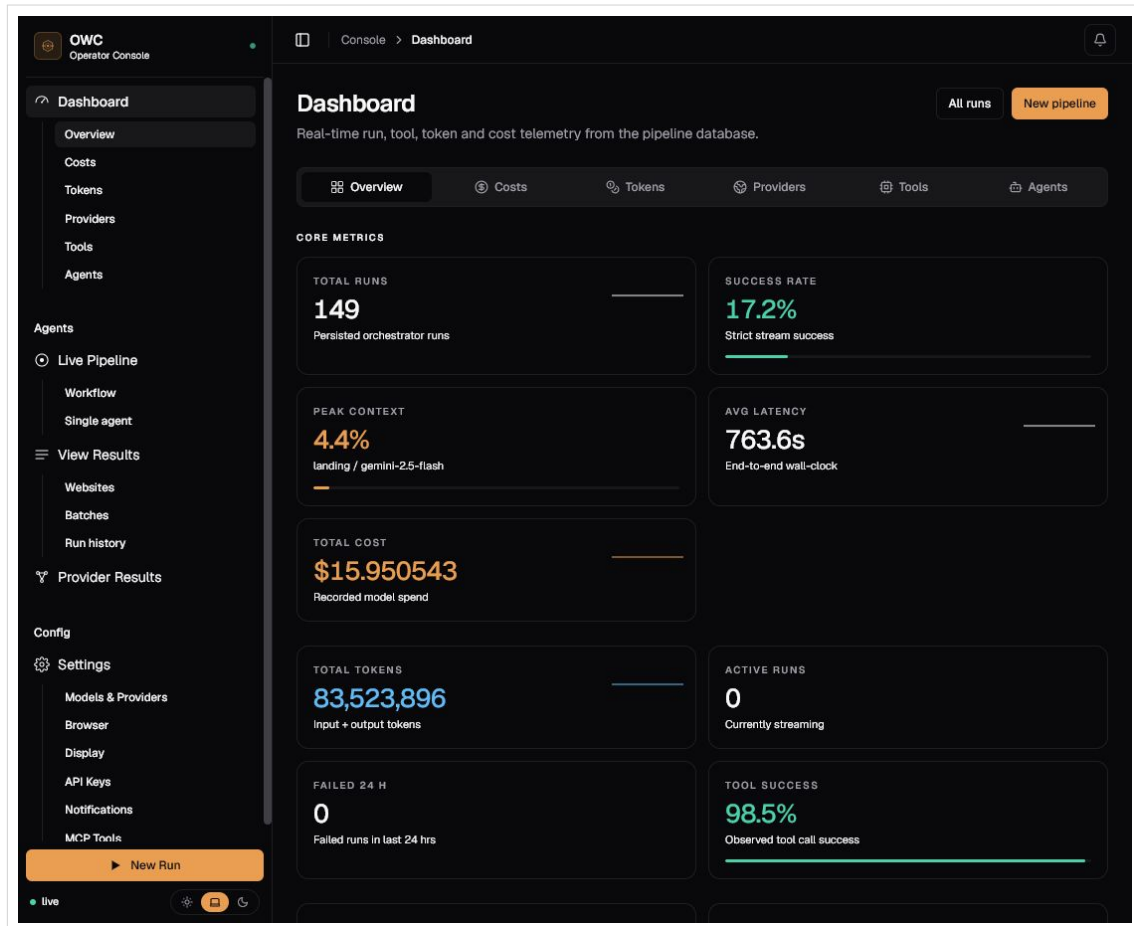


Figure 6.10: Overview dashboard metrics for Chapter 6.

Metric group	Current value	Interpretation
Evidence yield	25 runs with streams, 25 runs with emails, 136 total streams, 67 total emails.	Successful evidence production is tied to stream and email readiness, not only run completion.
Average yield	0.938 streams/run and 0.462 emails/run across terminal attempts.	Useful for batch-cost planning and expected review output.
Provider telemetry	136 provider analyses, 12 unique providers, 8.82% provider coverage.	Provider enrichment exists but remains incomplete across all attempts.
Tool reliability	3,113 observed tool calls, 3,066 successful, 47 failed, 98.49% success.	Browser-tool execution is mostly reliable; failures are often behavioral, site-specific, or availability-related.
Tool volume	3,209 total tool calls and 7.636 seconds average tool duration.	Agent work is interaction-heavy, especially for hosting pages.
Cost	\$15.950543 total model spend and \$0.110004 average cost per terminal run.	Cost is low enough for experiments but must be monitored for larger batches.
Tokens	83,523,896 total tokens, 54,252,590 cached input tokens, 110,760,668 new input tokens, and 1,017,267 output tokens.	These are dashboard telemetry cards used for cost review; the report does not recompute them into one billing formula.
Cache	32.9% cache hit ratio from cached and new input counters.	Repeated prompt and tool context reuse is visible but not dominant.

Table 6.7: Current evidence, provider, tool, cost, and token telemetry from the overview dashboard/API snapshot.

6.5.2 Token and Runtime Cost

Token telemetry is included because the project is also a data-science system evaluation. The cost of a browser agent depends on context size, cache reuse, output length, tool loops, and repeated attempts. Separating new input, cached input, and output tokens makes it possible to evaluate whether future prompt or tool changes are improving behavior without silently increasing cost.

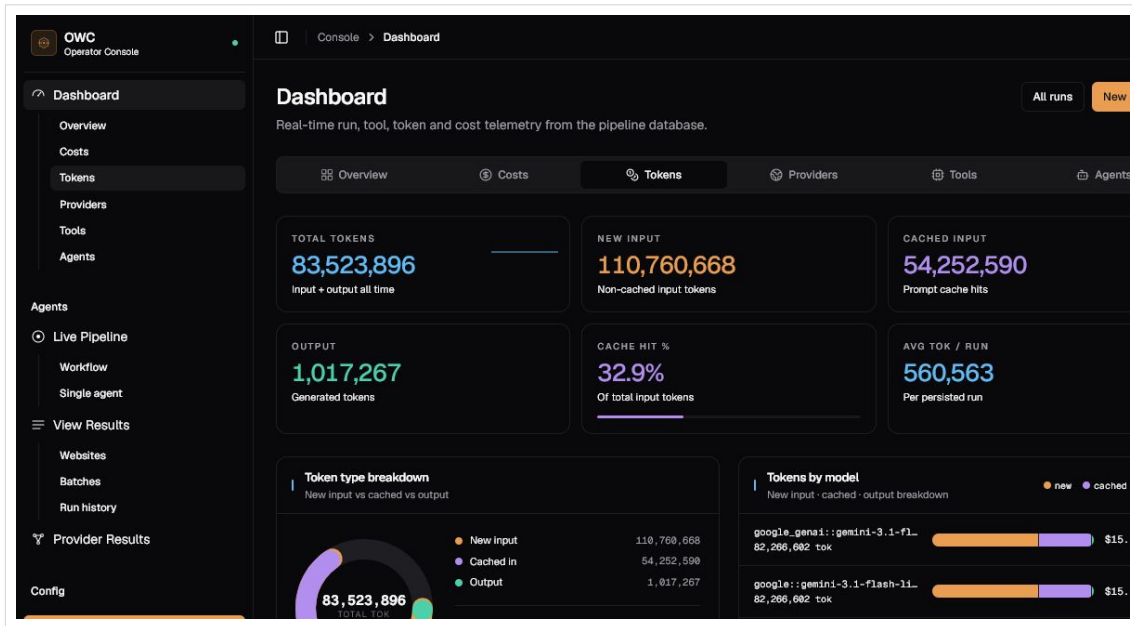


Figure 6.11: Token telemetry dashboard.

6.5.3 Model and Runtime Selection Rationale

The model evaluation is not presented as a universal model ranking. The relevant question is which runtime is practical for browser-agent evidence collection. The chosen runtime must support long browser context, disciplined tool use, visible pricing and cache counters, and usable rate-limit behavior. Gemini pricing documentation is cited as operating context for cost interpretation rather than as a model-quality proof [2].

Dimension	Why it matters for OWC	Evidence used here
Context handling	Browser runs contain page state, iframe paths, tool outputs, and trace history.	Selected run used 503,127 input tokens across 15 LLM calls.
Tool discipline	The model must inspect, navigate, activate, harvest, and verify in the right order.	Selected run recorded 25 tool calls and reached stream/provider evidence.
Latency and cost	Large regression batches become impractical if each run is too slow or expensive.	Selected run finished in about 5 minutes at \$0.12381188.
Cache visibility	Repeated prompts and tool descriptions should be observable as cached or new input.	Selected run recorded 72,995 cached input tokens.
Rate-limit practicality	Batch testing depends on token and request limits, not only one successful run.	Dashboard telemetry tracks call, token, and cost volume.
Failure interpretation	Useful models leave enough trace evidence to explain partial success or uncertainty.	The selected run kept trace, provider, screenshot, and email records.

Table 6.8: Model/runtime selection rationale used instead of unsupported model-ranking metrics.

6.6 Limitations and Interpretation

6.6.1 Operational Limitations

6.6.2 Interpretation of the Current Results

The current strict success rate is low, and that is a real limitation of the current system. However, the raw failure rate should not be interpreted as one number meaning “the agents failed”. The batch contains productive partials, external blockers, cancelled runs, and true agent/runtime failures. A data-science interpretation must keep those categories separate before comparing prompt versions, browser settings, or model runtimes.

The most useful conclusion is not that OWC has solved illegal-streaming evidence collection. It has not. The useful conclusion is that OWC now exposes the right evaluation objects: agent route, navigation path, server switching, screenshot timing, stream capture, provider enrichment, email readiness, repeatability, and cost.

Limitation	Effect on evaluation	Review action
Source site down	The seed URL can be inaccessible or dead before the agent can inspect it.	Record as a site blocker and rerun before drawing agent-quality conclusions.
Hosting page inaccessible	A landing page can list matches while the downstream hosting page is removed, blocked, or unreachable.	Treat <code>no_hosting_pages</code> separately from agent failures.
Embedded page inaccessible	A hosting page can expose an iframe URL that no longer loads or redirects away from the player.	Diagnose embedded availability separately from hosting behavior.
Server/player unavailable	Server buttons can point to inactive players or media paths with no stream.	Record server-switching order and screenshots before marking the case as no-stream.
Anti-bot drift	A website can change challenges, redirects, popups, or browser fingerprinting behavior after a successful run.	Treat traces and screenshots as time-bound evidence.
CDN/provider lookup gaps	Provider data can be missing, indirect, or routed through CDN and hosting intermediaries.	Use provider enrichment as triage and verify contacts before sending notices.
Partial-run telemetry	Strict success excludes partials, and some useful evidence can sit outside the headline status.	Report strict success, partial rate, and productive evidence separately.
Human/legal review	Screenshots, streams, and email drafts are evidence candidates, not legal determinations.	A trained reviewer must check rights context, jurisdiction, and notice wording.

Table 6.9: Limitations that remain after evidence-first evaluation.

6.7 Chapter Conclusion

This evaluation judges OWC by agent behavior and evidence quality. The selected run is successful because it provides a reconstructable path from:

- seed URL and route classification;
- landing navigation and hosting/player evidence;
- screenshots and HLS playlist evidence;
- provider enrichment, cost telemetry, and a generated email draft.

The broader batch snapshot shows that the system is measurable. It also shows that strict success remains low and many terminal rows require failure-category interpretation.

For a final-year data-science project, the main evaluation contribution is the measurement framework itself. Instead of hiding everything inside a generic accuracy score, the report evaluates OWC through strict success, partial evidence, external blockers, agent/runtime failures, repeatability, tool reliability, provider enrichment, email readiness, and operating cost.

General Conclusion

Chapter focus

The conclusion brings the internship contribution together. It restates the evidence collection problem, summarizes the n8n backup and OWC platform outcomes, identifies the main engineering contributions, explains the deployment choice, and closes with limitations and future work for the next iteration.

This PFE internship addressed a practical evidence-collection problem in the sports rights-protection domain. Soft Stars needed more than a list of suspicious URLs. It needed a way to preserve what happened in the browser and why the collected evidence could be reviewed later.

The main difficulty was that useful evidence rarely appears in the first HTML response. In practice, it often appears only after:

- the page finishes rendering JavaScript-heavy content;
- a channel group, match card, or server list is opened;
- a player is activated and the media state changes;
- an iframe or embedded page requests a short-lived media manifest.

The work followed Design Science Research Methodology as its methodological backbone. The problem was identified through the limits of static collection and manual review. The solution objectives then focused on classification, browser interaction, screenshots, stream evidence, channel metadata, provider enrichment, traceability, and human review.

The artifact was built in two steps:

- **n8n prototype:** four browser-agent workflows for classification, landing, hosting, and embedded-player extraction. Only the landing agent was deployed as the focused company backup for dynamic targets.
- **Open Web Catcher:** the complete platform built around FastAPI, Next.js, PostgreSQL, MCP browser tools, Puppeteer/Chrome, specialist agents, model telemetry, and reviewable evidence outputs.

The main engineering contribution is the separation of the investigation into clear page roles and agent responsibilities:

- the orchestrator controls the run, prepares handoffs, manages queues, and aggregates results;
- the classification agent decides whether the URL is a landing, hosting, embedded, or irrelevant page;
- the landing agent discovers match, event, channel, hosting, and embedded candidates;

- the hosting agent activates server and player controls while preserving screenshots and diagnostics;
- the embedded agent stays inside direct player contexts and harvests final media evidence.

Provider enrichment and takedown-email drafting remain downstream review stages. They are not automatic enforcement actions, which keeps the system aligned with the need for human verification.

OWC also contributes a more reliable browser-tool layer. Tool profiles separate broad inspection, precise context reads, navigation, interaction, screenshot capture, media-state checks, stream harvesting, and memory hints. Runtime controls then handle profile-scoped sessions, fingerprint behavior, proxy validation, sticky proxy selection, popup recovery, iframe recovery, and media verification.

This makes failures easier to understand. A failed run can be linked to a specific cause: classification uncertainty, missing hosting candidates, player activation failure, iframe drift, proxy failure, missing streams, or skipped provider analysis. It is no longer reduced to a vague failed scrape.

The implementation also produced a review-oriented operator experience. The console shows run status, traces, screenshots, streams, model usage, cached tokens, costs, tool calls, provider results, persistence health, and runtime events. The database supports that review by storing runs, agent runs, runtime events, LLM calls, tool calls, screenshots, streams, provider analyses, email drafts, prompt compilations, and memory hints.

The deployment decision remained pragmatic. The complete OWC platform defines the long-term architecture, while the operational deployment focuses on the validated n8n landing-agent backup:

- target URLs arrive through the `agent_tasks` Azure Service Bus queue;
- a focused container runs the landing-agent worker, Puppeteer MCP, and Chrome;
- matched URLs, metadata, screenshots, rejected URLs, and failure information are returned through the configured results queue.

This keeps the primary company workflow and the AI browser agent decoupled while still covering dynamic pages that need rendered-state reasoning.

The project remains a foundation rather than a finished enforcement platform. Some pages will still fail because of anti-bot behavior, CAPTCHA or challenge pages, unstable player code, server-switching variance, misleading channel labels, popup-heavy navigation, or unavailable providers. Human review is still necessary before evidence is used operationally, and larger controlled batches are needed before making strong accuracy or cost claims.

7.1 Perspectives

The next iteration should focus on concrete improvements:

- CAPTCHA and challenge-page handling, with clear blocked-state evidence instead of silent retries;
- larger labeled website batches and stronger regression checks;
- better reuse of successful selectors and server patterns without hardcoding individual sites;
- OCR-assisted screenshot review for channel labels, player overlays, and blocked pages;
- screenshot annotation and reviewer notes inside the operator console;
- provider-enrichment validation before email drafting, especially when CDN or intermediary hosts hide the true origin;
- gradual deployment from the landing-agent backup toward more queue-driven specialist agents when the operational risk and cost are acceptable.

Overall, the internship demonstrated that an AI-assisted browser workflow can complement static monitoring when it is constrained, observable, and designed around evidence. OWC does not claim to permanently solve every illegal streaming website. Its value is more practical. It gives Soft Stars a foundation for:

- classifying pages and following controlled handoffs;
- interacting with the browser and verifying player state;
- preserving traces and making the result understandable for human review.

Bibliography

- [1] Google, *Gemini API Rate Limits*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/rate-limits>
- [2] Google, *Gemini Developer API Pricing*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/pricing>
- [3] Google, *Context Caching in the Gemini API*, Google AI for Developers documentation, 2026. Available: <https://ai.google.dev/gemini-api/docs/caching>
- [4] Google Chrome Developers, *Puppeteer API Reference*, Official documentation, 2026. Available: <https://pptr.dev>
- [5] Microsoft, *Playwright Documentation*, Official documentation, 2026. Available: <https://playwright.dev/docs/intro>
- [6] Chrome DevTools Protocol, *Chrome DevTools Protocol: Network Domain*, Official documentation, 2026. Available: <https://chromedevtools.github.io/devtools-protocol/tot/Network/>
- [7] A. R. Hevner, S. T. March, J. Park, and S. Ram, *Design Science in Information Systems Research*, *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. Available: <https://www.jstor.org/stable/25148625>
- [8] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, *A Design Science Research Methodology for Information Systems Research*, *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. Available: DOI landing page
- [9] S. Ramírez, *FastAPI: Modern, Fast Web Framework for Building APIs with Python*, 2024. Available: <https://fastapi.tiangolo.com>
- [10] Vercel, *Next.js: The React Framework for the Web*, Vercel, 2024. Available: <https://nextjs.org/docs>
- [11] LangChain, *LangChain Agents and Tools Documentation*, Official documentation, 2026. Available: <https://docs.langchain.com/oss/python/langchain/agents>
- [12] LangChain, *LangGraph Overview*, Official documentation, 2026. Available: <https://docs.langchain.com/oss/python/langgraph/overview>
- [13] Model Context Protocol, *Model Context Protocol Specification*, Official documentation, 2026. Available: Model Context Protocol specification
- [14] n8n GmbH, *n8n: Workflow Automation for Technical People*, 2024. Available: <https://docs.n8n.io>
- [15] Microsoft, *Jobs in Azure Container Apps*, *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/container-apps/jobs>

- [16] Microsoft, *Azure Service Bus Messaging Service*, *Microsoft Learn*, 2024. Available: <https://learn.microsoft.com/en-us/azure/service-bus-messaging/>
- [17] Cloudflare, *How Cloudflare Challenges Work*, Official documentation, 2026. Available: <https://developers.cloudflare.com/cloudflare-challenges/concepts/how-challenges-work/>
- [18] R. Pantos and W. May, *HTTP Live Streaming*, RFC 8216, Internet Engineering Task Force (IETF), August 2017. Available: <https://www.rfc-editor.org/rfc/rfc8216>
- [19] ISO/IEC, *Information technology—Dynamic adaptive streaming over HTTP (DASH)—Part 1: Media presentation description and segment formats*, ISO/IEC 23009–1, 2019. Available: <https://www.iso.org/standard/79329.html>
- [20] R. Hill, *uBlock Origin: An Efficient Blocker for Chromium and Firefox*, GitHub, 2024. Available: <https://github.com/gorhill/uBlock>



ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889 114/114

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454